



**isec**  
**Engenharia**

LICENCIATURA EM ENGENHARIA  
ELETROTÉCNICA

**Desenvolvimento de Robô - MicroMouse  
para resolução de labirintos**

Autores

**Pedro Daniel Carvalho Rocha**

**José Manuel Dias Ferreira**

Orientadores

**Marco Silva**

**Nuno Ferreira**

**Pedro Amaro**

INSTITUTO POLITÉCNICO  
DE COIMBRA

INSTITUTO SUPERIOR  
DE ENGENHARIA  
DE COIMBRA

Coimbra, 17 de Setembro, 2021



## AGRADECIMENTOS

---

Quero deixar os meus agradecimentos à minha família e aos meus amigos pela força e orgulho que me transmitem.

Quero agradecer aos meus colegas e professores pela partilha de conhecimentos e experiência que me levaram a evoluir tanto nestes últimos anos.

Não podia de deixar de agradecer ao Peter Harisson pela sua dedicação à investigação e pela paixão na difusão da ciência em torno de robôs educacionais.

Pedro Daniel Rocha

Sobre o trabalho conseguido, os primeiros agradecimentos vão para a internet por estar aberta a qualquer hora, principalmente à noite, madrugada fora, mas que por sua vez também é responsável por me dispersar, pois tem muita matéria relacionada e alguma muito detalhada, e que, uma vez aguçada a curiosidade, só dei conta do tempo passado, depois do sol substituir a noite...

Agradeço a minha família pelo facto de tomarem conta do meu pequeno enquanto trabalhei neste projeto e ele não dormia. Ele assim, apenas me interrompeu com uma frequência um pouco inferior à do ESP32.

Aos professores orientadores, que nos deram os impulsos iniciais na escolha dos materiais, como adquiri-los, e depois como configurá-los e soldá-los, ajustá-los, explicá-los, enfim, até a vida deve ter uma explicação de forma a compreendê-la.

De modo genérico, a todos os professores que tive ao longo da licenciatura, porque de forma direta ou indireta, todas as disciplinas foram necessárias para trabalhar neste projeto, sendo que há matérias base e essenciais e professores que transmitem bem a sua “enciclopédia interna”.

Principalmente aos meus colegas do projeto, porque foram eles que me incentivaram a escolher este projeto, e também disponibilizaram todo o seu auxílio, assim como, são as pessoas a entender o porquê do muitíssimo tempo dedicado necessário a um projeto desta envergadura.

José Ferreira



## RESUMO

---

Esta dissertação descreve a implementação de um mini robô capaz circular num labirinto com células e corredores à sua escala, fazendo navegação até ao centro do labirinto no mínimo tempo possível. O seu objetivo principal será navegar autonomamente num labirinto desconhecido pelo robô, guiando-se apenas pelos seus sensores. O projeto firmou-se no The Amazing MicroMouse Contest<sup>1</sup>, uma competição geek de ratos robóticos em labirintos se tornou um fenómeno internacional. O robô inclui um chassi, rodas, drivers de motor e motores, sensores de parede integrados, microcontroladores e baterias. A sua construção envolve muitos desafios de design interdisciplinar.

O robô desenvolvido é um conjunto de subsistemas, nomeadamente base robótica, sistema de visão e perceção, sistema de navegação. Aqui desenvolvemos a escolha dos materiais, testes, protótipos, ensaios, PCB base, código para Arduino, e montagem.

O final encontra-se inacabado, mas o processo já passou grande parte das fases, faltando desenvolver o algoritmo de procura e ajustes das leituras dos sensores.

### **Palavras-Chave: ....**

Microrato; Controlo do movimento de robô; Programação de robôs autónomos; Sensores magnéticos (encoders); Sensores infravermelho e laser; Navegação em labirinto

---

<sup>1</sup> <https://spectrum.ieee.org/the-amazing-micromouse-contest>



## ABSTRACT

---

This dissertation describes the implementation of a mini robot capable of circulating in a maze with cells and corridors to its scale, navigating to the center of the maze in the shortest possible time. Your main objective will be to autonomously navigate a maze unknown by the robot, guided only by its sensors. The project established itself in The Amazing MicroMouse Contest , a geek competition of robotic mice in mazes has become an international phenomenon. The robot includes a chassis, wheels, motor and motor drivers, integrated wall sensors, microcontrollers and batteries. Its construction involves many interdisciplinary design challenges.

The developed robot is a set of subsystems, namely robotic base, vision and perception system, navigation system. Here we develop the choice of materials, tests, prototypes, tests, base PCB, Arduino code, and assembly.

The end is unfinished, but the process has already passed most of the steps, and the algorithm for searching and adjusting the readings of the sensors remains to be developed.

### **Keywords: ...**

Micromouse; Control of robot movement; Programming of autonomous robots; Magnetic sensors (encoders); Infrared and laser sensors; Labyrinth navigation



|                                      |      |
|--------------------------------------|------|
| AGRADECIMENTOS.....                  | i    |
| RESUMO .....                         | iii  |
| ABSTRACT .....                       | v    |
| ÍNDICE .....                         | vii  |
| ÍNDICE DE FIGURAS .....              | ix   |
| ÍNDICE DE TABELAS .....              | xi   |
| SIGLAS E ACRÓNIMOS.....              | xiii |
| 1. INTRODUÇÃO .....                  | 1    |
| 1.1. Motivação e objetivos .....     | 1    |
| 1.2. Estrutura deste documento ..... | 1    |
| 2. ESTADO DA ARTE .....              | 3    |
| 2.1. Micromouse Book.....            | 3    |
| 2.2. Introdução.....                 | 3    |
| 2.3. História.....                   | 3    |
| 2.4. Regras.....                     | 4    |
| 2.4.1. Regras dos MicroMouses .....  | 4    |
| 2.4.2. Regras da Competição .....    | 5    |
| 2.5. Labirintos e Resoluções .....   | 7    |
| 2.6. Navegação .....                 | 8    |
| 2.7. Comando e Controlo .....        | 9    |
| 2.8. Baterias.....                   | 10   |
| 2.9. Sensores.....                   | 10   |
| 2.10. Modelos Consagrados .....      | 11   |
| 3. PROJETO .....                     | 13   |
| 3.1. Conceitos.....                  | 13   |
| 3.1.1. Agilidade .....               | 13   |
| 3.1.2. Aderência .....               | 14   |
| 3.1.3. Aceleração .....              | 14   |
| 3.1.4. Travagem.....                 | 15   |
| 3.2. Componentes .....               | 16   |
| 3.2.1. Alimentação.....              | 16   |
| 3.2.1.1. Fonte de Energia .....      | 16   |
| 3.2.1.2. Níveis de Tensão .....      | 17   |
| 3.2.2. Atuadores .....               | 18   |
| 3.2.2.1. Rodas .....                 | 18   |
| 3.2.2.2. Motores.....                | 18   |

|          |                                           |    |
|----------|-------------------------------------------|----|
| 3.2.2.3. | Drivers .....                             | 20 |
| 3.2.3.   | Sensores.....                             | 22 |
| 3.2.3.1. | Encoders .....                            | 22 |
| 3.2.3.2. | LED IR e Foto-Transístores.....           | 23 |
| 3.2.3.3. | LIDAR.....                                | 24 |
| 3.2.4.   | Processador.....                          | 26 |
| 3.3.     | Design .....                              | 28 |
| 3.3.1.   | 3D CAD .....                              | 28 |
| 3.3.2.   | PCB.....                                  | 29 |
| 3.3.3.   | Protótipos .....                          | 31 |
| 4.       | RESULTADOS.....                           | 33 |
| 4.1.     | Sensores.....                             | 33 |
| 4.1.1.   | LIDAR .....                               | 33 |
| 4.1.2.   | Pares Emissor-Recetor Infravermelho ..... | 35 |
| 4.1.3.   | Encoders.....                             | 37 |
| 4.2.     | Atuadores.....                            | 38 |
| 4.2.1.   | Motores .....                             | 38 |
| 4.2.2.   | Resposta a Degrau .....                   | 39 |
| 4.2.3.   | Controlo.....                             | 41 |
| 5.       | IMPLEMENTAÇÃO .....                       | 43 |
| 5.1.     | Deteções de Paredes.....                  | 44 |
| 5.2.     | Movimentos do Robô.....                   | 47 |
| 5.3.     | Localização do MicroMouse .....           | 49 |
| 5.4.     | Navegação no Labirinto .....              | 50 |
| 5.5.     | Simulação de Algoritmos.....              | 50 |
| 6.       | CONCLUSÕES .....                          | 51 |
| 6.1.     | Conclusões Gerais.....                    | 51 |
| 6.2.     | Conclusões Específicas .....              | 51 |
| 6.3.     | Conclusões Finais .....                   | 51 |
|          | REFERÊNCIAS.....                          | 53 |
|          | ANEXOS.....                               | 55 |

## ÍNDICE DE FIGURAS

---

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Fig. 1 – EduMouse 1.5 Lunghwa Uni-Taiwan .....                              | 11 |
| Fig. 2 – RoboMaster Turbo - Tiajin Uni - China.....                         | 11 |
| Fig. 3 – Long Turbo V2.0 - Tianjin Uni – China .....                        | 11 |
| Fig. 4 – Diu Gow 5 - Xin Han Cai – Taiwan .....                             | 11 |
| Fig. 5 – Resposta transitória de um motor DC .....                          | 15 |
| Fig. 6 – Resposta transitória de um motor DC .....                          | 15 |
| Fig. 7 – Células de Lítio 16340 .....                                       | 16 |
| Fig. 8 – Suporte para Células CR123A .....                                  | 16 |
| Fig. 9 – L4940V5 .....                                                      | 17 |
| Fig. 10 – Condensadores Eletrolíticos .....                                 | 17 |
| Fig. 11 – Interruptor de 3A.....                                            | 17 |
| Fig. 12 – Interruptor 100mA .....                                           | 17 |
| Fig. 13 – Jantes Alumínio 17,2x12,5mm .....                                 | 18 |
| Fig. 14 – Pneus de Borracha .....                                           | 18 |
| Fig. 15 – HPCB 6V 30:1 .....                                                | 18 |
| Fig. 16 – HPCB vs HP .....                                                  | 18 |
| Fig. 17 – Suportes de Motor .....                                           | 18 |
| Fig. 18 – Gráfico de desempenho do motor a 6V.....                          | 19 |
| Fig. 19 – Principais Circuitos da ponte H .....                             | 20 |
| Fig. 20 – DRV8838 .....                                                     | 21 |
| Fig. 21 – Esquema de Aplicações do DRV8838 .....                            | 21 |
| Fig. 22 – Exemplos de PWM e respetivos $V_{med}$ .....                      | 21 |
| Fig. 23 – Encoder Magnético .....                                           | 22 |
| Fig. 24 - Diagrama Temporal e Estados de Operação do TLE4946-2K.....        | 22 |
| Fig. 25 – SFH 4550 .....                                                    | 23 |
| Fig. 26 – BPW85C .....                                                      | 23 |
| Fig. 27 – ULN2003ADR .....                                                  | 23 |
| Fig. 28 – Esquema de Ligação do Senso IR.....                               | 23 |
| Fig. 29 – Esquema Interno do Darlington Array .....                         | 23 |
| Fig. 30 – Princípio de funcionamento de um SONAR .....                      | 24 |
| Fig. 31 – Dados recolhidos por um LIDAR .....                               | 24 |
| Fig. 32 – VL53L3CX.....                                                     | 24 |
| Fig. 33 – Lidar ToF10120 .....                                              | 25 |
| Fig. 34 – Acelerómetro e Giroscópio MPU .....                               | 25 |
| Fig. 35 – MPU6050.....                                                      | 25 |
| Fig. 36 – ESP32 DevKitC .....                                               | 26 |
| Fig. 37 – PinOut Esp32 DevKit C .....                                       | 26 |
| Fig. 38 – Protótipo de MicroMouse desenvolvido no FreeCAD.....              | 28 |
| Fig. 39 – Esquemático da PCB do MicroMouse .....                            | 29 |
| Fig. 40 – Última versão do Projeto da PCB .....                             | 30 |
| Fig. 41 - Última versão da PCB em Processo de Montagem .....                | 30 |
| Fig. 42 – Placa de Ensaios dos Sensores.....                                | 33 |
| Fig. 43 – Esquema da placa do sensor VL53L3CX.....                          | 33 |
| Fig. 44 – Fluxo de aquisição de dados do VL53L0X .....                      | 34 |
| Fig. 45 – Ensaaios com ToF10120.....                                        | 35 |
| Fig. 46 – Polarização do LED .....                                          | 35 |
| Fig. 47 – Gráfico dos valores lidos no ADC do ESP32 .....                   | 36 |
| Fig. 48 – Divisor de Tensão na Entrada do ADC .....                         | 36 |
| Fig. 49 – Duração da Polarização do Foto-Transistor e da Conversão ADC..... | 37 |

|                                                                                      |    |
|--------------------------------------------------------------------------------------|----|
| Fig. 50 – Contagem de Pulsos de Encoder em Quadratura .....                          | 37 |
| Fig. 51 – Valores Cruos de Pulsos de Encoder com PWM de 300 e TimeStep de 6 ms ..... | 38 |
| Fig. 52 – Gráfico da velocidade aplicando filtro IIR com PWM=300 .....               | 39 |
| Fig. 53 – Gráfico de referência com vários coeficientes amortecimento $\zeta$ .....  | 39 |
| Fig. 54 – Gráfico de step response em função da velocidade .....                     | 40 |
| Fig. 55 – Gráfico de determinação da velocidade em função de PWM.....                | 41 |
| Fig. 56 – Diagrama de Controlo dos Motores .....                                     | 41 |
| Fig. 57 – Fluxo Base do Programa Desenvolvido .....                                  | 43 |
| Fig. 58 – Fluxo da Função Procurar Objetivo .....                                    | 44 |

## ÍNDICE DE TABELAS

---

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Tabela 1 – Pódio do APEC 2019 USA .....                            | 11 |
| Tabela 2 – Características do Motor Escolhido.....                 | 19 |
| Tabela 3 – Funcionamento Lógico do DRV8838.....                    | 21 |
| Tabela 4 – Versões Desenvolvidas de Protótipos do MicroMouse ..... | 31 |



## SIGLAS E ACRÔNIMOS

---

|                |                                                                                  |
|----------------|----------------------------------------------------------------------------------|
| <b>ADC</b>     | <i>analog-to-digital converter</i>                                               |
| <b>ADU</b>     | <i>Application Data Unit</i>                                                     |
| <b>API</b>     | <i>Application Programming Interface</i>                                         |
| <b>CAD</b>     | <i>Computer Aided Design</i>                                                     |
| <b>DC</b>      | <i>direct current)</i>                                                           |
| <b>DWG</b>     | <i>extensão de arquivos de desenho em 2D e 3D nativa do software AutoCAD</i>     |
| <b>EPROM</b>   | <i>Erasable Programmable Read-Only Memory</i>                                    |
| <b>GPIO</b>    | <i>General Purpose Input/Output</i>                                              |
| <b>I2C</b>     | <i>Inter-Integrated Circuit</i>                                                  |
| <b>IDE</b>     | <i>Integrated Development Environment</i>                                        |
| <b>IEEE</b>    | <i>Instituto de Engenheiros Eletricistas e Eletrônicos</i>                       |
| <b>IIR</b>     | <i>Infinite Impulse Response</i>                                                 |
| <b>IoT</b>     | <i>Internet of Things</i>                                                        |
| <b>IR</b>      | <i>Infra Red</i>                                                                 |
| <b>LED</b>     | <i>light-emitting diode)</i>                                                     |
| <b>LIDAR</b>   | <i>Light Detection And Ranging</i>                                               |
| <b>MCU</b>     | <i>MicroController Unit</i>                                                      |
| <b>MIT</b>     | <i>Massachusetts Institute of Technology</i>                                     |
| <b>NPN</b>     | <i>Sequência NP-PN formado por três cristais de silício, sendo dois N e um P</i> |
| <b>PCB</b>     | <i>Printed Circuit Board</i>                                                     |
| <b>PWM</b>     | <i>Pulse Width Modulation</i>                                                    |
| <b>RADAR</b>   | <i>Radio Detection And Ranging</i>                                               |
| <b>RAM</b>     | <i>Random Access Memory</i>                                                      |
| <b>ROM</b>     | <i>Read Only Memory</i>                                                          |
| <b>RTC</b>     | <i>Real Time Clock</i>                                                           |
| <b>RX-TX</b>   | <i>Transmit/TX mode to Receive/RX mode</i>                                       |
| <b>SDA-SCL</b> | <i>SDA – Serial DATA; SCL – Serial Clock</i>                                     |
| <b>SONAR</b>   | <i>Sound Navigation And Ranging</i>                                              |
| <b>SPI</b>     | <i>Serial Peripheral Interface</i>                                               |
| <b>SRAM</b>    | <i>Static Random Access Memory</i>                                               |
| <b>ToF</b>     | <i>Time of Light</i>                                                             |
| <b>UART</b>    | <i>Universal Asynchronous Receiver-Transmitter</i>                               |
| <b>USB</b>     | <i>Universal Serial Bus</i>                                                      |
| <b>XSHUT</b>   | <i>pin low puts the sensor into hardware standby</i>                             |



## 1. INTRODUÇÃO

Este documento visa apresentar o trabalho desenvolvido ao longo do último semestre da licenciatura em engenharia eletrotécnica no âmbito da cadeira de projeto de automação.

O projeto elaborado consiste na criação de um robô caracterizado como micromouse, robô esse habilitado a participar em competições robóticas internacionais.

O espírito competitivo em torno dos concursos de robôs impõe uma motivação adicional e uma ambição sem limites devido ao inalcançável nível de excelência.

A área da robótica combina princípios de física, eletrónica, programação e controlo de sistemas. Permitindo assim a consolidação dos conhecimentos adquiridos ao longo do curso.

### 1.1. Motivação e objetivos

Motivados pela procura de conhecimento e perceção, tanto das últimas descobertas tecnológicas, como dos princípios e fundamentos que as possibilitaram, surgiu a possibilidade de nos desafiar a nós próprios, através da elaboração de um projeto prático e estabelecendo os seguintes objetivos:

- ✓ Análise de ofertas de mercado, elegendo componentes que permitam alcançar uma solução económica, maximizando a qualidade de desempenho.
- ✓ Familiarização com softwares de CAD (desenho assistido por computador), elaborando peças personalizadas e placas de circuito impresso (PCB).
- ✓ Aquisição de competência prática através da instalação de componentes eletrónicos.
- ✓ Desenvolvimento de um programa de raiz em linguagem C com o Arduino IDE capaz de comandar, controlar, analisar e decidir o rumo a seguir pelo robô num labirinto desconhecido, reconhecendo a rota mais eficaz.
- ✓ Produzir soluções através da análise dos possíveis problemas emergentes, fomentando o espírito crítico.

### 1.2. Estrutura deste documento

O documento está dividido em 6 capítulos, distribuindo a informação recolhida, o trabalho desenvolvido, e os resultados obtidos da seguinte forma:

- Capítulo 1 – INTRODUÇÃO – Contextualiza o âmbito do projeto;
- Capítulo 2 – ESTADO DA ARTE – Apresentada os avanços alcançados atualmente;
- Capítulo 3 – PROJETO – Clarifica a preparação e a construção do modelo real;
- Capítulo 4 – RESULTADOS – Expõe o funcionamento isolado dos vários constituintes;
- Capítulo 5 – IMPLEMENTAÇÃO – Aborda a simbiose entre os sistemas do conjunto;
- Capítulo 6 – CONCLUSÕES – Sumariza os feitos alcançados e futuras perspetivas;



## 2. ESTADO DA ARTE

Este capítulo apresenta informação relevante acerca do popular robô navegador de labirintos “micro mouse”, referindo a sua origem e percurso, passando pelos concursos oficiais e respetivas regras, e incluindo abordagens utilizadas e feitos alcançados por indivíduos consagrados na comunidade micromouse.

### 2.1. Micromouse Book

O Micromouse Book<sup>2</sup> reúne uma variedade de informações e descobertas que Peter Harrison, professor inglês, e múltiplas vezes campeão de concursos micromouse, realizou ao longo das últimas décadas. Um resumo de carater introdutório será apresentado de seguida.

### 2.2. Introdução

Basicamente, existe um labirinto, base em madeira pintada de preto e paredes com peças geralmente em plástico branco, capazes de dividir a área labiríntica num padrão com células de dimensão 16cm x 16cm. Numa primeira fase, ir á descoberta, os robôs devem encontrar o caminho de uma posição inicial predeterminada (genericamente num dos 4 cantos) sendo a meta, as células centrais do labirinto sem ajuda humana. O mouse precisa de perceber onde está, detetar paredes e aberturas entre células enquanto explora, mapear o labirinto por onde passa e identificar quando atingiu o objetivo.

Na segunda fase, o robô vencedor é o que conseguir mais rapidamente partir do início pré-estabelecido e quase que “voar” pelo caminho que lhe permita mais rapidamente chegar a meta. Existem muitas versões das regras completas on-line e uma série de pequenas variações sobre como a pontuação do mouse é determinada.

Grande parte das informações contidas nestas páginas foi adaptada dos artigos publicados por Dave Otten na Circuit Cellar INK junho / julho de 1990 e agosto / setembro de 1990.

### 2.3. História

A competição de micromouse existe desde o final dos anos 1970 em todo o mundo, tendo evoluído uma forma moderna da competição em 1980 ou depois.

**1977** - A revista IEEE Spectrum introduziu o conceito de micromouse. Em maio de 1977, a Spectrum anunciou o 'Amazing Micromouse Competition' que aconteceria em 1979 em Nova York. Havia 15 concorrentes finais no meio de 6.000 inscrições iniciais. Esta competição obrigava os robôs a descobrir o caminho para fora de um labirinto de 10 por 10. Quando a competição foi realizada, o vencedor foi um básico seguidor de parede em alta velocidade.

**1980** - O professor John Billinsley, da Portsmouth Polytechnic, modificou as regras e introduziu a primeira competição europeia - realizada em Londres, no Euromicro. as mudanças nas regras exigiam que os ratos encontrassem uma meta no centro do labirinto e os seguidores da parede poderiam ser impedidos de encontrar a meta. Houve 200 pesquisas e 100 inscrições, mas apenas 9 ratos nas finais. O Sterling Mouse de Nick Smith tornou-se o primeiro (e naquele ano o único) micromouse a encontrar

---

<sup>2</sup> <http://www.micromouseonline.com/micromouse-book/>

o centro e saber que o tinha feito. Embora o desempenho tenha sido menos do que impressionante em cerca de 0,18 m/s, foi e ainda é um feito notável.

1981 - Na Exposição Micro Expo em Paris houve um calor de competição de micro-casa cinco ratos estavam presentes. O primeiro lugar foi para Nick Smith com Sterling Mouse, alcançando o centro em menos de 3 minutos e meio.

1982 - As finais britânicas da competição Euromouse '82 foram realizadas na feira Earls Court Computer em abril. De um leque de sete finalistas, Alan Dibley conquistou o primeiro e o segundo lugares. O T3 de Alan, o terceiro da família Thezeus, venceu com o melhor tempo de 1min 13seg enquanto o Filho de Thezeus conseguiu o melhor tempo de 3min 21seg.

1985 – O 'First World Micromouse Competiton' foi realizado em Tsukuba, Japão. Os labirintos foram enviados a vários países a fim de incentivar as inscrições. O campeão mundial foi Noriko-1 do Japão. Os seis primeiros lugares foram ocupados por concorrentes japoneses. O sétimo foi Dave Woodfield da Inglaterra com a Enterprise.

1986 – Os Estados Unidos tiveram sua primeira competição, realizada em Atlantic City, organizada, pelo IEE. Dave Otten do MIT teve sua primeira inscrição em competição com Mitee Mouse I. Focou em último.

1987 – O IEE World Micromouse Championship em Londres presenciou 13 micromouses em competição. O vencedor foi Dave Otten, que conseguiu ganhar o primeiro e o segundo prémios com Mitee Mouse I e Mitee Mouse II. Este foi também o ano do primeiro concurso de Singapura. o vencedor deste concurso foi MIR3 + do Instituto Tecnológico de Nanyang. Este mouse ficou em terceiro lugar na competição IEE UK de 1988, em Londres.

## **2.4. Regras**

Existem pequenas variações regionais das regras para as competições. As regras relativas ao labirinto são abordadas nas páginas do labirinto. As restantes das regras dizem respeito ao próprio micromouse ou ao funcionamento da competição e são relatadas de acordo com UK Micromouse Maze Solver Rules, 2000.

### **2.4.1. Regras dos MicroMouses**

- Embora a estrutura dos mouses possa ultrapassar em altura as paredes do labirinto, não havendo limite nesta dimensão, estão sujeitos a restrições de tamanho, 25cm de largura e 25cm comprimento.
- Os mouses devem ser totalmente autossuficientes e não devem receber assistência externa.
- Todos os micromouse devem ser equipados com um gancho ou argola adequado para levantar o rato do centro do labirinto, caso seja necessário.
- O método de deteção de paredes fica ao critério dos seus construtores, no entanto, o mouse não deve exercer força em nenhuma parede porque pode causar danos.

- O método de propulsão fica a critério dos construtores, desde que a fonte de energia não seja poluente - os motores de combustão interna provavelmente seriam desclassificados nesta cláusula.
- Se os juízes considerarem que um mouse tem alto risco de danificar ou manchar o labirinto, podem impedir a participação. Nada pode ser depositado no labirinto.
- O mouse deve contornar o labirinto, não pode saltar, escalar, arranhar, danificar ou destruir as paredes do labirinto.

### 2.4.2. Regras da Competição

O tempo gasto para viajar da célula de partida até ao destino é chamado de tempo de 'corrida'. Viajar do destino para voltar ao início não é considerado uma corrida.

O tempo total decorrido desde a primeira ativação do micromouse até o início de cada corrida também é medido. É chamado tempo de 'labirinto' ou 'pesquisa'.

Cada mouse tem um máximo de 10 minutos para correr, mas pode ser reduzido para 6 minutos se houver muitos mouses com alto desempenho.

Os juízes têm o poder de solicitar que um mouse se retire mais cedo se, por falta de progresso, tornar-se enfadonho, ou se por comportamento errático colocar em risco o estado do labirinto.

Se o micromouse necessitar de qualquer assistência manual a qualquer momento durante do concurso, será considerado 'tocado'. A pontuação é obtida calculando o tempo com penalizações para cada corrida da seguinte maneira:

Pontuação de tempo com penalizações = tempo da volta + penalidade de pesquisa + penalidade de toque, onde:

Penalidade de pesquisa =  $1/30$  do labirinto ou tempo de pesquisa, em segundos, associado a essa execução.

Penalidade de toque = 3 segundos mais  $1/10$  do tempo de execução, em segundos, se o mouse tiver sido tocado em qualquer momento antes de a corrida.

Por exemplo, se um mouse, depois de estar no labirinto por 4 minutos sem ser tocado, inicia uma corrida que leva 20 segundos, a corrida terá uma pontuação de tempo com penalizações de  $20 + 1/30 (4 \times 60) = 28$  segundos. No entanto, se o mouse for tocado antes da corrida, tem penalidade adicionada de toque igual a  $(3 + (1/10 \times 20))$  segundos, resultando uma pontuação de tempo com penalizações de 33 segundos.

Quando o mouse chega a célula de destino, pode parar e aí permanecer, ou então continuar a explorar outras partes do labirinto, ou fazer o caminho de volta ao ponto de partida. Se o mouse optar por parar no centro, pode ser levantado manualmente e reiniciado pelo manipulador. Levantar manualmente será considerado tocar o mouse e causará uma penalidade de toque a ser adicionada em todas as corridas subsequentes. Se o mouse não optar por permanecer na célula de destino, não pode ser interrompido manualmente e reiniciado.

O tempo para cada corrida (tempo de execução) deve ser medido a partir do momento em que o mouse deixa o início até chegar ao destino. O tempo total no labirinto (labirinto ou tempo de busca) deve ser medido a partir do momento em que o mouse é ativado pela primeira vez.

O tempo gasto para percorrer o labirinto deve ser medido manualmente pelos oficiais da competição, ou por sensores ajustados na largada e no destino. Se forem usados sensores infravermelhos, o sensor de partida deve ser posicionado no limite entre a célula inicial e a sua adjacente unitária. O feixe infravermelho de cada sensor deve ser horizontal e posicionado aproximadamente 1 cm acima do pavimento.

O procedimento de inicialização do mouse deve ser simples e não deve oferecer escolhas de estratégias ao manipulador. Por exemplo, a decisão de fazer uma corrida rápida para o centro à medida que o tempo se esgota deve ser tomada pelo próprio mouse. O procedimento de partida deve ser apresentado aos juízes quando o mouse for registado no dia da competição.

O manipulador do mouse tem 1 minuto, a partir do momento em que o mouse é retirado da gaiola, para fazer quaisquer ajustes ou limpezas nos sensores ou rodas do mouse. No entanto, nenhuma seleção de estratégias deve ser feita e nenhuma informação sobre a configuração do labirinto deve ser inserida ou capturada na memória.

A contagem do tempo de busca começará depois de expirar o limite de tempo de 1 minuto, mesmo se o manipulador ainda estiver a fazer ajustes no mouse.

Se um mouse 'tiver problemas', os manipuladores podem pedir permissão ao juiz para abandonar a execução e reiniciá-lo do início. Um mouse não pode ser reiniciado simplesmente porque deu uma volta errada - a decisão dos juízes é final. Os juízes podem adicionar uma penalidade de tempo para o reinício.

Se qualquer parte de um mouse for substituída durante o seu desempenho - como baterias ou EPROMs – ou se qualquer ajuste significativo for feito, a memória do labirinto deve ser apagada antes de reiniciá-lo. Pequenas manipulações de sensores poderão ser consentidas, mas a operação de controles de velocidade ou estratégia é expressamente proibida sem antes apagar a memória. Presume-se que os mouses terão software armazenado em EPROMs. No entanto, está a consideração dos juízes, se os mouses com memória RAM e com bateria de reserva, podem fazer download do software de controle se essa memória for apagada acidentalmente durante uma corrida. Os manipuladores, neste caso, devem demonstrar aos juízes que apenas carregaram o software original.

Se nenhuma corrida bem-sucedida foi conseguida, o juiz fará uma avaliação qualitativa do desempenho do mouse, com base na distância alcançada, 'determinação' versus comportamento aleatório e qualidade do controlo.

Se um mouse decidir parar devido a problemas técnicos, os juízes podem permitir que ele execute novamente mais tarde. O mouse será considerado como tendo levado três minutos extras em busca (ou seja, se um mouse parar depois de quatro minutos, quando recomeçar, o cronometro começa no sétimo minuto, restando apenas três minutos para acabar). É provável que esta permissão seja cancelada se o campeonato estiver repleto ou atrasado.

Os prémios são selecionados pelos jurados, e além dos prémios principais, podem incluir prémios para classes específicas dos mouses, como para o menor custo, o mais engenhoso, a melhor apresentação e o mais divertido.

Antes da revelação do labirinto, os mouses devem estar fechados numa caixa pelos oficiais do concurso. Os manipuladores apenas podem colocar os mouses na célula de partida sob instruções dos oficiais.

Em circunstâncias normais, nenhuma parte do mouse pode ser transferida para outro mouse. No entanto, os juízes podem permitir a troca de baterias ou controlador em casos excepcionais. Portanto, se um chassi for usado com dois controladores diferentes, serão considerados o mesmo e deverão funcionar numa única alocação de tempo de 10 minutos. A memória deve ser apagada com a mudança do controlador.

## 2.5. Labirintos e Resoluções

O labirinto usado para micromouse é descrito nas regras. Para conveniência, resume-se a descrição completa do labirinto das regras do IEE.

O labirinto deve ser uma matriz quadrada de 16 x 16 de quadrados unitários de 180 mm x 180 mm.

As paredes que constituem o labirinto devem ter 50 mm de altura e 12 mm de espessura.

As passagens entre as paredes devem ter 168 mm de largura.

A parede externa deve envolver todo o labirinto.

Os lados do labirinto devem ser brancos e o topo das paredes deve ser vermelho.

O piso do labirinto deve ser de madeira e acabamento com tinta preta sem brilho.

O revestimento na parte superior e nas laterais das paredes deve ser selecionado para refletir a luz infravermelha e o revestimento do piso deve absorvê-la.

O início do labirinto deve estar localizado em um dos quatro cantos.

O quadrado inicial deve ter paredes em três lados.

A orientação do quadrado inicial deve ser tal que, quando a parede aberta está para o "Norte", as paredes externas do labirinto devem estar no "Oeste" e "Sul".

No centro do labirinto deve haver uma grande abertura que é composta por 4 quadrados unitários. Esta praça central será o destino.

Um poste vermelho, com 200 mm de altura e 25 mm de cada lado, pode ser colocado no centro do grande quadrado de destino, se solicitado pelo manipulador.

Pequenos postes quadrados, cada um com 12 mm x 12 mm x 50 mm de altura, nos quatro cantos de cada unidade são chamados de pontos de rede.

O labirinto deve ser constituído de forma que haja pelo menos uma parede tocando cada ponto da treliça, exceto para o quadrado de destino.

As dimensões do labirinto devem ser precisas entre 5% ou 20 mm, o que for menor.

As juntas de montagem no piso do labirinto não devem envolver degraus maiores que 0,5 mm.

A mudança na inclinação numa junta de montagem não deve ser maior que 4 graus.

As lacunas entre as paredes de quadrados adjacentes não devem ser maiores que 1 mm.

## 2.6. Navegação

A competição é uma corrida em contrarrelógio. Objetivo é explorar o labirinto, encontrar a rota ideal para as células de destino e, em seguida, correr do início até ao destino o mais rápido possível. São fases com desafios diferentes para o mouse. Durante a exploração, deve ser relativamente cuidadoso, não pode perder a localização em nenhum momento. A velocidade do mouse provavelmente será menor e a atividade adicional de verificação da parede ajudará a garantir que não se perde os dados da parede devido às variações na refletividade das paredes. A calibração adicional em tempo real dos sensores pode ser possível.

Depois de encontrar o centro, decidimos se ele continua a explorar na expectativa de encontrar uma rota melhor ou correr, usando com as rotas já adquiridas. Um design de labirinto engenhoso pode ser penalizador se não explorar mais.

Com as rotas adquiridas, é executar uma corrida de velocidade. Supondo que as regras sejam compensadoras por uma corrida mais rápida, muitos mouses fazem várias corridas. Uma fase rápida do início ao centro deve ser seguida por uma corrida relativamente segura de volta ao início.

Quão rápido o mouse tem que ser?

No 17º All Japan Contest em novembro de 1996, o labirinto tinha uma rota para o centro com cerca de 75 células de comprimento e muitas retas. Para ser competitivo, era necessário executá-lo em menos de 15 segundos, o que dá uma velocidade média menor que 1m/s. Com curvas e curvas de aceleração, é necessário uma atingir velocidades cerca de 1,5 m/s ou mais, equivalente a 1,5mm por milissegundo ou 120ms por cada célula. Dentro de cada célula, é necessário realizar correções de direção com uma frequência suficiente para permanecer na linha reta e estreita. Isso deve estar nos limites das capacidades de qualquer processador moderno.

Por mais tentador pensar que a velocidade é essencial, não adianta rasgar o caminho se não se souber uma rota certa. Como não se vence sem chegar ao centro do labirinto, é este o primeiro desafio a resolver com a devida tranquilidade, sabedoria e engenharia, para depois fazê-lo rapidamente. Dito isto, convém garantir que o projeto seja capaz de ser vencedor com bom desempenho, depois de resolvida a tarefa mais crítica de achar o melhor caminho.

Muitos dos fatores a serem considerados em relação ao desempenho dinâmico do micromouse, o atrito, especialmente entre a roda e a superfície do labirinto de madeira prensada ou aglomerado, pintada de preto, é muito crítico para a capacidade de acelerar, travar e fazer curvas de maneira confiável. Embora existam regras para o labirinto, não pode considerar de a superfície seja lisa, limpa ou particularmente aderente para os pneus, e sempre uniforme em todas as células.

O coeficiente de atrito,  $\mu$ , é um número, geralmente entre 0 e 1, que descreve quanta resistência existe entre duas superfícies. O atrito estático é a força que impede o resvalamento das coisas. O atrito cinético é a força que se opõe ao resvalo depois de iniciado. O atrito cinético é sempre menor do que o atrito estático. Assim, quando se empurra uma caixa pelo chão, é mais difícil iniciar o movimento do que mantê-la em movimento. Por este motivo, depois que o objeto comece a deslizar, ele tende a deslizar continuamente. Um valor típico  $\mu$  para borracha em madeira pode ser cerca de 0,7.

Outro conceito importante é o de inércia ou momento linear. Os corpos tendem a resistir à mudança. Mesmo no espaço, sem atrito, é necessário esforço para fazer com que os corpos se movam e esforço

para pará-los quando se encontram em movimento, tanto no movimento linear, assim como também é necessário esforço para iniciar e parar os corpos a girar.

Parece que a física está de “costas voltadas” e as condições mudam constantemente a cada milímetro, e de certeza que fazer curvas é pelo menos tão difícil quanto ir em linha reta.

## 2.7. Comando e Controlo

Qualquer projeto como o micromouse é uma troca entre hardware e software. No entanto, uma coisa parece ser universalmente verdadeira: mesmo com a maneira mais fácil de fazer qualquer processo, ele só funcionará bem se despende uma grande quantidade de tempo, a quantidade suficiente.

A nível de controlo, as tarefas difíceis, poderiam ser o controle básico do motor, ou a tarefa mais difícil seja apenas correr em linha reta, ou parar onde quiser. A solução do labirinto e a navegação podem parecer mais difíceis ainda, mas existem algumas soluções bem conhecidas que podem ser testadas em simulação.

Bart Provo sugeriu uma sequência de desenvolvimento num artigo de jornal em 1990:

- Controlo do motor e deteção de velocidade
- Controlo da distância percorrida
- Controle de rotação e acompanhamento do sentido
- Movimento básico de célula para célula
- Malha aberta em velocidade
- Sensor de parede para controlo de direção
- Parede quadrada unitária e deteção e relatório de saída
- Circuito fechado dos encoders e verificação pelo acompanhamento de paredes, rotação e comportamento em beco sem saída

Com software de forma modular e adequada, o que foi descrito acima formará uma camada organizada que pode ser chamada a partir de rotinas relacionadas ao labirinto de nível superior.

Essencialmente, é necessário um conjunto de peças ligadas entre si, que consigam controlar com facilidade e precisão em termos a aceleração, velocidade e posição no mínimo de 2 rodas. Deve ser leve, pequeno e usar o mínimo de energia possível. Com um mínimo de componentes eletrônicos auxiliares e de preferência, fácil de montar e barato.

Os motores de passo são pesados e consomem muita energia, não são bons para grandes velocidades e necessitam de componentes eletrônicos para os controlar relativamente pesados.

Os motores DC adaptados com caixas de redutoras adequadas podem ser pequenos e leves, a eletrônica é simples e os requisitos de energia bastante pequenos. Para resolver o problema de controlo de maneira conveniente, é necessário software inteligente e um processador que possa fazer contas (multiplicações rápidas).

## 2.8. Baterias

Para fonte de energia, as baterias em células recarregáveis são a melhor escolha. Elas são mais caras inicialmente, mas rapidamente compensa após alguns ciclos de carregamento.

O tamanho é importante, as células maiores duram mais e podem ter tensões mais altas. Embora o processador precise apenas de uma fonte de 5 volts e umas dezenas de miliamperes, os motores e sensores exigem maiores tensões e consumos. Ter energia suficiente para a prova (15 minutos) é essencial contar com o(s) regulador(es) tensão a dissipá-la em forma de calor.

Usando motores DC, eles funcionam bem com 6 volts, podendo reduzir-se a 4 células de 1,5V e utilizando apenas umas centenas de miliamperes, correspondendo a uma economia de peso e melhor desempenho.

## 2.9. Sensores

Qualquer robô precisa de sensores para “sentir” o ambiente envolvente.

Os motores DC adaptam-se bem aos sistemas de controlo em malha fechada utilizando odometria, que relaciona a distância com a velocidade que o robô percorreu.

Os sensores de parede são usados para detetar a presença ou ausência de paredes e verificar sua posição no labirinto e também para certificar que o mouse mantém um caminho sem bater nas paredes. Para estes sensores, o mais importante é ter boa repetibilidade do que precisão absoluta, pois que o essencial é evitar bater no labirinto.

Importante é colocar os sensores de paredes o mais direcionado possível às paredes ou no centro de rotação do mouse. E quanto maior for a distância, melhor será a capacidade de manter um curso direito centrado nas células do labirinto.

Seria bom que os sensores fiquem muito tempo ativos, mas pode não ser adequado deixá-los sempre ativos com alta frequência porque além de não beneficiar de redução de interferência, eles são um grande consumo de energia.

Os sensores virados para a frente são essenciais para permitir detetar paredes antes de lhes bater. É necessário controlar a deteção da posição para a frente e a velocidade para que, na fase de exploração, não ande tão rápido ao ponto de não conseguir parar quando uma parede for detetada. Os sensores dianteiros também fornecem uma referência posicional conhecida que deve representar um múltiplo de 180 mm a partir do último ponto de partida, independentemente do que os encoders exponham.

Uma vantagem de ter dois conjuntos de sensores para a frente, podem para detetar erros de deslocamento.

## 2.10. Modelos Consagrados

Os modelos dos micromouse que colecionam prêmios nas competições por esse mundo fora são o trabalho de anos de desenvolvimento, investido muitos recursos nos seus protótipos.

É de salientar que para aumentar a aderência a pista em alta velocidade, muitos deles possuem um ventilador para criar sucção melhorando a aerodinâmica e agarrando o robô ao pavimento.

Todos eles têm 4 rodas e apenas 2 motores. As 4 rodas apoiadas em mini rolamentos, entende-se que aumenta para o dobro a aderência com o pavimento, e apenas dois bons motores para que não duplicar consumos. Na tabela das vantagens é o facto de não necessitar de patim.

Claramente que 6 sensores IR aos pares, será ideal, dado a possibilidade de rapidez de leitura para as distâncias dos obstáculos, e geralmente em duplicado para consolidar e minimizar erros, das leituras.

Tabela 1 – Pódio do APEC 2019 USA <sup>3</sup>

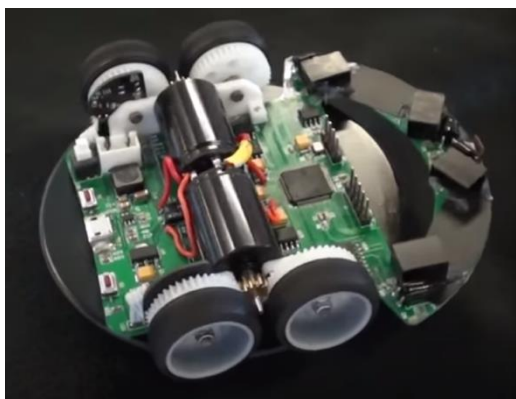


Fig. 1 – EduMouse 1.5 Lunghwa Uni-Taiwan

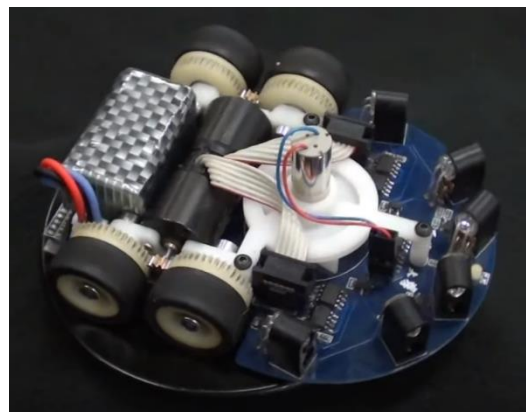


Fig. 2 – RoboMaster Turbo - Tiajin Uni - China

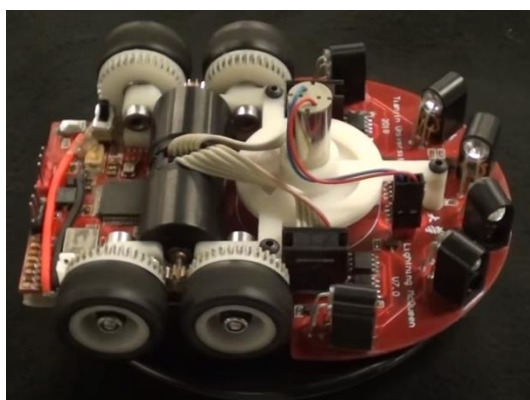


Fig. 3 – Long Turbo V2.0 - Tianjin Uni – China



Fig. 4 – Diu Gow 5 - Xin Han Cai – Taiwan

<sup>3</sup> Obtido de: [https://www.youtube.com/watch?v=cAIwvIA\\_XA](https://www.youtube.com/watch?v=cAIwvIA_XA) – Acedido em Setembro, 2021



### 3. PROJETO

Este capítulo mostra detalhadamente as fases iniciais da concepção do projeto do robô, refletindo sobre a teoria que fundamenta as escolhas exercidas, percorrendo as especificações dos elementos selecionados, e completando com o fabrico e construção do modelo idealizado.

#### 3.1. Conceitos

A fim de maximizar o sucesso no desempenho do robô é fundamental ponderar as variáveis que condicionam a interação com o meio físico.

##### 3.1.1. Agilidade

Por agilidade entende-se a habilidade de mudar de posição com eficácia, ou seja, alcançando o resultado pretendido, que é atingir determinada posição o mais rapidamente possível.

Neste caso, as grandezas que condicionam esta capacidade são inúmeras, começando pela massa, que deve ser mínima, dado influenciar diretamente, tanto o momento de inércia como o momento linear.

O momento linear quantifica o produto da massa pela velocidade em kg.m/s ou N.s<sup>4</sup>

$$\vec{p} = m\vec{v}$$

O impulso pode ser visto como variação do momento linear:

$$\vec{I} = \vec{p} - \vec{p}_0 = \Delta\vec{p}$$

E para fazer variar esse momento, obviamente que tem de se aplicar uma força num determinado período de tempo:

$$\vec{I} = \int_{t_0}^t \vec{F} dt$$

Em suma, quanto menor for a massa, menor é a força necessária para atingir determinada velocidade, e mais eficiente se torna o veículo.

O momento de inércia de um corpo rígido  $I_C$  é definido pelo somatório do produto da massa de cada partícula  $m_i$  pelo quadrado da distância da mesma  $r_i$  ao eixo de rotação. Podemos transformar o somatório numa integral, integrando para todo o corpo  $C$ , o produto da massa  $m$  em cada ponto pelo quadrado da distância  $r$  até o eixo de rotação.<sup>5</sup>

$$I_C = \sum_{i=1}^n m_i r_i^2 \quad I_C = \int_C r^2 dm$$

Por isto, além do cuidado com massa reduzida, posicionar os elementos mais pesados o mais próximo do centro também deve ser uma preocupação.

Outra grande questão prende-se naturalmente com as dimensões do robô, para atingir um elevado desempenho é importante maximizar o espaço livre à sua volta, sendo a dimensão da pista regulamentada, a única opção é mesmo reduzir o tamanho do micromouse dentro do possível.

<sup>4</sup> Obtido de: [https://pt.wikipedia.org/wiki/Momento\\_linear](https://pt.wikipedia.org/wiki/Momento_linear) (Irodov, 1988) - Acedido em Setembro, 2021

<sup>5</sup> Obtido de: [https://pt.wikipedia.org/wiki/Momento\\_de\\_inércia#Cálculo](https://pt.wikipedia.org/wiki/Momento_de_inércia#Cálculo) - Acedido em Setembro, 2021

### 3.1.2. Aderência

A aderência do robô ao piso é de extrema importância, é o que fornece tração ao veículo, a perda de tração resulta em perda de controle do mesmo.

O termo científico mais correto é atrito, mais especificamente o atrito seco, que surge de uma combinação de adesão entre superfícies, rugosidade, deformação e contaminação das mesmas. O atrito seco é subdividido em atrito estático entre superfícies imóveis, e atrito cinético entre superfícies móveis. Perder tração significa passar de atrito estático para cinético, isso acontece quando a força aplicada pelo pneu no pavimento supera a força de atrito estático máxima  $F_{a.e.max}$  que é dada pelo produto do coeficiente de atrito estático  $\mu_e$  e da força normal  $N$  <sup>6</sup>

$$|F_{a.e.max}| = \mu_e \cdot N$$

Considerando o caso específico de plano nivelado em que a força normal é perpendicular ao mesmo, em prol de aumentar a força de atrito máxima sem sacrificar a agilidade só nos resta aumentar o coeficiente de atrito estático.

O coeficiente de atrito entre duas superfícies é uma grandeza empírica, ou seja, é determinada a partir de dados experimentais <sup>7</sup> e compreende a natureza dos materiais em contato, a extensão da área de superfície e, no caso do atrito cinético, inclui ainda o tempo que as superfícies permaneceram em contato. <sup>8</sup>

Assim sendo o material dos pneus escolhidos é de extrema importância, visto que a área da superfície em contacto é limitada por outros fatores.

### 3.1.3. Aceleração

O ritmo com que se altera a posição, ou em linguagem mais científica, a derivada de segunda ordem da posição de uma partícula representativa de um objeto rígido é vulgarmente denominada de aceleração.

Elevada aceleração aumenta a agilidade, no entanto, compromete a aderência.

Apesar da aceleração poder ser controlada por software, evitando perdas de tração, não existe necessidade de munir o veículo com motores demasiado potentes, se depois limitarmos a potência. Além disso, quanto mais potentes forem os motores, mais massa adicionamos ao robô.

---

<sup>6</sup> Obtido de: [https://pt.wikipedia.org/wiki/Atrito#Atrito\\_estático](https://pt.wikipedia.org/wiki/Atrito#Atrito_estático) - Acedido em Setembro, 2021

<sup>7</sup> Obtido de: [https://pt.wikipedia.org/wiki/Atrito#Atrito\\_estático](https://pt.wikipedia.org/wiki/Atrito#Atrito_estático) - Acedido em Setembro, 2021

<sup>8</sup> Obtido de: <https://pt.wikipedia.org/wiki/Atrito#História> - (Dowson, 1997) Acedido em Setembro, 2021

### 3.1.4. Travagem

Comparando ambos os regimes transitórios, aceleração e travagem, à primeira vista parecem simétricos, enquanto no regime de aceleração a variação de velocidade angular é positiva tal com o sentido da corrente nos enrolamentos do motor, no regime de travagem dinâmica a variação de velocidade é negativa tal como o sentido da corrente imposta pelo circuito de controlo de travagem dinâmica.

Se analisarmos graficamente os regimes descritos <sup>9</sup> verificamos a coerência das suposições enunciadas.

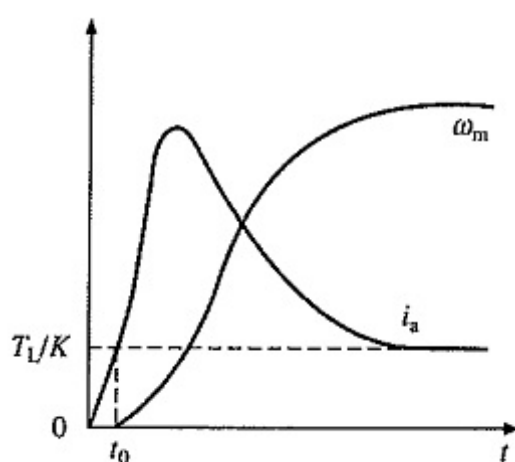


Fig. 5 – Resposta transitória de um motor DC em regime de Aceleração

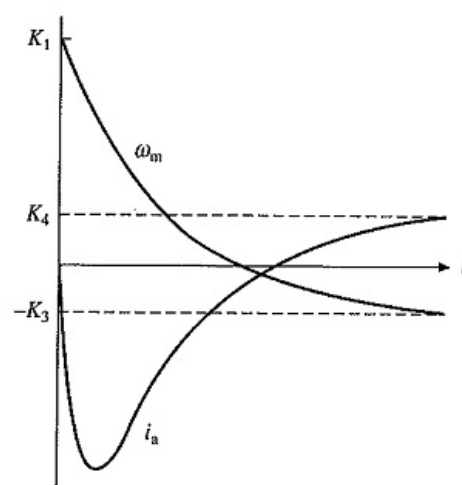


Fig. 6 – Resposta transitória de um motor DC em regime de Travagem Dinâmica

Optando pelo controlo dinâmico da travagem corremos o risco de a velocidade angular continuar a diminuir depois de ser nula, tornando-se negativa. Outro problema seria o elevado binário gerado ter como consequências a perda de tração.

O controlo por circuito aberto limita-se a dissipar a energia cinética acumulada através das perdas mecânicas do veículo. Uma das vantagens é não comprometer a tração, no entanto a resposta pode ser prolongada.

Uma forma de controlo intermédia é curto-circuitar os enrolamentos da máquina DC, que funciona como gerador no intervalo considerado, a corrente gerada cria uma força contra eletromotriz que se opõe ao movimento, acabando por travar o veículo mais rapidamente sem promover os problemas anteriormente enunciados.

<sup>9</sup> Obtido de: <https://www.eceguide.com/transient-analysis-of-dc-motor> - Acedido em Setembro, 2021

## 3.2. Componentes

Uma das facetas de um Engenheiro é a de Gestor, para gerir os recursos disponíveis com eficácia e eficiência é preponderante analisar as ofertas disponíveis no mercado, comparar as suas características, averiguar se a real utilidade de um determinado produto assim as exige, e seleccionar aquele com melhor relação preço/qualidade.

A escolha dos componentes e elementos constituintes deste projeto teve como objetivo alcançar uma solução acessível, que permita conquistar desempenhos razoavelmente satisfatórios.

### 3.2.1. Alimentação

As baterias são os elementos mais pesados que constituem no nosso robô micromouse, a energia armazenada por elas deve ser suficiente para garantir o correto funcionamento do robô durante o período máximo de uma prova. No entanto, não devem ser excessivamente potentes, tendo como consequência um aumento desnecessário da massa do veículo.

#### 3.2.1.1. Fonte de Energia

Duas células (16340) de 3,7V, pequenas em tamanho, são a versão recarregável de uma pilha CR123A. É uma bateria cilíndrica de íões de lítio. Geralmente têm uma voltagem de 3,6V ou 3,7V, terminal de botão, e a maioria usa química LiCoO<sub>2</sub> (óxido de carbono de lítio). Algumas veem com portas de carregamento USB integradas, mas todas as baterias 16340 são recarregáveis com carregadores inteligentes de íões de lítio. Estas células são frequentemente usadas em lanternas ou outros pequenos componentes eletrónicos compactos.

Optando por duas pilhas recarregáveis de íões de lítio CR123A cumprimos com vários objetivos, devido a sua dimensão reduzida, Ø16mm × 34mm e com apenas 18g cada unidade, dispõe de suportes aplicáveis em placas de circuito impresso para uma facilitada instalação e substituição.



Fig. 7 – Células de Lítio 16340 <sup>10</sup>



Fig. 8 – Suporte para Células CR123A <sup>11</sup>

Se considerarmos que os motores consomem no mínimo metade da potência, num regime de funcionamento normal, com um total de 7,4V temos 650mAh para gastar durante 10 minutos

$$P = \frac{E}{\Delta t} = \frac{7.4 \times 0.65 \times 3600}{10 \times 60} = 28.86 \text{ W}$$

Que nos estabelece uma referência para a potência máxima dos dois motores.

<sup>10</sup> Obtido de: <https://www.tme.eu/pt/details/accu-16340-0.6-1.9/pilhas-recarregaveis/xtar/16340-650-1-95a-button-top/>

<sup>11</sup> Obtido de: <https://www.tme.eu/pt/details/bhc-cr123a/pilhas-suportes/com/> - Acedido em Setembro, 2021

### 3.2.1.2. Níveis de Tensão

Motores esses que são alimentados diretamente à tensão das baterias, não é o caso do processador que precisa de ser alimentado a 5V, a melhor forma de o fazer é criar uma linha de alimentação de 5V através de um regulador de tensão. O L4940V5 tem “*ultra low dropout voltage*”, cerca de 0,45V com uma carga de 1A, suportando até 1,5A de carga máxima. Para ajudar a manter a tensão constante, perturbada pelos picos de consumo de corrente utilizamos condensadores eletrolíticos com capacidades de 100 $\mu$ F e 22 $\mu$ F respetivamente na entrada e na saída do regulador de tensão.



Fig. 9 – L4940V5 <sup>12</sup>



Fig. 10 – Condensadores Eletrolíticos

Outra coisa a não esquecer são os comandos das linhas de alimentação, um de potência para controlar a alimentação do robô, sem ele teríamos de remover e colocar as baterias cada vez que quiséssemos ligar ou desligar o micromouse.

E outro para controlar a alimentação dos periféricos alimentados pela placa do processador a 3,3V.



Fig. 11 – Interruptor de 3A



Fig. 12 – Interruptor 100mA

Sem isso, o carregamento do programa na EPROM tem tendência a falhar, por excesso de corrente fornecida por parte do regulador de tensão da placa do processador, que origina uma queda de tensão superior á permitida para o bom funcionamento da etapa de carregamento do programa.

<sup>12</sup> Obtido de: <https://www.tme.eu/pt/details/l4940v5/reguladores-de-tensao-nao-regulada-ldo/stmicroelectronics/>

### 3.2.2. Atuadores

Para que o nosso robô seja um veículo autônomo precisamos certamente de o equipar com um sistema motriz constituído por rodas, motores e controladores dos motores.

#### 3.2.2.1. Rodas

Para as rodas, a solução encontrada foi um conjunto de jantes de alumínio, leves e resistentes, com 17,2mm de diâmetro e 12,5mm de largura, envolvidas por pneus de borracha macia, com 24mm de diâmetro e 13mm de largura.



Fig. 13 – Jantes Alumínio 17,2x12,5mm <sup>13</sup>



Fig. 14 – Pneus de Borracha <sup>14</sup>

#### 3.2.2.2. Motores

Para transformar energia elétrica em energia mecânica optamos pela aquisição de um par de motores de corrente contínua com caixa redutora incluída.

Um motor miniatura com apenas 40x12x10mm, elevada potência, e escovas de carvão de longa duração. São construídos pela Pololu e existem várias versões com diferentes gearbox, tensão de alimentação, e potência disponibilizada. Existem ainda acessórios de fixação e codificadores apropriados.



Fig. 15 – HPCB 6V 30:1 <sup>15</sup>



Fig. 16 – HPCB vs HP <sup>16</sup>



Fig. 17 – Suportes de Motor <sup>17</sup>

As principais características da versão escolhida são apresentadas na Tabela 3:

<sup>13</sup> Obtido de: <https://slotarrabida.pt/catalogo/pt/132/268-jantes-aluminio-158x85mm-para-veio-332.html>

<sup>14</sup> Obtido de: <https://slotarrabida.pt/catalogo/pt/pneus/886-pneus-p6-borracha-20x11mm-slick-p6.html>

<sup>15</sup> Obtido de: <https://www.pololu.com/product/3072> - Acedido em Setembro, 2021

<sup>16</sup> Obtido de: <https://www.pololu.com/product/3072#lightbox-picture0J6412> - Acedido em Setembro, 2021

<sup>17</sup> Obtido de: <https://www.pololu.com/product/989> - Acedido em Setembro, 2021

Tabela 2 – Características do Motor Escolhido

| Tipo de Motor       | HPCB       |
|---------------------|------------|
| Tensão              | 6 V        |
| Corrente Max        | 1,5 A      |
| Potência Max        | 1,2 W      |
| Corrente em Vazio   | 0,1 A      |
| Caixa Redutora      | 30:1       |
| Velocidade em Vazio | 1100 rpm   |
| Binário Max         | 0,45 kg.cm |

A potência máxima consumida pelos dois motores é de  $2 \times 7.4 \times 1.5 = 22.2W$ , abaixo dos  $28.86W$  anteriormente enunciados, que nos deixa seguros de que mesmo nas condições mais adversas, este conjunto de baterias e motores permitem completar uma prova de 10 minutos.

Para perceber melhor o desempenho deste motor podemos analisar o gráfico, obtido através de vários ensaios, fornecido pelo fabricante.<sup>18</sup>

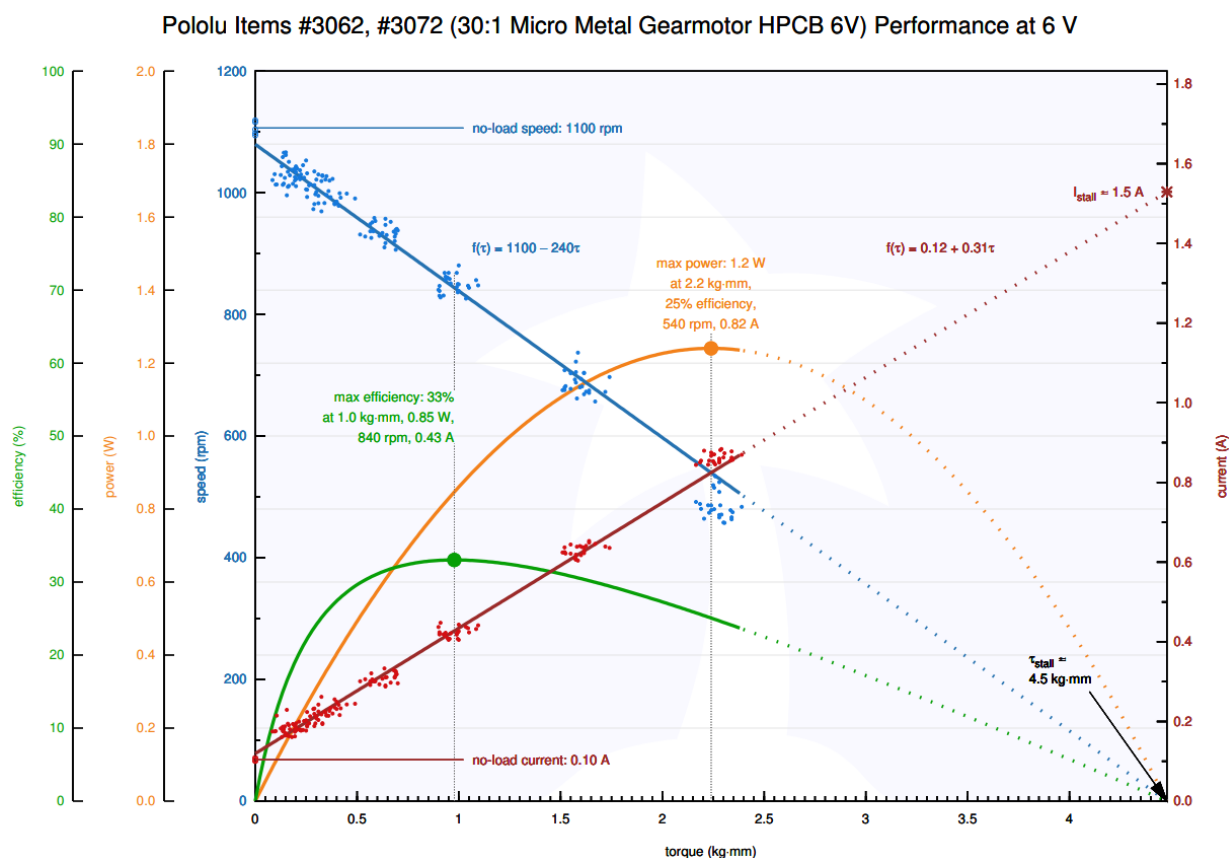


Fig. 18 – Gráfico de desempenho do motor a 6V

De onde retiramos que o ponto de potência máxima ocorre quando o motor está a trabalhar a metade da velocidade máxima, e metade do binário máximo, para lá desse ponto é desaconselhável operar, o motor começa a perder potência, não consegue vencer o binário resistente e acaba por parar. Observamos também que a velocidade diminui linearmente com a carga, e que a eficiência máxima é de 33% fornecendo  $0,85W$  a  $840 \text{ rpm}$ .

<sup>18</sup> Obtido de: <https://www.pololu.com/file/0J1487/pololu-micro-metal-gearmotors-rev-4-2.pdf> - Acedido em Setembro, 2021

Para perceber a performance do robô munido com estes motores podemos fazer os seguintes cálculos:

O perímetro da roda  $P_R$  com 24 mm de diâmetro é dado por:

$$P_R = 2\pi \times \frac{24}{2} = 75.4 \text{ mm}$$

A velocidade angular máxima  $\omega_{max}$  do motor é dada por:

$$\omega_{max} = 1100 \text{ rpm} = \frac{1100}{60} = 18.33 \text{ rps}$$

A velocidade linear máxima  $v_{max}$  do robô é dada por:

$$v_{max} = \omega_{max} \cdot P = 18.33 \times 75.4 = 1382.3 \text{ mm/s}$$

A força resultante  $F_R$  transmitida pelos dois motores ao pavimento, considerando o binário máximo do motor  $\tau_M$  é dada por:

$$F_R = 2 \cdot F_M = 2 \cdot \frac{\tau_M}{r} = 2 \times \frac{2.2 \times 9.78}{12} = 3.586 \text{ N}$$

A aceleração máxima que se pode atingir se for possível manter a tração é dada por:

$$a_{max} = \frac{F_R}{m} = \frac{3.586}{0.25} = 14.344 \text{ ms}^{-2}$$

O mesmo motor com caixa 15:1 atinge o dobro da velocidade, mas metade da aceleração. O motor com caixa 50:1 tem mais aceleração, mas menos velocidade. O motor com a caixa de 30:1 parece ser a escolha acertada, combinando velocidade e binário.

### 3.2.2.3. Drivers

Para que as saídas do microcontrolador limitadas em corrente possam controlar os motores é necessário um intermediário, um driver controlador, também conhecido como ponte H. O nome resulta da sua configuração em ponte, com a forma de um H. É constituída por quatro elementos comutadores, que separam o circuito de potência de o de comando. Permitem formar dois principais circuitos que alimentam o motor em sentidos opostos, controlando assim o sentido de rotação (Euler, 2002).

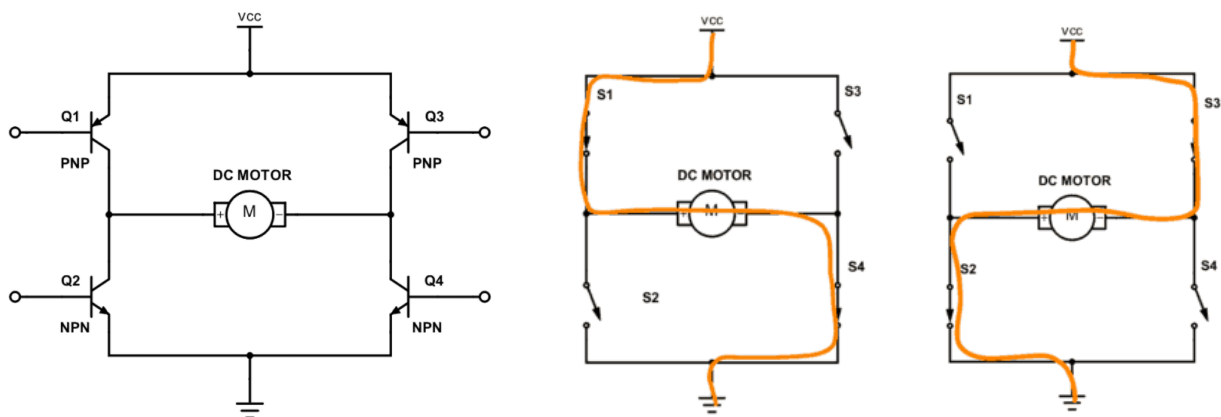


Fig. 19 – Principais Circuitos da ponte H <sup>19</sup>

<sup>19</sup> <https://www.build-electronic-circuits.com/h-bridge/>

É possível ainda deixar o motor à deriva, mantendo quaisquer 3 comutadores em aberto, ou ligar ambos os polos do motor ao mesmo potencial, forçando o motor a travar através da força contra eletromotriz gerada pela transformação da energia cinética acumulada pelo veículo.

Foi utilizado para cada um dos motores um controlador DRV8838 igual ao da figura 21, ligados de forma idêntica ao esquema apresentado na figura 22.



Fig. 20 – DRV8838 <sup>20</sup>

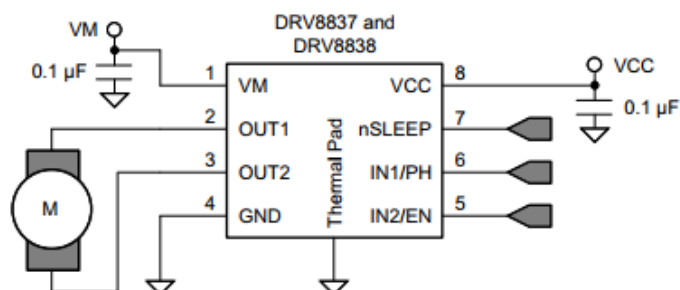


Fig. 21 – Esquema de Aplicações do DRV8838 <sup>21</sup>

O modo de funcionamento dos motores é controlado pelas entradas lógicas do driver DRV8838, tal como descreve a Tabela 4:

Tabela 3 – Funcionamento Lógico do DRV8838

| nSLEEP | PH | EN | OUT1 | OUT2 | FUNCTION |
|--------|----|----|------|------|----------|
| 0      | X  | X  | Z    | Z    | Coast    |
| 1      | X  | 0  | L    | L    | Brake    |
| 1      | 1  | 1  | L    | H    | Reverse  |
| 1      | 0  | 1  | H    | L    | Foward   |

- PH de phase que define o sentido de rotação do motor
- EN de enable que habilita o motor a rodar, caso contrário trava-o
- nSLEEP que ativa os modos anteriores, caso contrário o motor encontra-se em alta impedância

Para controlar a velocidade de um motor DC basta controlar a tensão aos seus terminais. O valor analógico da tensão média pode ser controlado digitalmente através da modelação da largura dos pulsos. Integrando um sinal periódico obtém-se o valor médio da tensão, sendo um sinal de pulsos quadrados com largura e periodicidade conhecida, verifica-se que  $V_{med}$  depende da razão entre o tempo ON e o tempo OFF, denominado ciclo de trabalho, ou *duty cycle* D [%]

$$V_{med} = \frac{1}{T} \int_0^T v(t) dt = D \cdot V_{max}$$

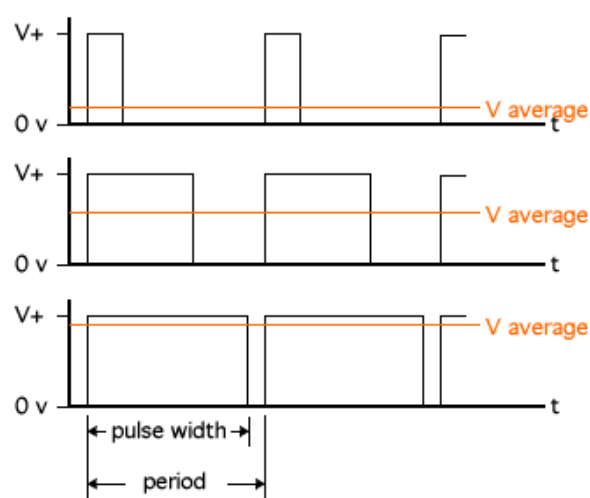


Fig. 22 – Exemplos de PWM e respectivos  $V_{med}$  <sup>22</sup>

<sup>20</sup> Obtido de: <https://www.pololu.com/product/2990> - Acedido em Setembro, 2021

<sup>21</sup> Obtido de: <https://www.ti.com/lit/ds/symlink/drv8838.pdf> - Acedido em Setembro, 2021

<sup>22</sup> Obtido de: <https://wechoosethemoon.es/2011/07/21/arduino-motor-dc-circuito-pwm-mosfet/> - Acedido em 2021

### 3.2.3. Sensores

Os sensores existentes no nosso micromouse têm todos algo em comum: Fornecem-nos valores de distância, por vezes indiretamente, mas a informação por eles cedida acaba por dar conhecimento ao robô acerca da sua posição relativamente ao meio que o rodeia e relativamente a ele mesmo em instantes anteriores.

#### 3.2.3.1. Encoders

Existem dois tipos de codificadores, o mais utilizado é o encoder incremental, pois permite medir distância relativas, que podem ser transformadas em posições caso exista uma referência fixa. No nosso caso vamos utilizá-lo para medir distância percorrida, velocidade e até aceleração. Os encoders adquiridos são especificamente desenvolvidos para trabalharem com os micro-motores pololu como se pode observar na Fig. 23.

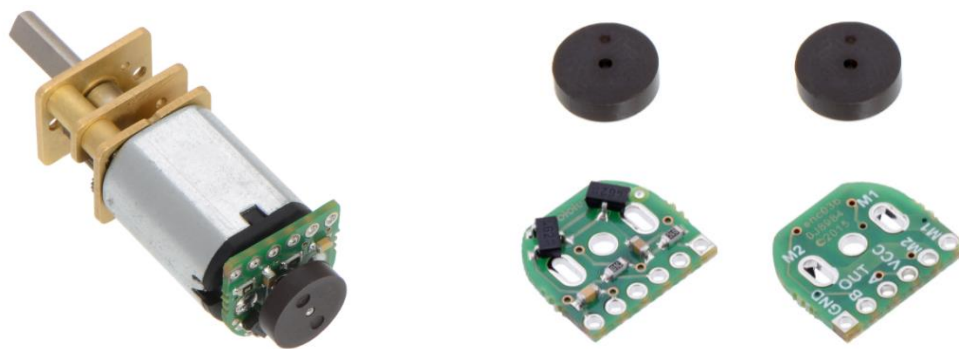


Fig. 23 – Encoder Magnético <sup>23</sup>

Utilizam 2 sensores de efeito de hall TLE4946-2K <sup>24</sup>, desfasados um quarto de volta, para detetar os 3 pares de polos magnéticos presentes em cada disco. O polo Sul, considerado campo magnético positivo, polariza a base de um transístor que conduz a saída à massa. Devido a histerese magnética existe um *jitter* de  $1\ \mu s$  e a frequência máxima de comutação, quando a amplitude do sinal magnético é  $\sqrt{2}$  maior que o campo magnético residual, é  $15kHz$  devido ao filtro passa baixo de  $-3dB$ .

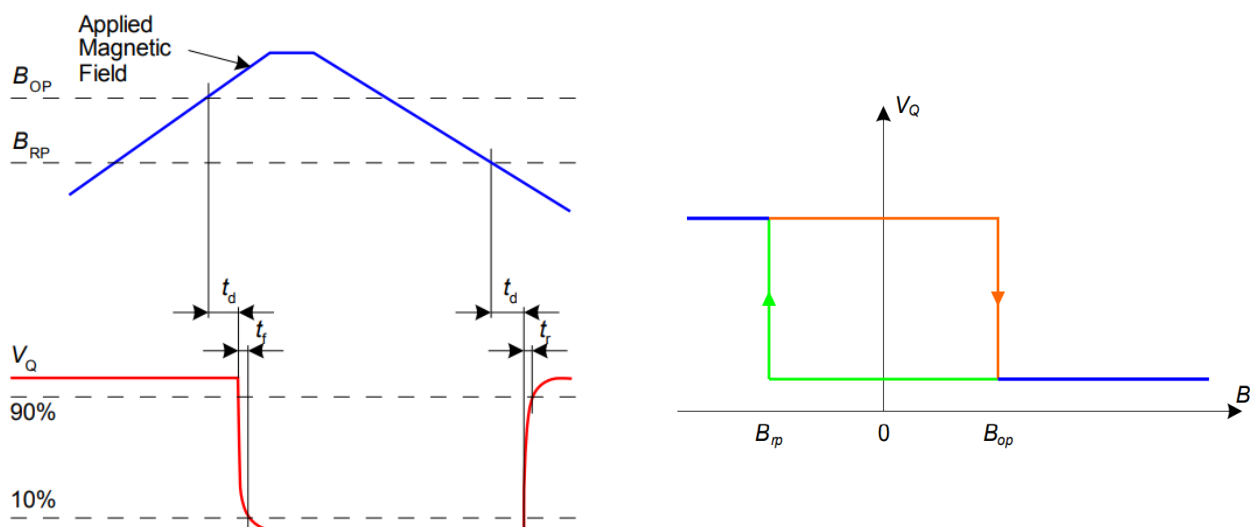


Fig. 24 - Diagrama Temporal e Estados de Operação do TLE4946-2K

<sup>23</sup> Obtido de: <https://www.pololu.com/product/3081> - Acedido em Setembro, 2021

<sup>24</sup> Obtido de: <https://www.pololu.com/file/0J815/TLE4946-2K.pdf> - Acedido em Setembro, 2021

### 3.2.3.2. LED IR e Foto-Transístores

Os sensores de eleição para detetar a existência de paredes do labirinto na proximidade do micromouse, e até mesmo para determinar a distância do robô às mesmas, são formados pela conjugação de um Díodo Emissor de Luz Infravermelha com um Foto-Transístor sensível a radiação com o mesmo comprimento de onda, no nosso caso  $850\text{ nm}$ . O LED emite luz invisível num feixe cónico com  $3^\circ$  de amplitude direcionado às paredes brancas do labirinto, que refletem parte dos fotões existentes no feixe emitido em direções aleatórias, se as paredes fossem espelhadas, o feixe incidente e o feixe refletido teriam ângulos simétricos em relação à normal do plano.

Fig. 25 – SFH 4550<sup>25</sup>Fig. 26 – BPW85C<sup>26</sup>Fig. 27 – ULN2003ADR<sup>27</sup>

Havendo então fotões a colidir violentamente, a mais de mil milhões de quilómetros por hora, contra o nosso foto-transístor, conseqüentemente alguns deles atingem os eletrões dos iões presentes na região dopada central, libertando-os e colocando a junção NPN, previamente polarizada de acordo com a montagem apresentada na Fig. 28, em condução.

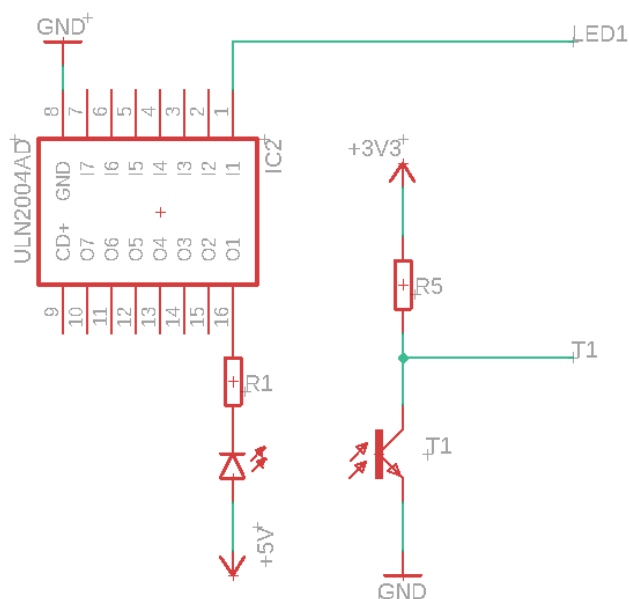
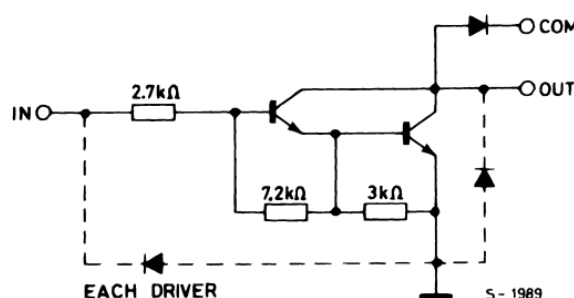


Fig. 28 – Esquema de Ligação do Senso IR



ULN2003 (each driver)

Fig. 29 – Esquema Interno do Darlington Array

Para que os LED's sejam controlados pelo microcontrolador e suas saídas de baixa potência é crucial a utilização de um *Darlington Array* como o da Fig. 27 que estabelece uma ligação de baixa impedância entre a saída e a massa quando a entrada tiver um potencial de  $3.3\text{ V}$ , tal como ilustrado no esquema da Fig. 29.

<sup>25</sup> Obtido de: <https://www.tme.eu/pt/details/sfh4550/diodos-led-infravermelhos/osram/sfh-4550/>

<sup>26</sup> Obtido de: <https://www.tme.eu/pt/details/bpw85c/fototransistores/vishay/> Acedido em Setembro, 2021

<sup>27</sup> Obtido de: <https://www.tme.eu/pt/en/details/uln2003adr/drivers-integrated-circuits/texas-instruments/>

### 3.2.3.3. LIDAR

Enquanto os sistemas de localização SONAR utilizam ondas sônicas numa ampla gama de frequências para detetar e localizar objetos subaquáticos e os sistemas de RADAR utilizam ondas radio de baixa frequência para detetar obstáculos a longas distâncias. Os sistemas de LIDAR utilizam radiação eletromagnética da região infravermelho próxima da visível e são tipicamente utilizados em aplicações que requerem elevada quantidade de dados num curto espaço de tempo.

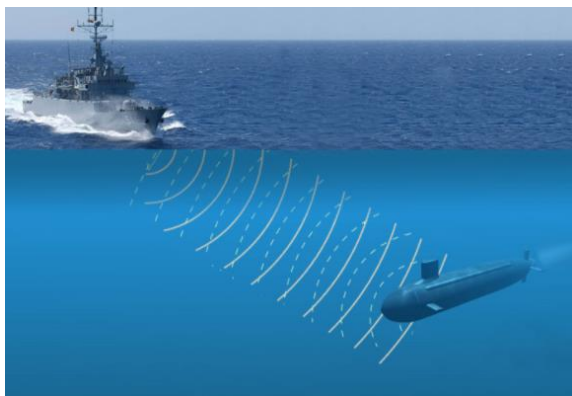


Fig. 30 – Princípio de funcionamento de um SONAR <sup>28</sup>



Fig. 31 – Dados recolhidos por um LIDAR <sup>29</sup>

Os sensores ToF<sup>30</sup> tornaram-se muito comuns devido à sua confiabilidade e facilidade de uso. O princípio de funcionamento é enviar um sinal (Ultrasound-Laser wave-Infrared), e esperar que ele regresse. A duração entre o envio e receção é chamada de Tempo de Voo: a velocidade da luz é conhecida, a duração é calculada e dividida por 2 e assim obtém-se a distância a cada pedido. A vantagem dos LASER é que eles são muito pequenos e podem ser usados para aplicações de alta frequência.

### VL53L3CX Time-of-Flight Distance Sensor Carrier<sup>31</sup>



Fig. 32 – VL53L3CX

Este sensor é uma placa para com sensor de distância a laser VL53L3CX da ST, que oferece alcance rápido e preciso de até 5 metros. Usa o tempo de voo (ToF) de pulsos de laser invisíveis e seguros para os olhos para medir distâncias absolutas a vários alvos simultaneamente, independentemente das condições de iluminação do ambiente e das características do alvo, como cor, forma e textura (embora essas realidades afetem o alcance máximo) As medições de distância podem ser lidas através da interface digital I<sup>2</sup>C. A placa inclui um regulador linear de 2,8 V e deslocadores de nível que permitem que ela trabalhe em uma faixa de tensão de entrada de 2,6 V a 5,5 V.

<sup>28</sup> <https://www.naval.com.br/blog/2018/01/17/como-funciona-o-sonar-ativo/>

<sup>29</sup> <https://www.gim-international.com/content/article/multibeam-lidar-for-mobile-mapping-systems>

<sup>30</sup> Saber Mais: <https://www.terabee.com/time-of-flight-principle/> - - Acedido em Setembro, 2021

<sup>31</sup> Data Sheet: <https://drive.google.com/file/d/1uaMZ421sm10FN40iJgn-ZKQG42KUvFfh/view> - Acedido em Setembro, 2021

## TOF10120 Laser Range Sensor Module<sup>32</sup>

ToF 10120 (Time-of-Flight), é barato, é pequeno, tem 2 formas de comunicação, é rápido, e nas desvantagens não encontramos nada para escrever aqui.

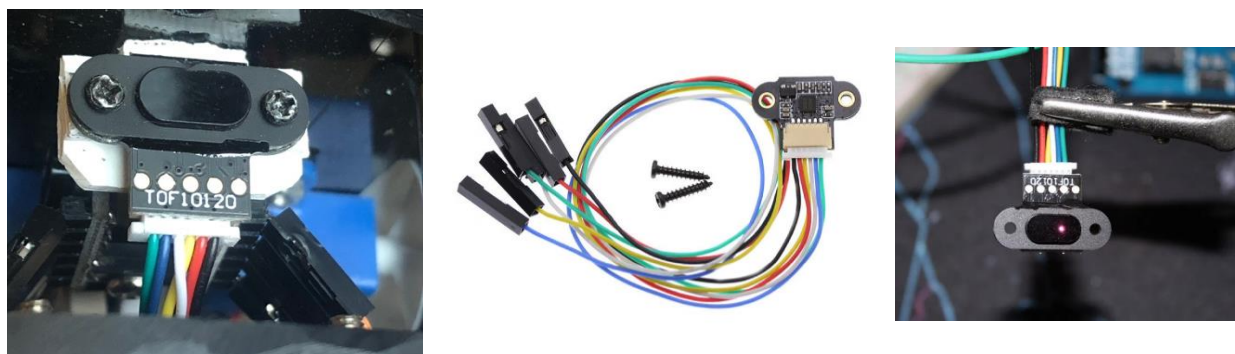


Fig. 33 – Lidar ToF10120 <sup>33</sup>

O sensor tem 6 pinos (GND-VCC), (RX-TX) para UART e (SDA-SCL) para i<sup>2</sup>c. Neste projeto usamos a comunicação por i<sup>2</sup>c para conectá-lo com o Arduino. Este minúsculo sensor requer também umas miseras 15 linhas de código divididas em 2 funções para obter com grade precisão a distância aos obstáculos, e isto é chama bom rendimento.

## MPU6050 giroscópio e acelerómetro <sup>34</sup>

O MPU-6050 combina um giroscópio de 3 eixos e um acelerômetro de 3 eixos no mesmo chip de silício, em conjunto com um Digital Motion Processor <sup>TM</sup> (DMP <sup>TM</sup>) integrado, que processa algoritmos complexos de MotionFusion de 6 eixos. O dispositivo pode aceder a magnetómetros externos ou outros sensores por meio de um barramento I<sup>2</sup>C mestre auxiliar, permitindo que os dispositivos obtenham um conjunto completo de dados do sensor sem intervenção do processador do Arduino.

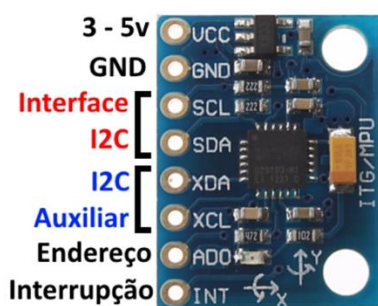


Fig. 34 – Acelerómetro e Giroscópio MPU <sup>35</sup>

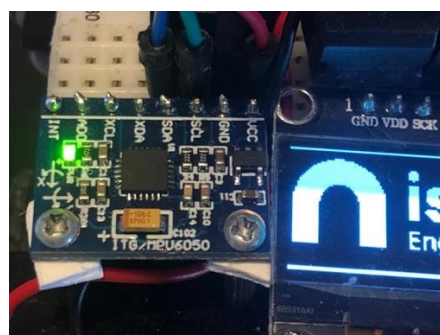


Fig. 35 – MPU6050

Este sensor é responsável por muito tempo gasto, sem obter resultados aceitáveis para este projeto. Conseguiu-se adquirir resultados aceitáveis, mas para movimentos suaves, mas como se pretende um robô ágil e rápido, esta ideia foi abandonada.

<sup>32</sup> Data Sheet: <https://drive.google.com/file/d/1uaMZ421smI0FN40iJgn-ZKQG42KUvFfh/view> - Acedido em 2021

<sup>33</sup> Obtido de: <https://surttech.com/2019/03/18/easy-use-of-tof-10120-laser-range-finder-to-measure-distance-with-arduino-lcd/> - Acedido em Setembro, 2021

<sup>34</sup> Data Sheet: <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf> - - Acedido em 2021

<sup>35</sup> Obtido em: <https://www.filipeflop.com/blog/tutorial-acelerometro-mpu6050-arduino/> - Acedido em Setembro, 2021

### 3.2.4. Processador

ESP32-DEVKITC<sup>36</sup>, designado apenas por ESP32 daqui em diante, é um MCU abastado em recursos com Wi-Fi integrado e Conectividade Bluetooth para uma ampla gama de aplicações. Foi projetado para dispositivos móveis, eletrônicos versáteis e aplicativos de IoT, o ESP32 atinge um consumo de energia ultrabaixo com uma combinação de vários tipos de software. O ESP32 também inclui recursos de última geração, como gating de relógio de baixa granularidade, vários modos de energia e escalonamento de energia dinâmico.

ESP32 é capaz de funcionar de forma confiável em ambientes industriais, com uma temperatura de operação variando de  $-40^{\circ}\text{C}$  a  $+125^{\circ}\text{C}$ . Alimentado por circuitos de calibração avançados, o ESP32 pode remover dinamicamente as imperfeições do circuito externo e adaptar-se às mudanças das condições externas.

O ESP32 pode funcionar como um sistema autónomo completo ou como dispositivo escravo para um MCU host, reduzindo a sobrecarga da pilha de comunicação no processador principal. O ESP32 pode fazer interface com outros sistemas para fornecer funcionalidade Wi-Fi e Bluetooth por meio de suas interfaces SPI ou I2C<sup>37</sup>/ UART.



Fig. 36 – ESP32 DevKitC

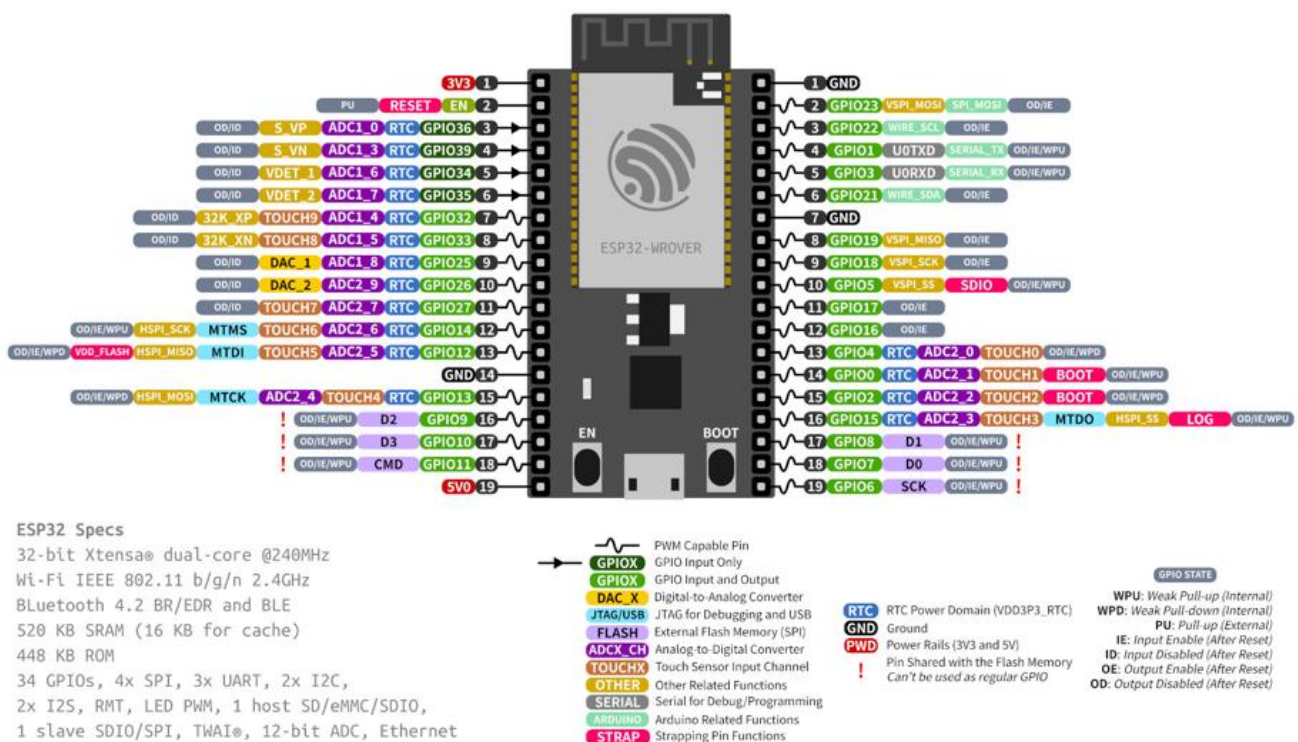


Fig. 37 – PinOut ESP32 DevKit C<sup>38</sup>

<sup>36</sup> Detalhes em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32c3/hw-reference/esp32c3/user-guide-devkitc-02.html#getting-started> - Acedido em Setembro, 2021

<sup>37</sup> I2C = <https://www.nxp.com/docs/en/user-guide/UM10204.pdf> - Acedido em Setembro, 2021

<sup>38</sup> Obtido em: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html> - Acedido em Setembro, 2021

A programação desta placa pode ser elaborada em linguagem C e C++ na plataforma Arduino IDE onde escrevemos de forma mágica os programas a serem carregados. Para o ESP32 é necessário instalar no IDE, e em menos de 1 minuto fica pronta para carregar magia para a memória do ESP32 DEVKITC. De memória podemos contar com 448 KB ROM, 520 KB SRAM, 16 KB SRAM em RTC. Este IDE permite ver no monitor, os dados que queremos adquirir dos vários sensores ligados ao ESP32, pelo Monitor Serial, ou via Web Serial, ou ainda para smartphone por Bluetooth.

O ESP32 engloba dois microprocessadores Xtensa® de 32 bits LX6 de baixa potência pipeline de 7 estágios para suportar a frequência do clock de até 240 MHz, o conjunto de instruções de 16/24 bits fornece alta densidade de código, suporte para Unidade de Virgula Flutuante, suporte para instruções DSP (Digital Signal Processor), como um multiplicador de 32 bits, um divisor de 32 bits e um MAC de 40 bits, suporte para 32 vetores de interrupção de cerca de 70 fontes de interrupção, só não tem imaginação para se tirar todo o partido dos seus recursos. Incorpora quatro temporizadores de uso geral embutidos, e, todos são cronômetros genéricos de 64 bits que se baseiam em prescalers<sup>39</sup> de 16 bits e temporizadores de up/down com reload automático de 64 bits.

O recurso de temporizadores são um prescaler de relógio de 16 bits, de 2 a 65536, um cronômetro de 64 bits, temporizador de aumento/diminuição configurável (aumenta ou diminui), parar e retomar o contador de base de tempo, recarregar automaticamente em caso de alarme, recarga instantânea controlada por software e geração de interrupção de nível e borda.

O ESP32 tem 34 pinos GPIO que podem ser atribuídos a várias funções programando-os com registros apropriados. Existem vários tipos de GPIOs: apenas digitais, habilitados para analógico e habilitados para toque capacitivo e estes podem ser configurados como GPIOs digitais.

A maioria dos GPIOs digitais pode ser configurada como pull-up ou pull-down interno, ou definida para alta impedância. Quando configurado como uma entrada, o valor de entrada pode ser lido por meio do registro. A entrada também pode ser definida como trigger de nível para gerar interrupções do CPU. A maioria dos pinos IO digitais são bidirecionais, não invertidos e tri-state, incluindo buffers de entrada e saída com controle triplo. Esses pinos podem ser multiplexados com outras funções, como SDIO, UART, SPI, etc.

O ESP32 integra ADCs de 12 bits e oferece suporte para medições de 18 canais (pinos habilitados para analógico). O coprocessador ESP32 também é projetado para medir a tensão, durante a operação no modo de espera, que permite baixo consumo de energia. A CPU pode ser ativada por uma configuração de limite ou por meio de outro tipo de triggers.

O controlador de modulação por largura de pulso (PWM) pode ser usado para acionar e controlar velocidade motores de forma digital. O controlador consiste em temporizadores PWM, o operador PWM e um sub módulo de captura dedicado. Cada temporizador fornece temporização de forma síncrona ou independente, e cada operador PWM gera uma forma de onda para um canal PWM. O sub módulo de captura dedicado pode capturar eventos com precisão com temporização externa.

---

<sup>39</sup> Prescaler é um dispositivo eletrônico que reduz uma frequência por um fator predeterminado. Por exemplo, converte um sinal de 1 MHz num sinal de 100kHz (regula a frequência por um fator de 10). Em microcontroladores é a parte que divide a frequência de oscilação do clock.

### 3.3. Design

A forma como os elementos constituintes do robô estão distribuídos é tão importante como a escolha dos mesmos, a prestação do micromouse está diretamente relacionada com a configuração idealizada.

Os pontos principais a ter em mente são:

- ✓ Modularidade dos elementos
- ✓ Baixo centro de massa
- ✓ Posicionar os elementos mais pesados o mais próximo do eixo de rotação
- ✓ Sensores IR nas extremidades do micromouse

#### 3.3.1. 3D CAD

Com a finalidade de formular conceções estruturais recorreu-se a um software de desenho assistido por computador, permitindo assim concretizar abstrações e desenvolver críticas construtivas.

O software utilizado foi o FreeCAD<sup>40</sup> devido ao seu interface sugestivo, elevada compatibilidade e distribuição aberta.

Um dos primeiros protótipos que não chegou a sair do ecrã está representado na Fig. 38 .

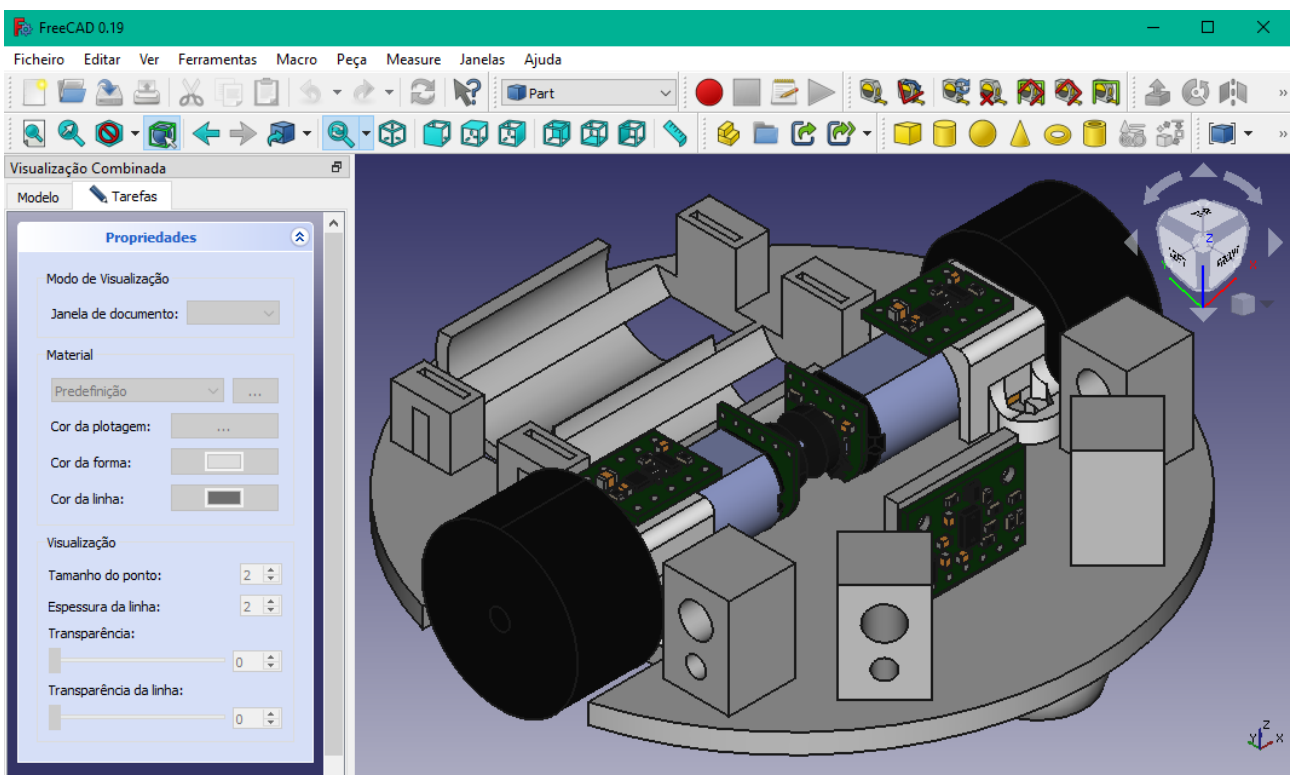


Fig. 38 – Protótipo de MicroMouse desenvolvido no FreeCAD

Apenas houve necessidade de desenhar e imprimir algumas peças 3D, como os suportes para os LED's e Foto-Transístores, suportes para os motores e patins para criar o terceiro ponto de apoio do nosso robô de simples eixo.

A base de suporte dos elementos dos micromouse viria a ser a placa de circuito impresso que interliga todos os componentes entre si.

<sup>40</sup> Obtido de: <https://www.freecadweb.org/features.php> - Acedido em Setembro, 2021

### 3.3.2. PCB

Para Desenvolver uma Placa de Circuito Impresso utilizamos o Software Eagle<sup>41</sup> que utiliza uma janela para o esquema de ligações, e outra para posicionar os componentes e definir as pistas. Desta forma torna-se mais fácil garantir que nenhuma ligação fica trocada ou esquecida.

Começámos por adicionar os conectores que servem também de suporte à placa do ESP32, aos Drivers dos motores, aos Encoders e ao LIDAR. Adicionou-se os restantes componentes e ligaram-se de acordo com o esquemático apresentado na Fig. 39.

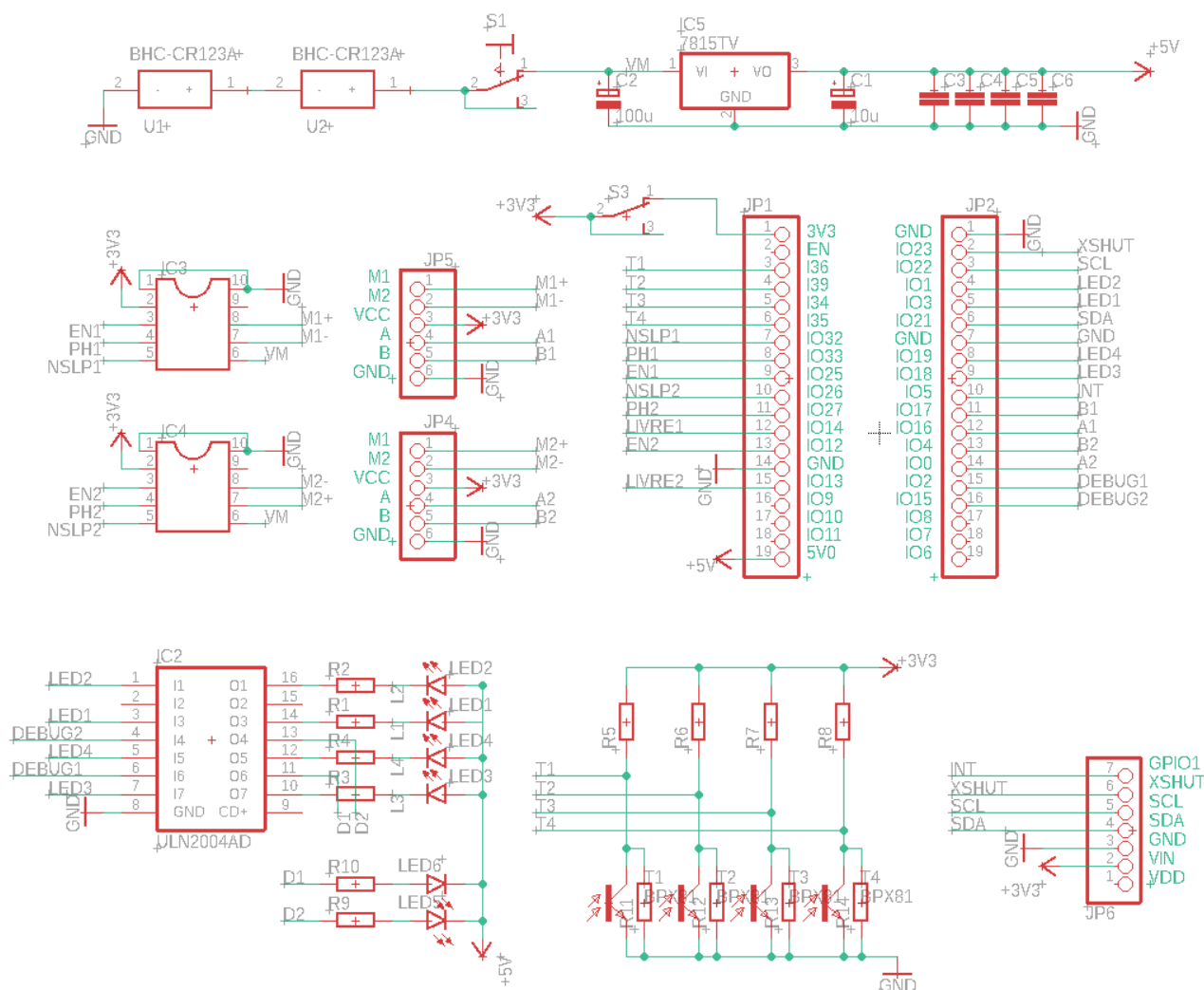


Fig. 39 – Esquemático da PCB do MicroMouse

Feito isto, restava posicionar os componentes nas posições desejadas. O contorno da placa e as furações para os suportes dos motores foram importados no formato DWG que teria sido anteriormente desenhado e exportado no AutoCAD.

A última versão da placa de circuito impresso concebida é apresentada na Fig. 40.

Para soldar os componentes na placa há que ter noção que existem temperaturas e intervalos de tempo de contacto térmico que devem ser respeitados, pondo em causa a destruição dos componentes.

<sup>41</sup> Obtido de: <https://www.autodesk.com/products/eagle/features> - Acedido em Setembro 2021

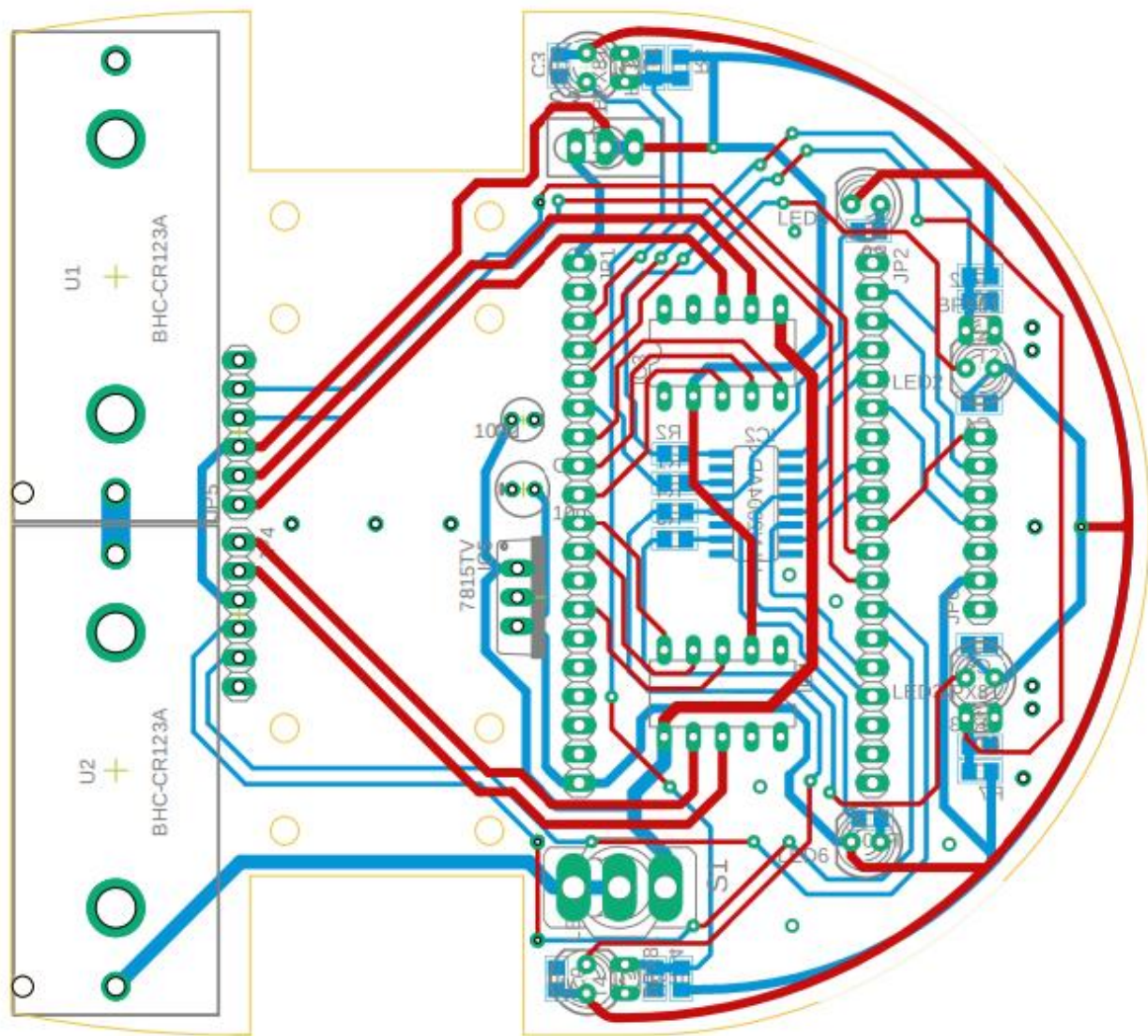


Fig. 40 – Última versão do Projeto da PCB

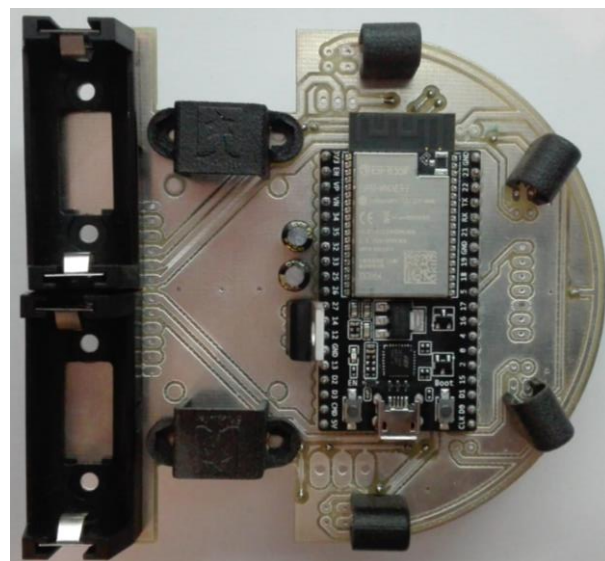
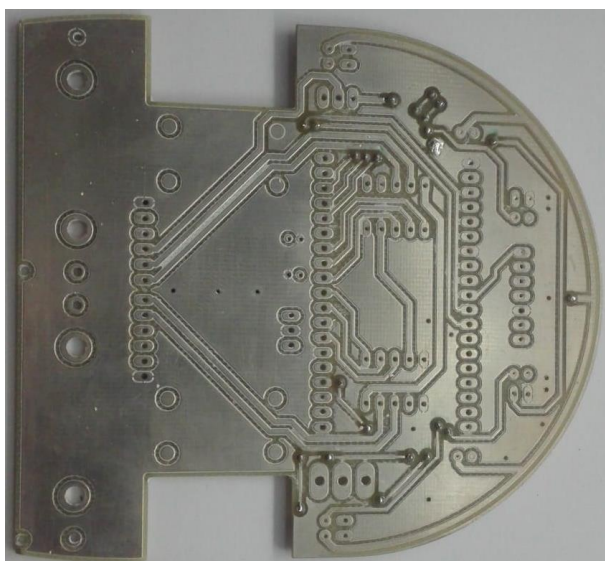


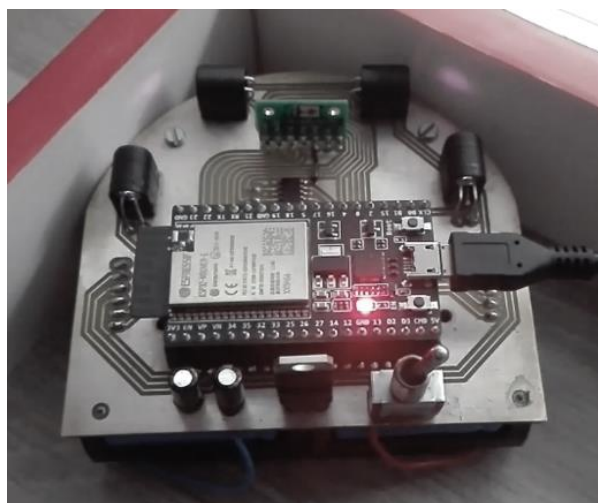
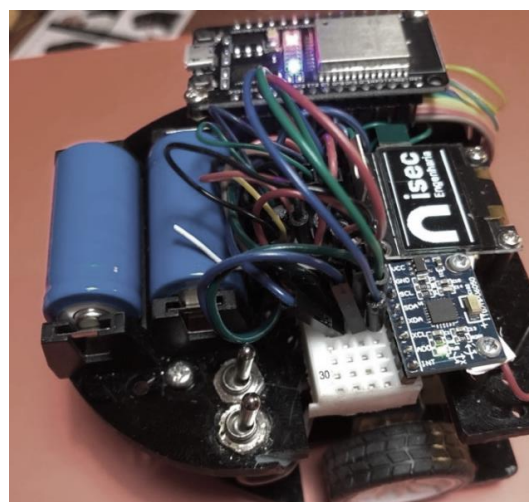
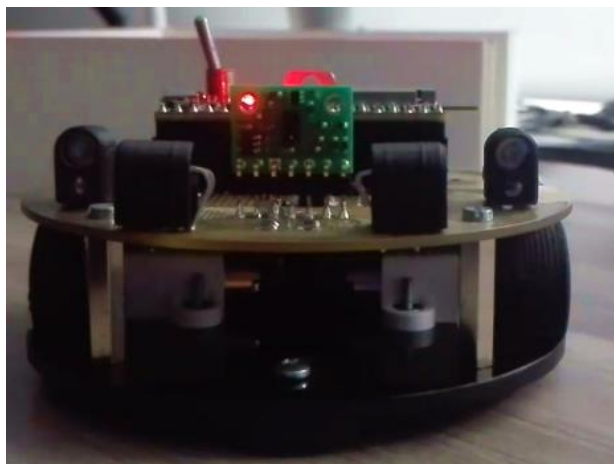
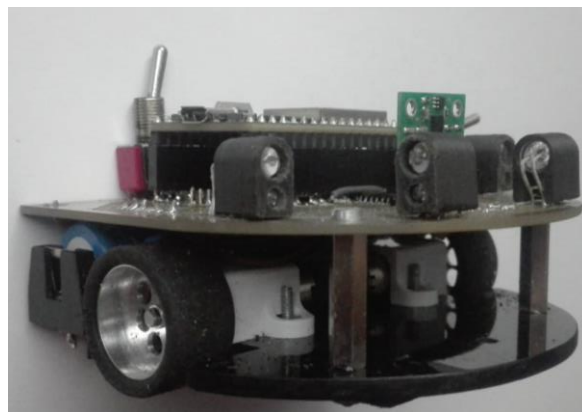
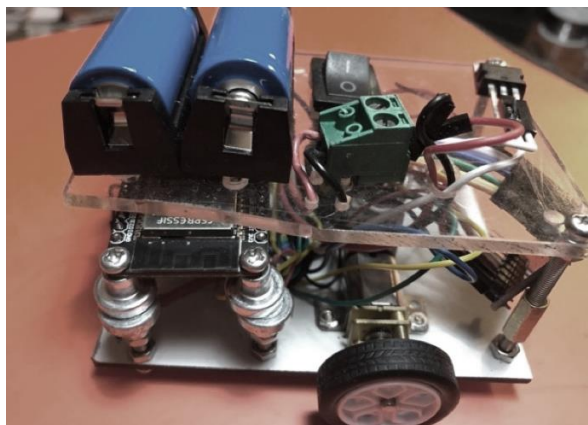
Fig. 41 - Última versão da PCB em Processo de Montagem

### 3.3.3. Protótipos

O nosso micromouse nasceu da evolução das suas versões anteriores. Houve necessidade de montar provisoriamente alguns componentes, de forma a testar hardware, e também de começar a desenvolver software. Para corrigir alguns problemas que se fomos apercebendo com o avanço do desenvolvimento houve necessidade de reestruturar a base de suporte e a placa de circuito impresso.

Algumas das versões anteriores de micromouses podem ser observadas na Tabela 4.

Tabela 4 – Versões Desenvolvidas de Protótipos do MicroMouse





## 4. RESULTADOS

Antes de entrar no campo de batalha e começar a resolver labirintos existe a necessidade de conhecer a informação que vamos trabalhar. Por um lado, para perceber se os componentes funcionam e como funcionam, e por outro, para perceber se a abordagem inicial para a implementação de software é prospera ou se precisa de ser repensada.

Neste capítulo iremos apresentar tabelas e gráficos dos resultados obtidos no teste, tanto das montagens dos circuitos dimensionados, como dos componentes prontos a utilizar.

### 4.1. Sensores

Para testar os sensores foi importante conhecer o seu pinout para os ligar corretamente, perceber como se utilizam as funções `analogRead()`, `digitalWrite()`, e outras bibliotecas existentes para os sensores mais avançados, que são ligados ao barramento I<sup>2</sup>C do microprocessador. Os ensaios foram efetuados numa breadboard como ilustra a Fig. 42.

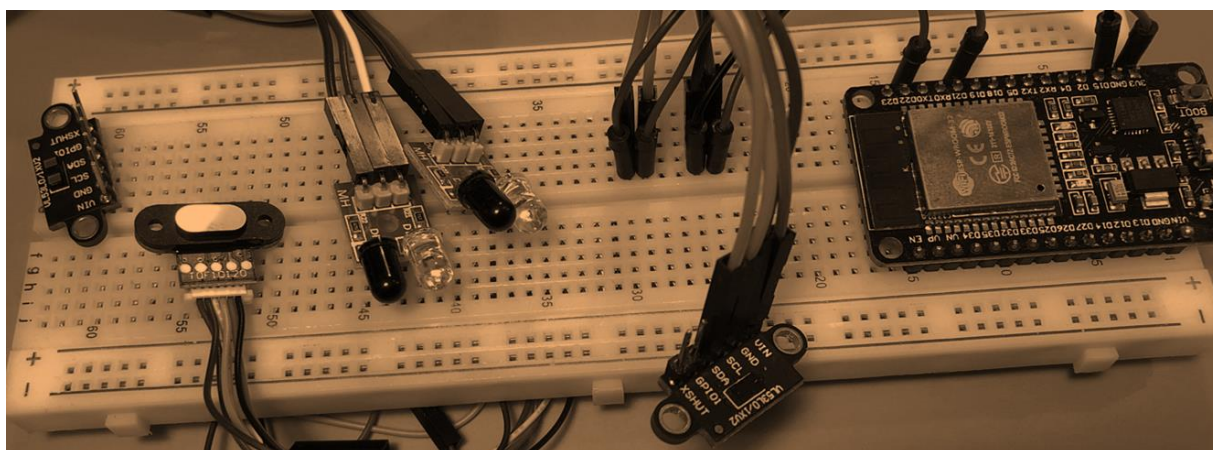


Fig. 42 – Placa de Ensaios dos Sensores

#### 4.1.1. LIDAR

O sensor VL53L3CX <sup>42</sup> adquirido vem integrado numa placa que faz a interface I<sup>2</sup>C e um regulador de tensão de acordo com o seguinte esquema da Fig. 43.

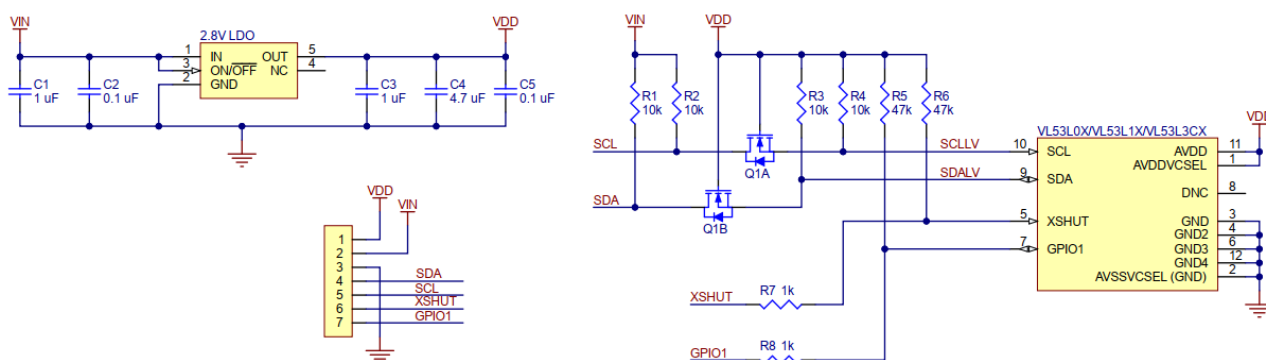


Fig. 43 – Esquema da placa do sensor VL53L3CX

<sup>42</sup> Obtido de: <https://www.pololu.com/file/0J1188/vl53l0x-vl53l1x-vl53l3cx-time-of-flight-sensor-schematic.pdf>

Ligou-se *VIN* aos 3.3V do ESP32, SDA e SCL ao GPIO21 e GPIO22 respetivamente, por serem os pinos por defeito para o bus I<sup>2</sup>C, e os pinos XSHUT e GPIO1 que servem respetivamente para mandar silenciar o sensor e para tomar conhecimento do veredicto da medição, em quaisquer outros pinos do ESP à escolha.

Compreender na totalidade os requisitos para funcionamento deste sensor de acordo com o fluxo da API apresentado na Fig. 44 exigiria demasiado tempo, para nossa benesse encontramos disponível uma biblioteca <sup>43</sup> pronta a utilizar, que com pequenas alterações conseguimos finalmente contemplar o seu correto funcionamento.

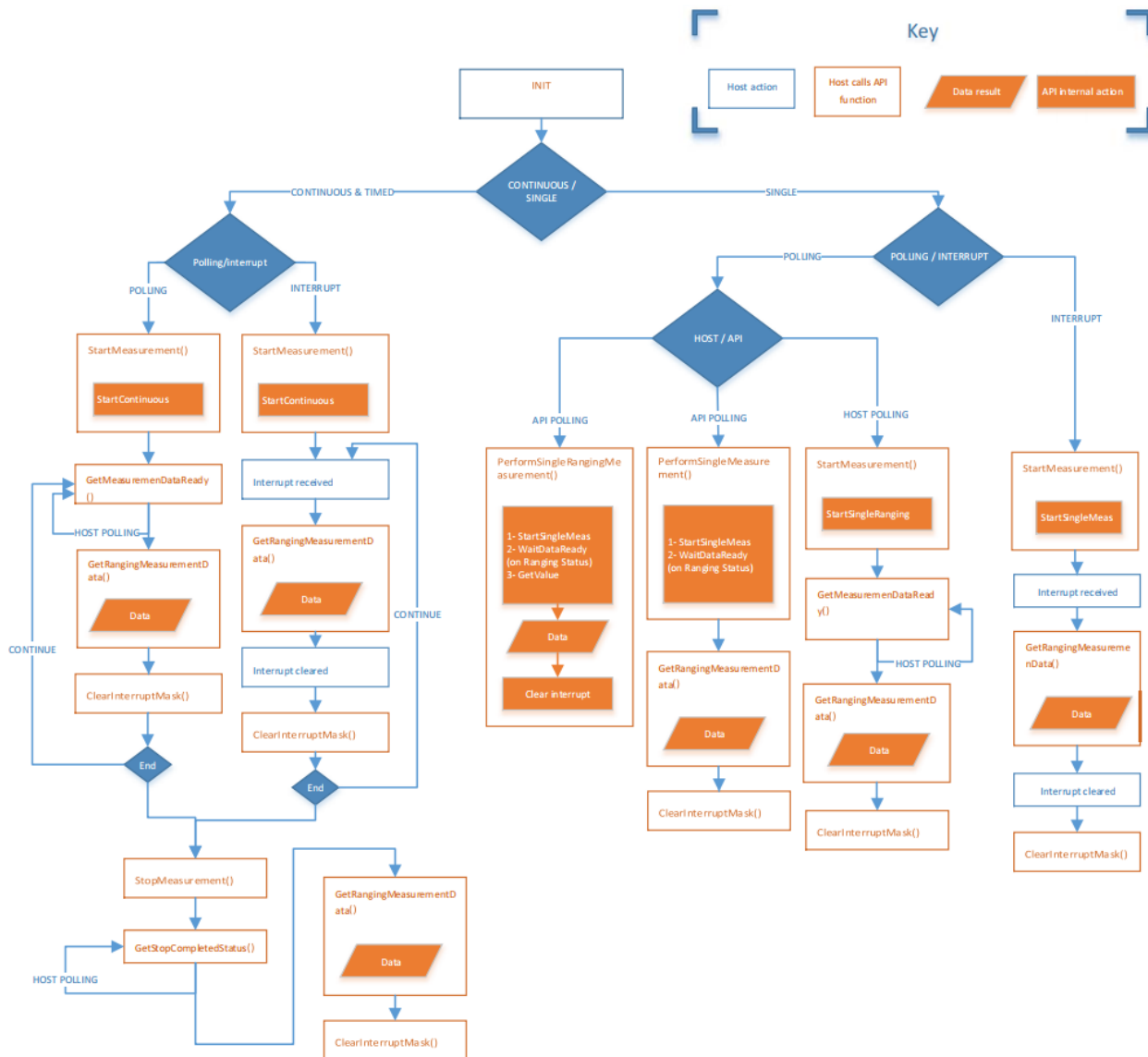


Fig. 44 – Fluxo de aquisição de dados do VL53L0X <sup>44</sup>

Os dados fornecidos pelo sensor são guardados numa variável do tipo inteira que representa o número de milímetros entre o sensor e o alvo. É possível obter as distancia de vários objetos em cada leitura, mas na fase atual em que se encontramos apenas iria aumentar a complexidade de análise de dados.

<sup>43</sup> Obtido de: <https://github.com/stm32duino/VL53L3CX> - Acedido em Junho, 2021

<sup>44</sup> Obtido de: [https://www.st.com/resource/en/user\\_manual/um2039-world-smallest-timeofflight-ranging-and-gesture-detection-sensor-application-programming-interface-stmicroelectronics.pdf](https://www.st.com/resource/en/user_manual/um2039-world-smallest-timeofflight-ranging-and-gesture-detection-sensor-application-programming-interface-stmicroelectronics.pdf) - Acedido em Junho, 2021

Os resultados dos testes efetuados ao sensor ToF10120 estão sumarizados no gráfico da Fig. 45

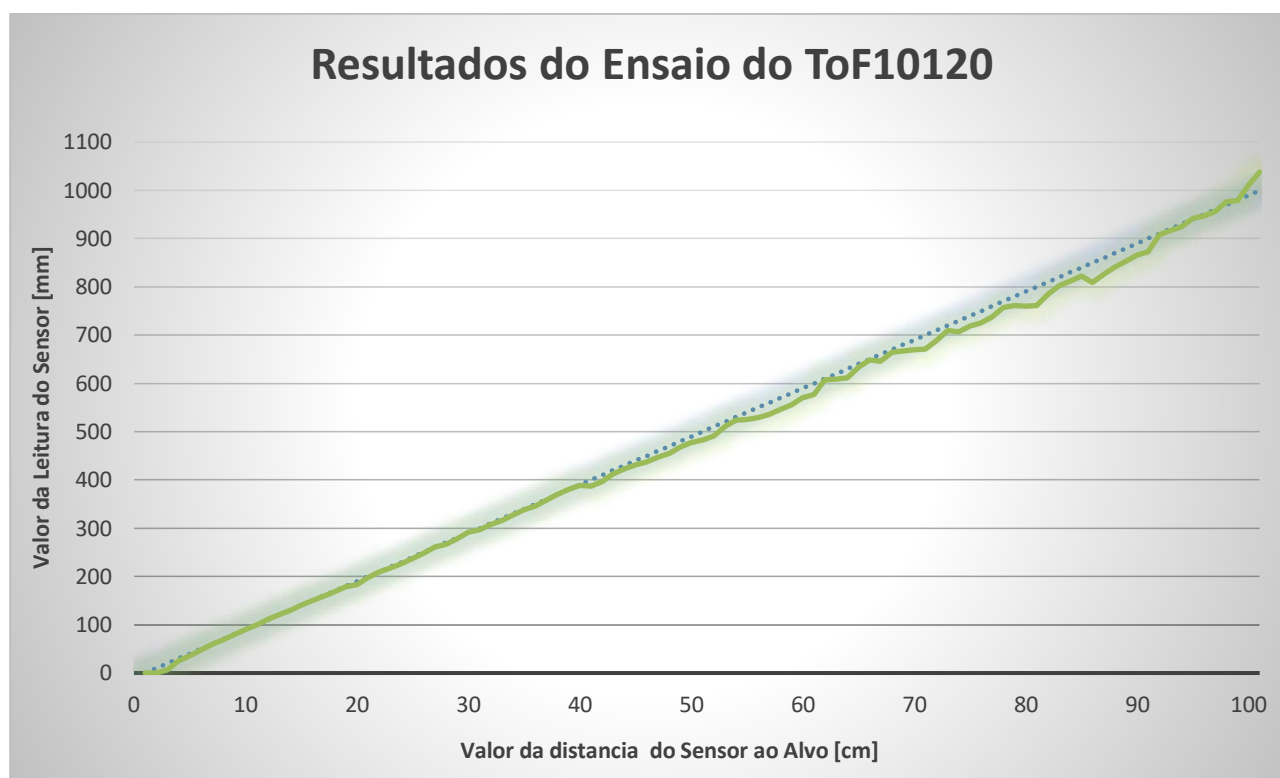


Fig. 45 – Ensaio com ToF10120

**Condições de teste:** timing budget = 33 ms, alvo branco 90%, luz ambiente = escuro.

Em média, existe um erro de 5%, tal como indica o Data Sheet. Por software aplicou-se filtros de modo a minimizar o erro, porque a partir de 500mm é bem visível.

#### 4.1.2. Pares Emissor-Recetor Infravermelho

O LED SFH4550 é utilizado em regime pulsado, tendo como vantagens: maior corrente admissível, maior intensidade do fluxo luminoso, e consequentemente maior sensibilidade e alcance. Segundo o fabricante, a corrente máxima admissível para um pulso de  $100 \mu s$  é de  $1 A$ , existindo uma queda de  $2.4 V$  aos terminais do mesmo. Se polarizarmos o LED de acordo com o esquema apresentado na Fig. 46, poderíamos no limite utilizar  $R = 2.6 \Omega$ . No entanto, operar no limite pode levar à sua destruição se o tempo de repouso não for respeitado. Para melhorar a fiabilidade do LED utilizámos  $R = 12 \Omega$ .



$$V_{CC} = V_f + RI \Leftrightarrow 5 = 1.7 + 12 \cdot I \Leftrightarrow I = \frac{5 - 1.7}{12} = 275 \text{ mA}$$

Considerando que em condições normais o SFH4550 consegue dissipar 180 mW como indica o fabricante, verificamos que para um pulso  $t_p$  de  $100 \mu s$ , num período  $T$  de  $1 ms$  não corremos o risco de danificar o componente.

$$V_f \cdot I \cdot \frac{t_p}{T} = 1.7 \times 275 \times \frac{100 \cdot 10^{-6}}{1 \cdot 10^{-3}} = 46.75 \text{ mW}$$

Fig. 46 – Polarização do LED

Comparando os valores observados no osciloscópio, com os valores observados no Serial Monitor provenientes da entrada analógica do ESP32 apercebemo-nos da inconsistência dos dados, principalmente na extremidade do alcance da deteção. A entrada analógica atingia o seu valor máximo quando a queda de tensão no foto-transistor atingia menos de 10%. Investigado o caso, rapidamente se deparamos com falta de linearidade da conversão ADC do ESP reportada de acordo com a Fig. 47

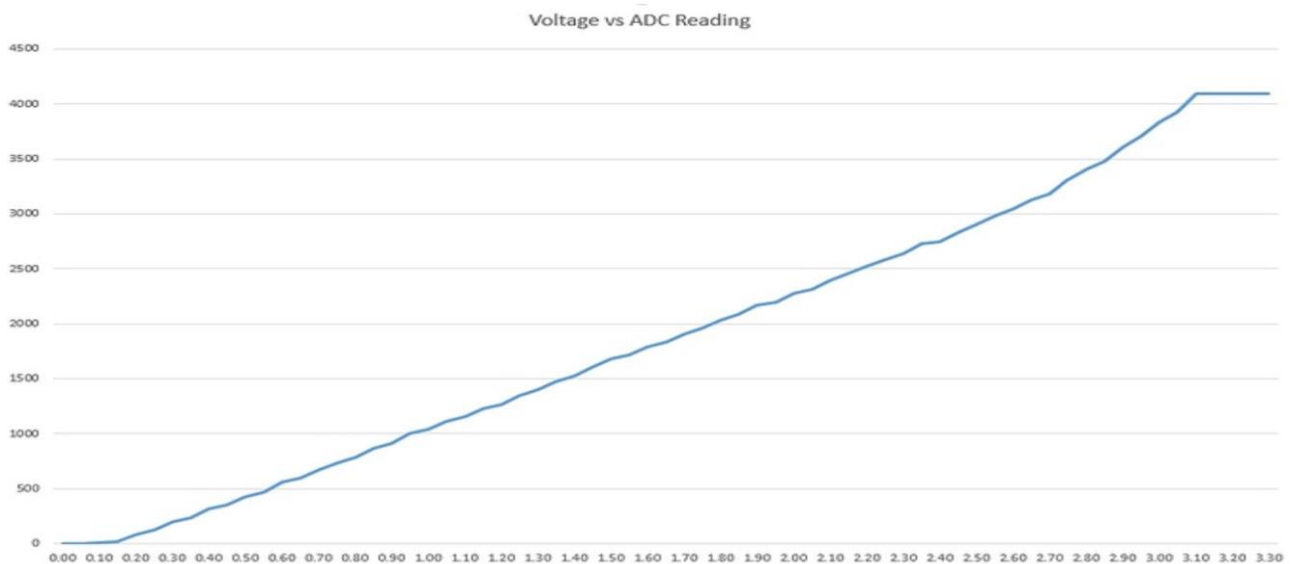
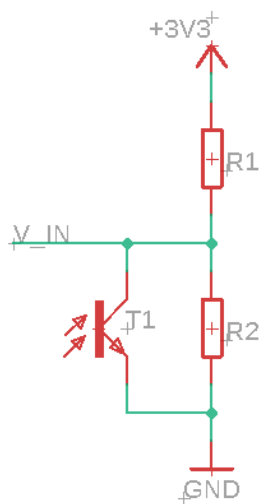


Fig. 47 – Gráfico dos valores lidos no ADC do ESP32<sup>45</sup>

A solução encontrada para impedir que a entrada analógica do microcontrolador atingisse a tensão de 3.3 V foi criar um divisor de tensão, onde no limite em que T1 está na zona de corte e comporta-se como circuito aberto a tensão  $V_{IN}$  é dada por:



$$V_{IN} = V_{cc} \cdot \frac{R_2}{R_1 + R_2} \Leftrightarrow R_2 = \frac{V_{IN} \times R_1}{V_{cc} - V_{IN}}$$

$$R_2 = \frac{3 \times 1.8}{3.3 - 3} = 18 \text{ k}\Omega$$

$$V_{IN} = \frac{V_{cc} \times R_2}{R_1 + R_2}$$

$$V_{IN} = \frac{3.3 \times 20}{1.8 + 20} = 3.027 \text{ V}$$

Fig. 48 – Divisor de Tensão na Entrada do ADC

Seguindo o exemplo da Globaltronic<sup>46</sup>, tomamos  $R_1 = 1.8 \text{ k}\Omega$ , o que faz algum sentido, para limitar corrente e consequentemente os consumos, e, no DataSheet do componente é referido que  $I_{c \text{ máx}} = 50 \text{ mA}$ , sendo que na pior situação, com o transistor saturado, limitamos a corrente a  $I_c$  a  $1.8 \text{ mA}$ , e garante o seu funcionamento na zona linear. Devido a stock existente, escolhemos  $R_2 = 20 \text{ k}\Omega$ , e com o transistor na zona de corte porque garante os 3 V máximos na entrada do ADC.

<sup>45</sup> Obtido em: <https://microcontrollerslab.com/adc-esp32-measuring-voltage-example/> - Acedido em Setembro, 2021

<sup>46</sup> Obtido em: <https://globaltronic.pt/> (desenvolvimento de eletrónica integrada )

O tempo despendido a efetuar uma leitura dos sensores de infravermelhos é uma informação de extrema importância. Para perceber a duração do efeito de polarização do foto-transistor monitorizou-se o estado de uma saída ativada logo após ativar a saída que controla LED, desligou-se passados  $70\ \mu\text{s}$  e iniciou-se a conversão ADC utilizado a função `analogRead()`, de seguida voltou-se a ligar a saída monitorizada e desligar o LED, o ciclo fica completo após uma pausa de 1ms. O ensaio foi observado no osciloscópio e registado tal como ilustra a Fig. 49.

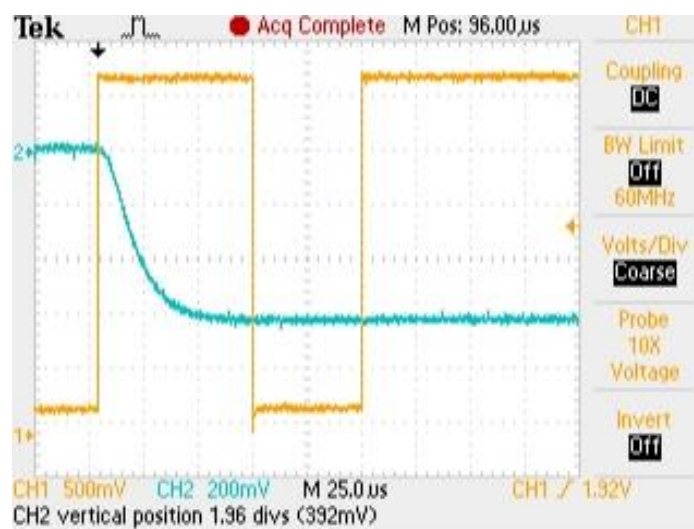


Fig. 49 – Duração da Polarização do Foto-Transistor e da Conversão ADC

Através da análise efetuada percebeu-se que, passados  $70\ \mu\text{s}$  o foto-transistor está totalmente polarizado, no entanto, para ter alguma margem de segurança, optou-se por ligar o LED  $100\ \mu\text{s}$  antes de efetuar a leitura da entrada analógica. Leitura essa que demora  $50\ \mu\text{s}$ , resultado num total de  $600\ \mu\text{s}$  necessários para efetuar as quatro leituras dos quatros sensores sequencialmente.

#### 4.1.3. Encoders

Se contarmos o número de vezes que um canal, A ou B, está alto durante uma volta do motor verificamos que são 3, devidos aos 3 pares de polos magnéticos existentes no disco. Considerando que os encoders utilizados têm 2 sensores de efeito de hall, e consequentemente 2 canais podemos contar 6 pulsos. E se tivermos uma rotina de interrupção que incrementa uma variável cada vez que existe uma mudança de estado nas entradas conhecidas como canal A e canal B temos 12 contagens por volta de motor, tal como ilustra a Fig. 50

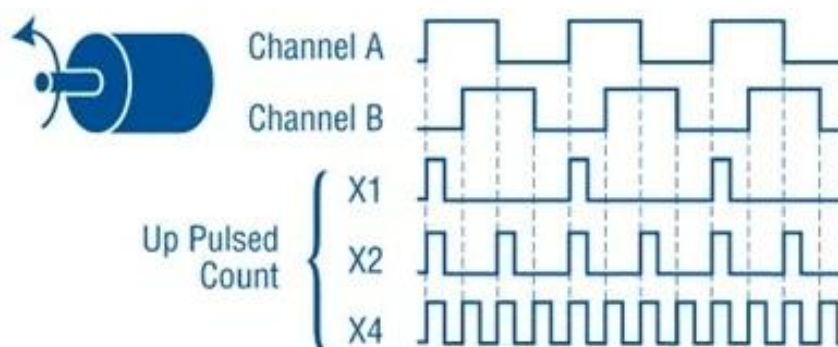


Fig. 50 – Contagem de Pulsos de Encoder em Quadratura<sup>47</sup>

<sup>47</sup> Obtido de: [https://www.dynapar.com/technology/encoder\\_basics/quadrature\\_encoder/](https://www.dynapar.com/technology/encoder_basics/quadrature_encoder/) - Acedido em Setembro, 2021

Considerando que a caixa redutora da versão do motor escolhido tem uma relação 30: 1, verificamos que em cada volta de roda contam-se  $12 \times 30 = 360$  pulsos de encoder. Tendo a roda 24 mm de diâmetro e 75.4 mm de perímetro, a resolução do robô é de  $75.4/360 = 0.2094$  mm por pulso. Fazendo o inverso ficamos a saber que por cada milímetro que uma roda anda, o encoder regista em média 4.77 pulsos.

## 4.2. Atuadores

Já sabemos como utilizar os sensores, mas ainda falta perceber como funcionam na realidade os motores, e que comportamento podemos esperar quando definimos determinado comando. Perceber o atraso na resposta e a frequência natural de oscilação ajudam-nos a aprimorar o controlo.

### 4.2.1. Motores

Para visualizar a Resposta a um comando Degrau, *Step Response*, definiu-se no programa o valor do comando, guardou-se em variáveis o número de impulsos de encoder em cada roda por cada instante igual ao período do timer que controla o *loop*, enviaram-se esses valores por comunicação serie e fez-se a representação gráfica em Excel de acordo com a seguinte

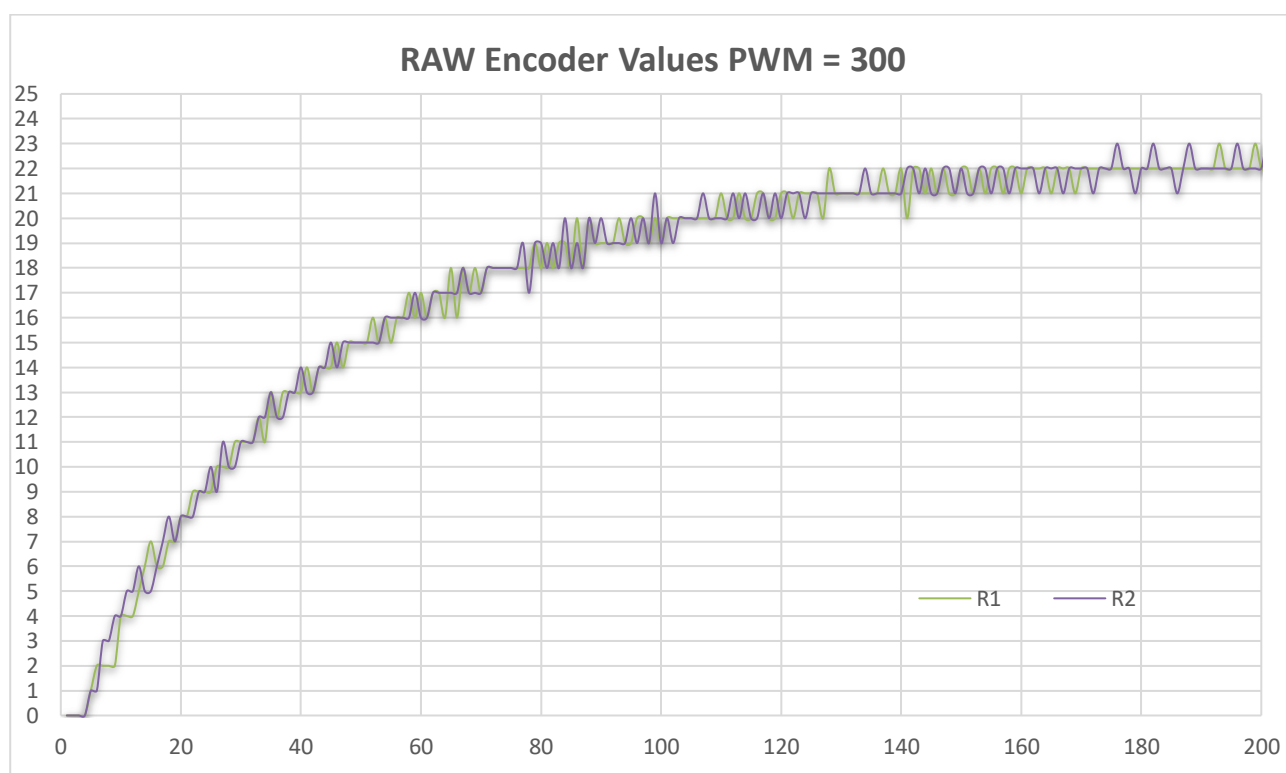


Fig. 51 – Valores Cruos de Pulsos de Encoder com PWM de 300 e TimeStep de 6 ms

Verifica-se uma variação abrupta que podemos chamar de ruído. Este ruído existe porque no instante em que a *flag* do *timer* “levanta” o polo magnético do disco do encoder pode estar mais próximo ou mais afastado do sensor que o deteta, o que faz com que à mesma velocidade existam ciclos que contam 21 pulsos, e noutros ciclos sejam contados 22.

Este ruído pode ser mitigado com a utilização de filtros passa baixo. Um exemplo de um filtro que consome poucos recursos de processamento é o filtro IIR.

Este tipo de filtro digital calcula a saída baseado não apenas nos sinais de entrada, mas também com base nas saídas anteriores. Ou seja, há um componente recursivo neste tipo de filtro, ele pode gerar uma saída mesmo na ausência de sinal de entrada, daí a qualidade "infinita". A ideia fácil, foi definir 2 coeficientes em que a soma é igual a 1, e de modo que o valor anterior tenha maior peso relativamente ao valor atual, dependendo disso, a celeridade da resposta. Por exemplo, coeficientes iguais tem uma resposta mais rápida com bastante ruído ainda, enquanto um maior peso do valor antigo retira mais ruído, devolve uma resposta mais suave, mas mais demorada.

$$Valor_{Filtrado} = V_{atual} \times Coef_{V_{atual}} + V_{anterior} \times Coef_{V_{anterior}}$$

#### 4.2.2. Resposta a Degrau

Em 6 ms, com PWM de 300, tendo como constantes o perímetro da roda de 75.4 mm, com caixa redutora de 30x, sabendo ainda que o encoder devolve 12 pulsos por cada volta que o motor gira, obtemos:

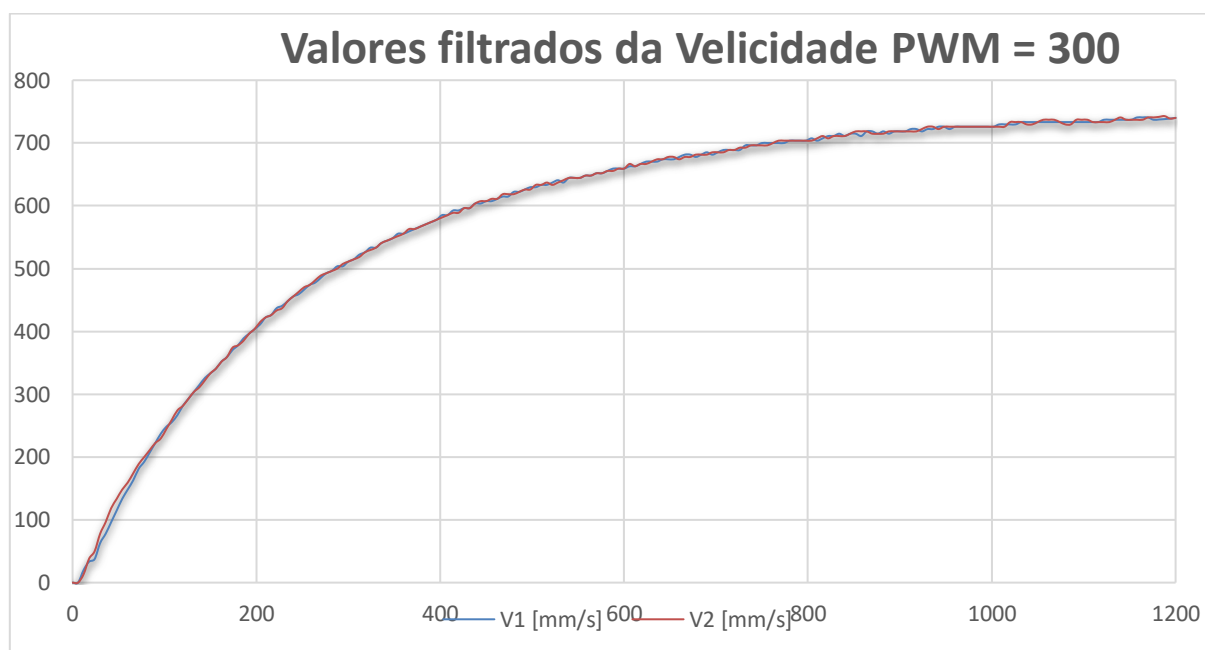


Fig. 52 – Gráfico da velocidade aplicando filtro IIR com PWM=300

Pela análise do gráfico, o rising time atinge 90% aos 800 ms (rising time = 0.8 s).

Através da observação deste gráfico de referência da Fig. 53, estimou-se um valor para o coeficiente de amortecimento de  $\zeta = 1,8$ .

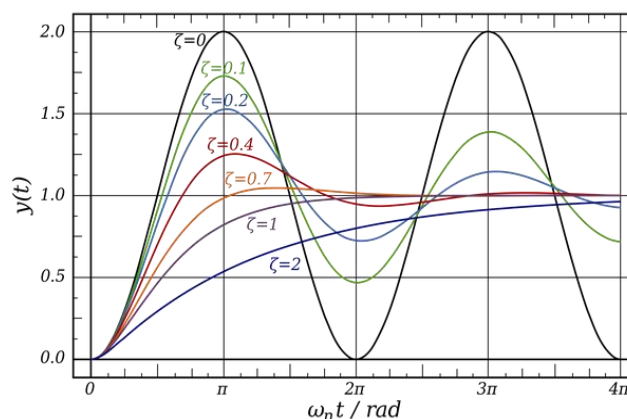


Fig. 53 – Gráfico de referência com vários coeficientes amortecimento  $\zeta$

A partir dos dados recolhidos torna-se possível definir um sistema de segunda ordem através dos seguintes cálculos:

$$\tau = 2\pi\xi = 11.31 \text{ rad}$$

$$\omega_n = \frac{\tau}{\text{rise time}} = \frac{11.31}{0.6} = 18.85 \text{ rad/s}$$

$$k_{DC} = \frac{\text{Velocidade(mm/s)}}{PWM} = \frac{777.94}{300} = 2.593$$

$$G(s) = k_{DC} \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2} = \frac{921.351}{s^2 + 2 \times 1.8 \times 18.85 \times s + 18.85^2}$$

Após o registo das várias *step response* para uma gama de input com intervalos de 100, obtivemos a linha gráfica seguinte:

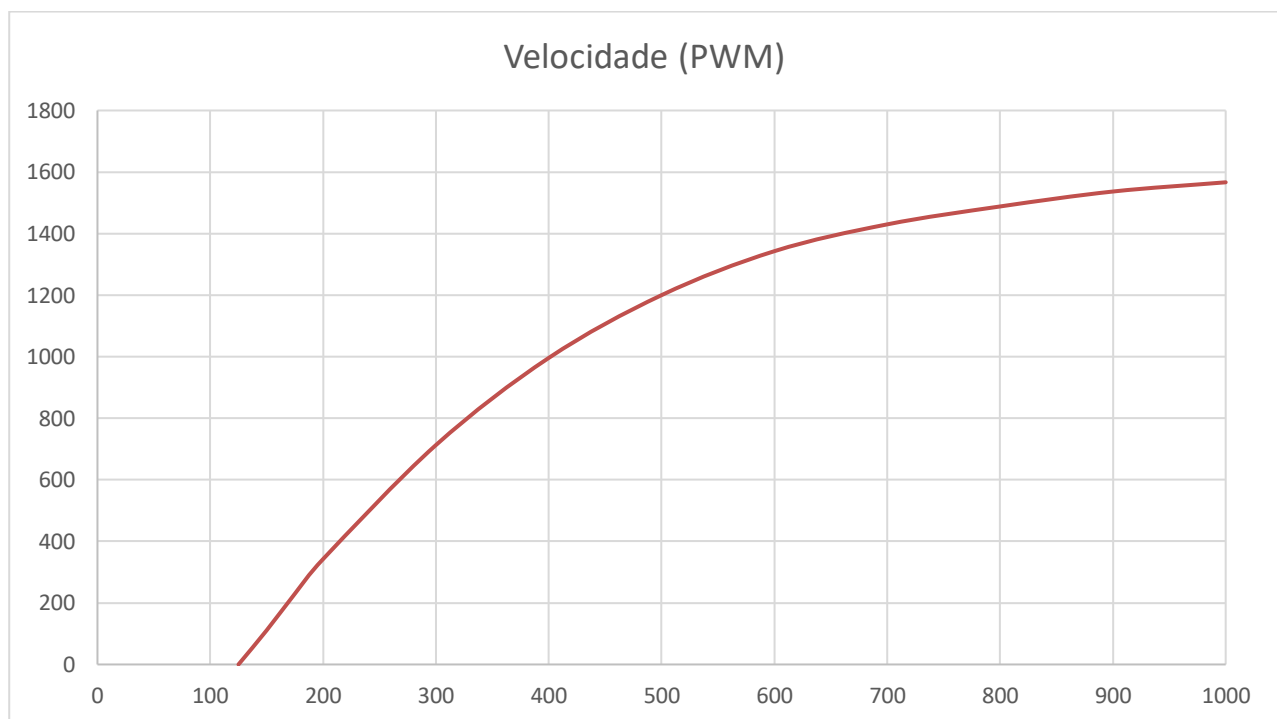


Fig. 54 – Gráfico de step response em função da velocidade

Invertendo a relação dos valores do gráfico anterior obteve-se a relação inversa da função apresentada no gráfico da Fig. 54, resultado esse apresentado de seguida no gráfico da Fig. 55.

Essa relação torna-se essencial para estimar o comando PWM necessário para atingir a velocidade alvo.

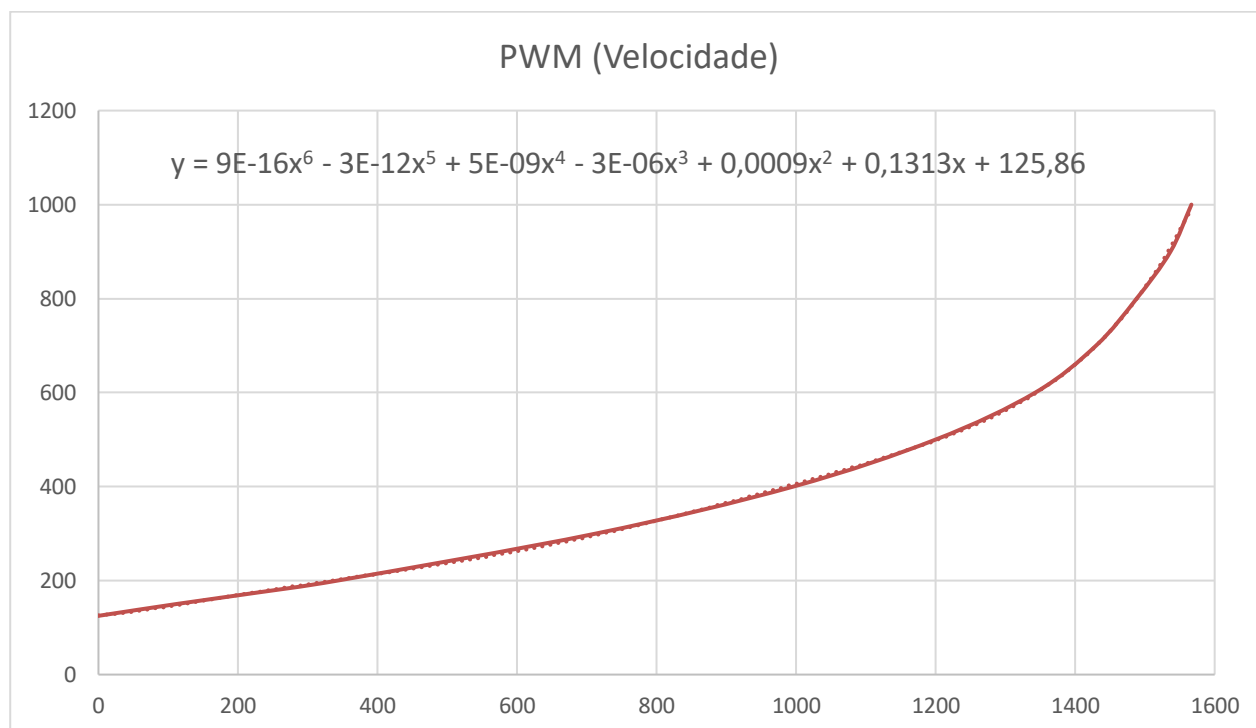
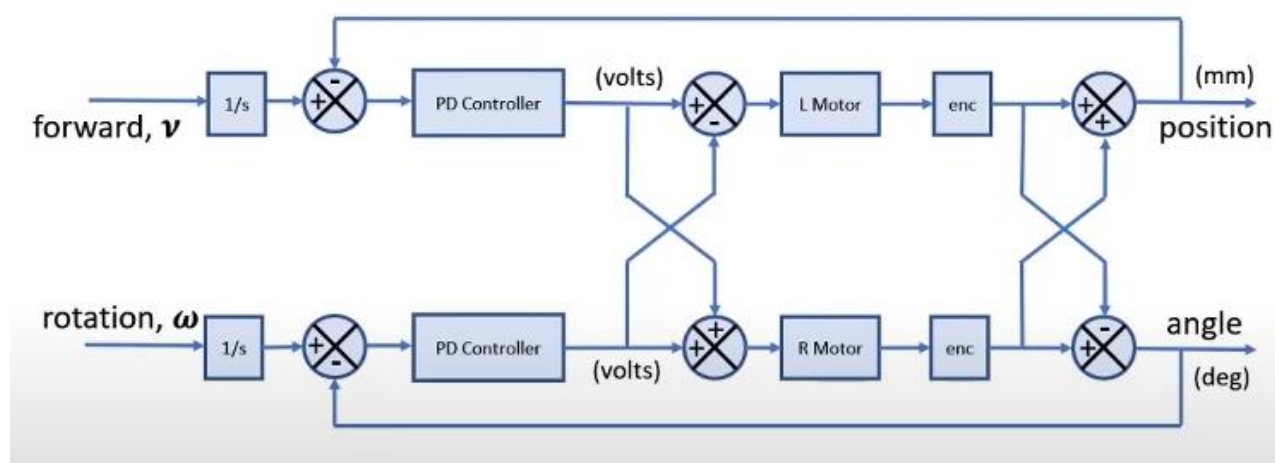


Fig. 55 – Gráfico de determinação da velocidade em função de PWM

De seguida criou-se uma regressão polinomial de 6º grau, obtiveram-se os coeficientes e criou-se uma tabela com os valores resultantes da função aplicada. Exportou-se os valores para um bloco de notas a fim de se formatar de acordo com um vetor em C para Arduino IDE para a parte do programa de controlo de velocidade.

#### 4.2.3. Controlo

Para controlar a velocidade dos motores e manter o robô paralelo às paredes quando está a andar em frente baseamo-nos no princípio de controlo utilizado pelo Peter Harisson.

Fig. 56 – Diagrama de Controlo dos Motores <sup>48</sup>

Utilizando as seguintes funções para pôr em prática o controlo estudado acima

<sup>48</sup> Obtido de: <https://www.youtube.com/watch?v=kLJyr8uiFg&t=1839s> – Acedido em Agosto, 2021

```
void controlarVelocidade(float targetSpeed) {

    calcularVelocidade();
    calcularAnguloParedes();

    float targetSpeed1 = targetSpeed + STEERING_ADJUST;
    float targetSpeed2 = targetSpeed - STEERING_ADJUST;

    erro10ld = erro1;
    erro1 = targetSpeed1 - V1[0];

    erro20ld = erro2;
    erro2 = targetSpeed2 - V2[0];

    vOut1 = PWM[(int)targetSpeed1] + FWD_KP * erro1 + FWD_KD * (erro1-erro10ld) ;
    vOut2 = PWM[(int)targetSpeed2] + FWD_KP * erro2 + FWD_KD * (erro2-erro20ld) ;
}

void calcularAnguloParedes (void) {

    erroEsqP = LEFT_CALIBRATION - Sensor[2][0];
    erroDirP = RIGHT_CALIBRATION - Sensor[3][0];

    erroEsqD = (Sensor[2][10] - Sensor[2][0]) / (V1[0]*LOOP_INTERVAL);
    erroDirD = (Sensor[3][10] - Sensor[3][0]) / (V2[0]*LOOP_INTERVAL);

    if (erroEsqP > 1000)
        erroEsqP = 1000;
    if (erroEsqP < -1000)
        erroEsqP = -1000;
    if (erroEsqD > 1000)
        erroEsqD = 1000;
    if (erroEsqD < -1000)
        erroEsqD = -1000;

    if (erroDirP > 1000)
        erroDirP = 1000;
    if (erroDirP < -1000)
        erroDirP = -1000;
    if (erroDirD > 1000)
        erroDirD = 1000;
    if (erroDirD < -1000)
        erroDirD = -1000;

    STEERING_ADJUST = STEERING_KP * erroEsqP + STEERING_KD * erroEsqD - STEERING_KP *
    erroDirP - STEERING_KD * erroDirD;

    if (STEERING_ADJUST > STEERING_ADJUST_LIMIT)
        STEERING_ADJUST = STEERING_ADJUST_LIMIT;

    if (STEERING_ADJUST < - STEERING_ADJUST_LIMIT)
        STEERING_ADJUST = - STEERING_ADJUST_LIMIT;
}
```

## 5. IMPLEMENTAÇÃO

Com o projeto calculado, fabricadas as placas PCB, montados os componentes, verificadas as soldas nos terminais dos componentes, ensaiado e testado, finalmente podemos começar a desenvolver o código do robô como um conjunto de rotinas de configurações iniciais e no ciclo infinito (o *loop* no Arduino) que contem as principais rotinas descritas de seguida.

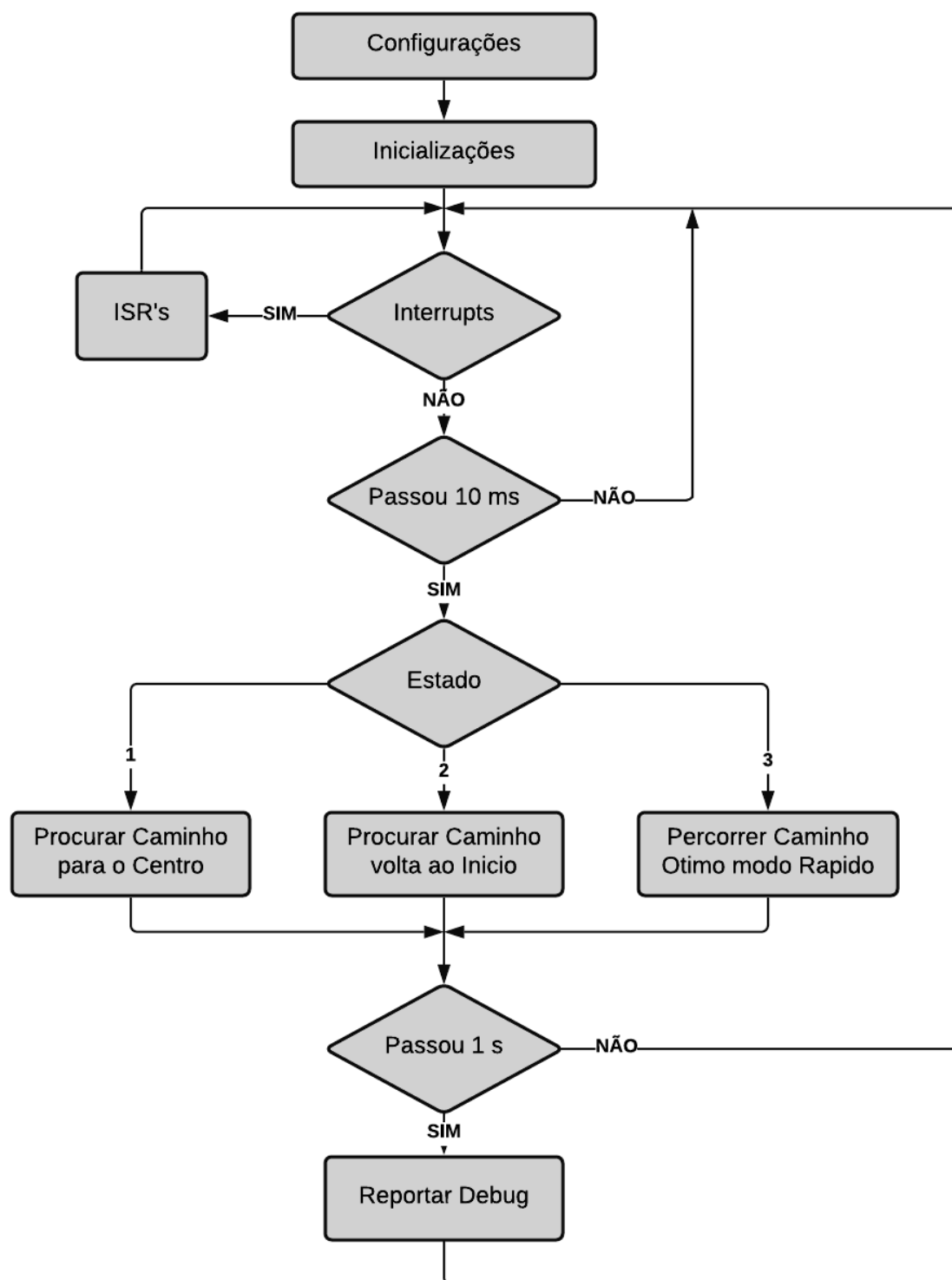


Fig. 57 – Fluxo Base do Programa Desenvolvido

Enquanto o estado for igual a 1 o micromouse encontra-se à procura do seu objetivo, que é o caminho para o centro do labirinto, o fluxo do programa na função de procurar o caminho para o centro foi reproduzindo na Fig. 58.

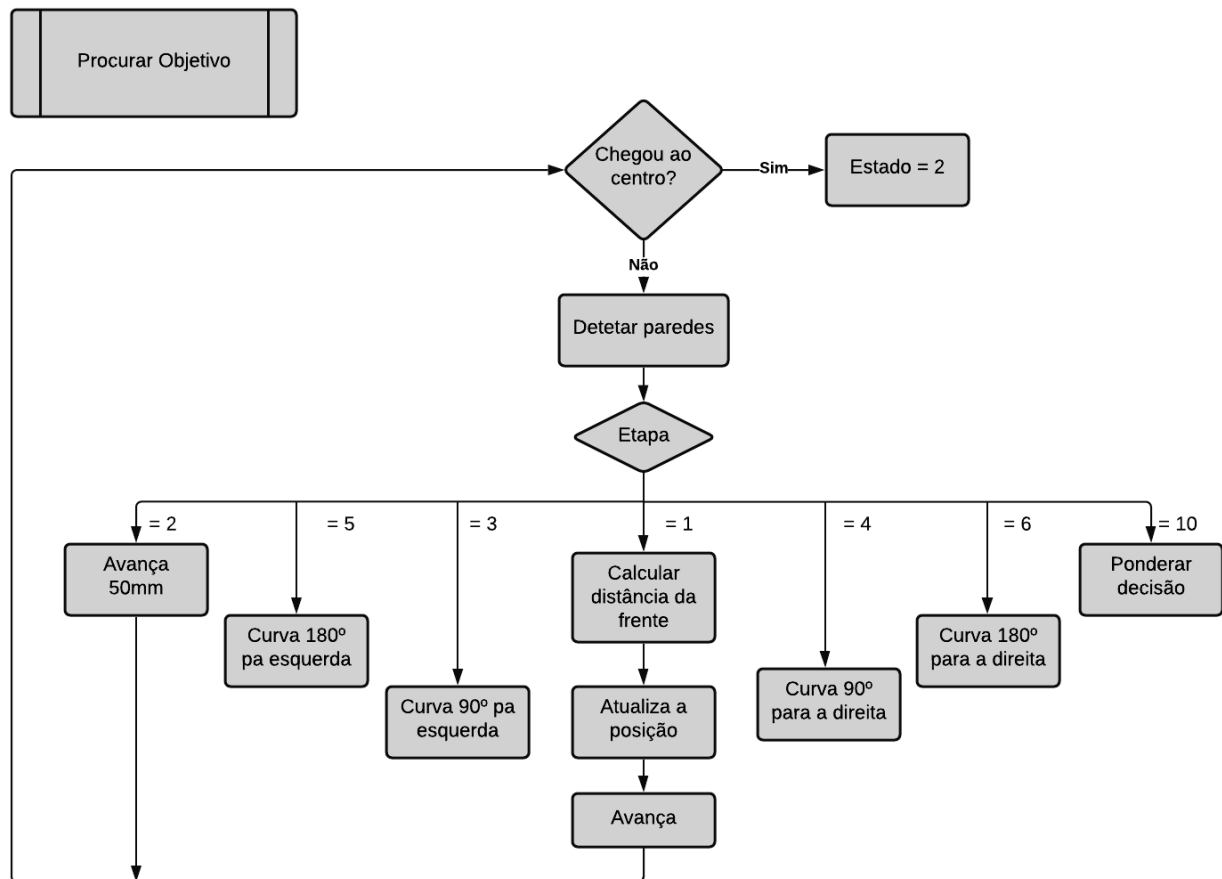


Fig. 58 – Fluxo da Função Procurar Objetivo

### 5.1. Deteções de Paredes

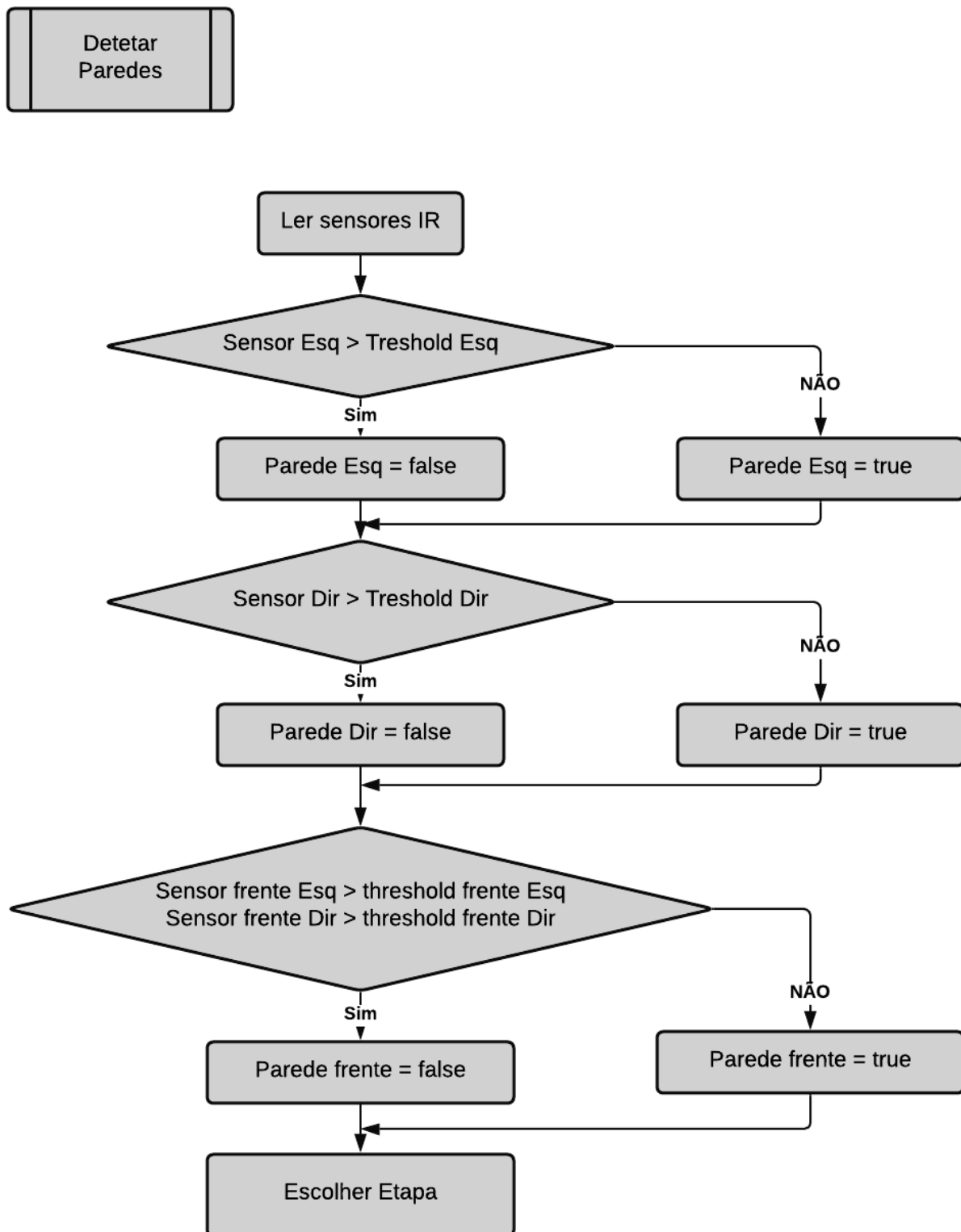
A leitura dos Sensores IR é feita da forma que está apresentada no excerto de código que se segue. Por razões de simplificação, as linhas expostas são replicadas para cada um dos sensores.

```

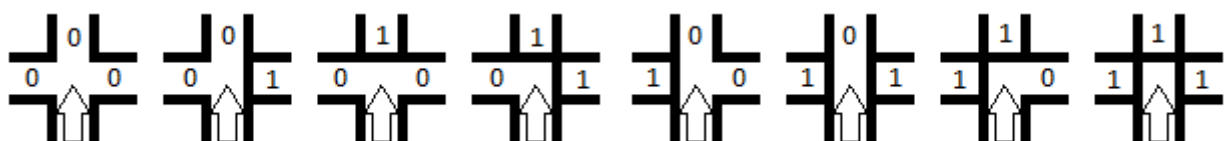
void IRread (void) {

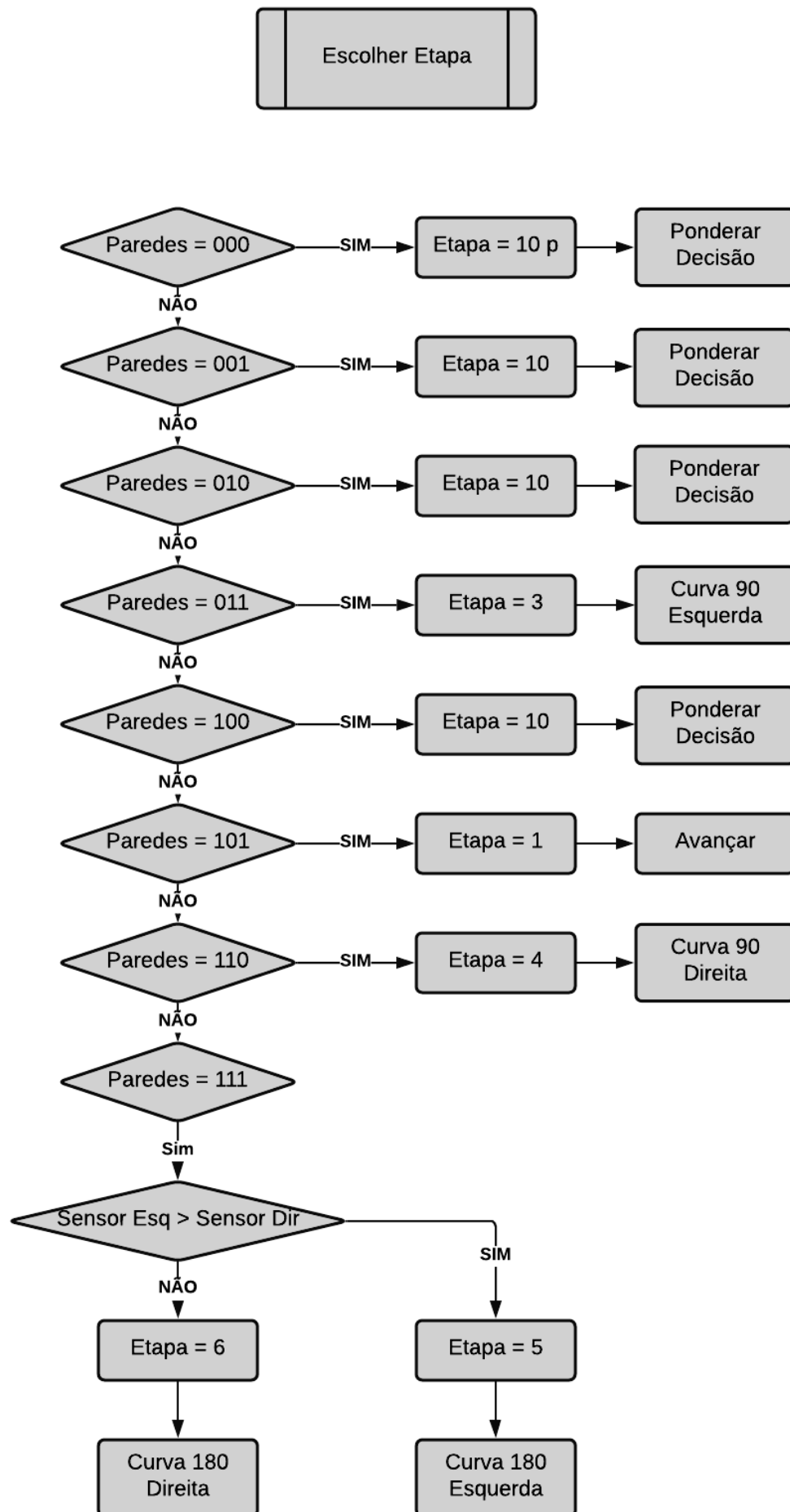
    digitalWrite(LED1, LOW);
    SensorDark[1] = analogRead(FT1);
    digitalWrite(LED1, HIGH);
    delayMicroseconds(100);
    SensorLight[1] = analogRead(FT1);
    digitalWrite(LED1, LOW);

    SensorOld[1] = SensorRead[1];
    SensorRead[1] = SensorLight[1] - (4095 - SensorDark[1]);
    Sensor[1] = pesoSensorAtual * SensorRead[1] + pesoSensorOld * SensorOld[1];
}
  
```

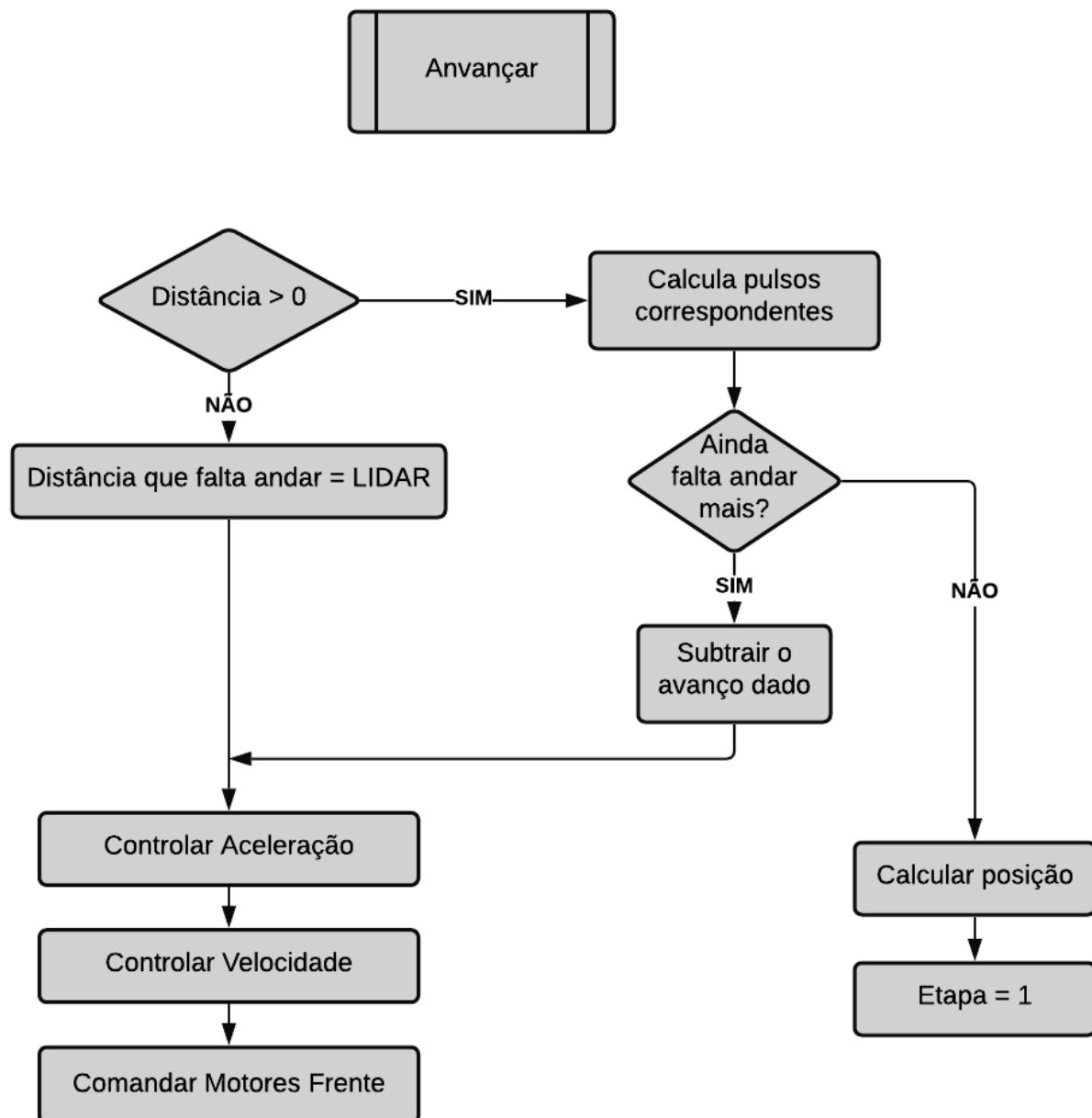


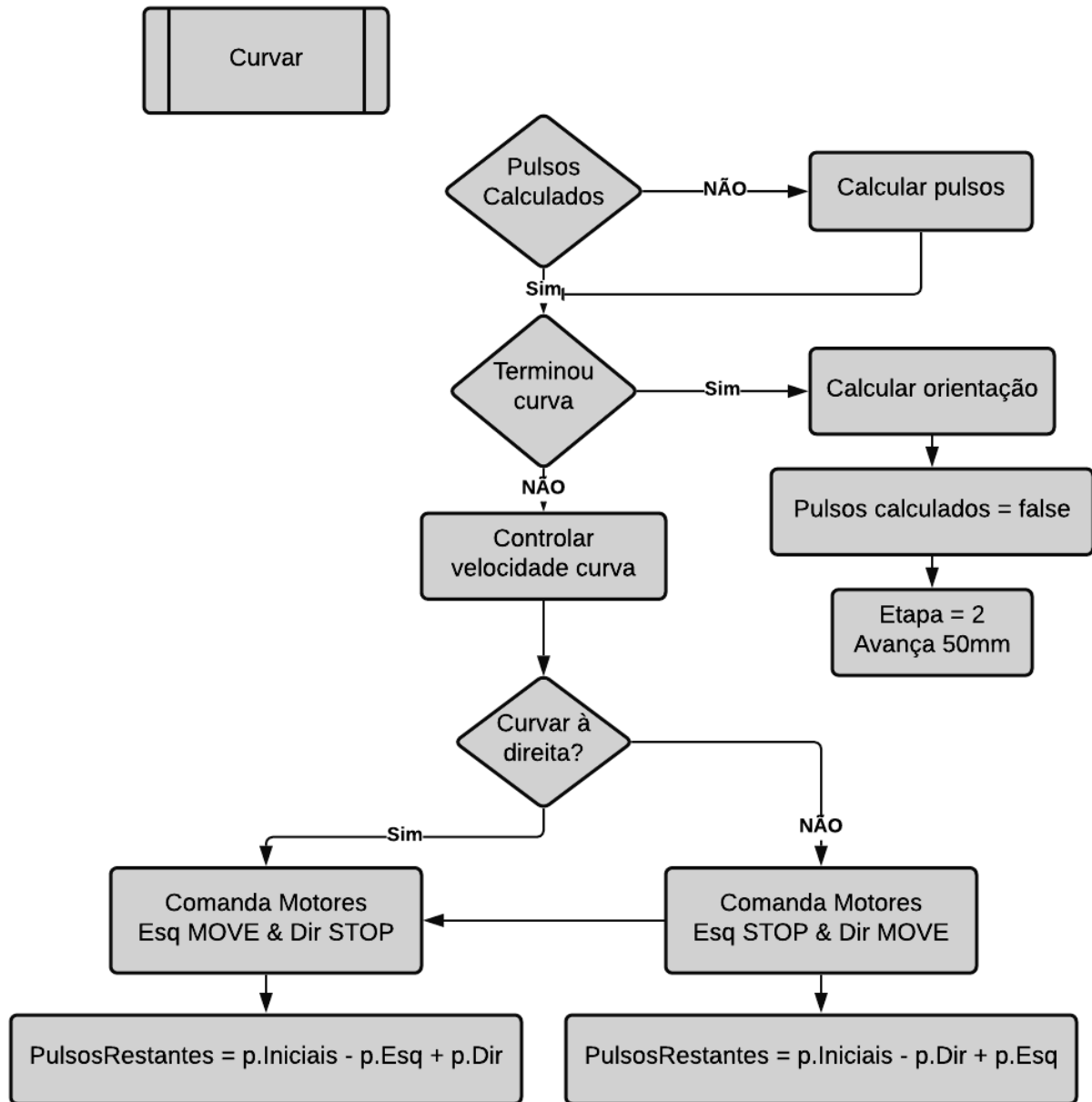
Considera-se que a escolha de etapa é feita de acordo com estas 8 combinações binárias das 3 paredes

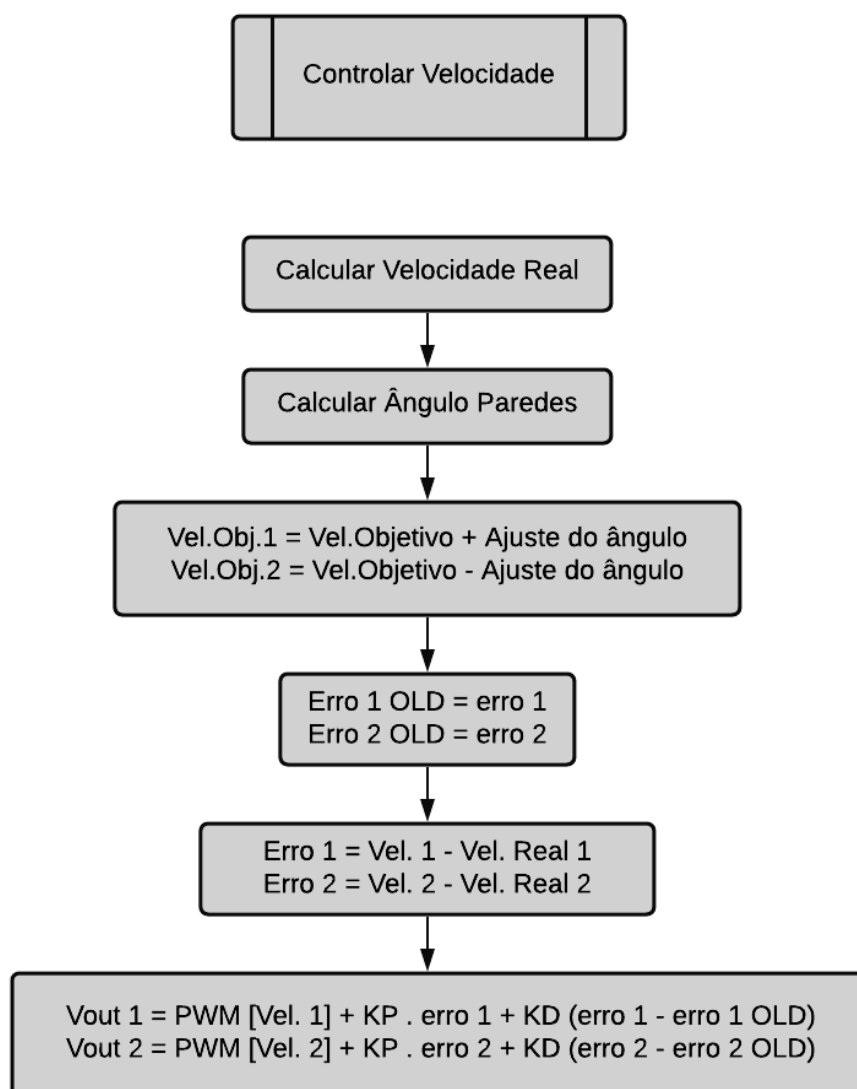




## 5.2. Movimentos do Robô







### 5.3. Localização do MicroMouse

A posição do micromouse é definida em coordenadas cartesianas ( x ; y ), sendo o início ( 0 ; 0 ) e o centro do labirinto ( 8 ; 8 ). É feita uma atualização de posição sempre que o robô percorre a distância correspondente a uma célula, ou seja 160 mm.

```

void calcPosicao (void) {
    if (orientacao == 'N') {
        posicaoY++;
    }
    if (orientacao == 'S') {
        posicaoY--;
    }
    if (orientacao == 'E') {
        posicaoX++;
    }
    if (orientacao == 'O') {
        posicaoX--;
    }
}

```

```
void calcOrientacao (int rotacao) {  
    if (orientacao == 'N'); {  
        if (rotacao == -2 || rotacao == 2)  
            orientacao = 'S';  
        else if (rotacao == -1)  
            orientacao = 'O';  
        else if (rotacao == 1)  
            orientacao = 'E';  
    }  
}
```

A logica deste exemplo repete-se para as restantes orientações.

#### 5.4. Navegação no Labirinto

Para implementar um algoritmo de navegação como sugere Peter Harisson<sup>49</sup> deve-se começar por definir dois mapas, um otimista, que começa com todas as paredes abertas, e outro mapa pessimista que é inicializado com todas as paredes fechadas. À medida que o micromouse vai explorando, esses mapas devem ser atualizados e uma nova execução do FloodFill deve ser iniciada. Assim que o percurso encontrado nos dois mapas for igual, a procura pelo percurso mais eficiente termina. Podendo então dar-se início às voltas em modo de alta velocidade.

#### 5.5. Simulação de Algoritmos

Para testar com facilidade, e observar rapidamente a eficácia dos algoritmos de procura de caminho e deliberação da rota mais curta foi desenvolvido um simulador em linguagem C numa aplicação de consola. O ambiente de interface tem o seguinte aspeto:



---

<sup>49</sup> Obtido de: <https://www.youtube.com/watch?v=WSik6XY2dhQ> – Acedido em Agosto, 2021

## 6. CONCLUSÕES

Este projeto foi começar do zero um robô autônomo e explorador de caminhos labirínticos, vendo os campeões e sonhando com esses momentos de ganhar, mas todo o contexto de distância social não deu mais tempo para reunir e distribuir tarefas, angariar material mais cedo. No entanto chegamos ao início da reta final, que nos permitia melhorar o desempenho do robô que temos e implementar a navegação pelos algoritmos conhecidos, e tentar um dia, ganhar uma competição. Na verdade, este projeto, poderá não ter fim, pois os melhores do mundo, depois de décadas de dedicação continuam a melhorar ano a após ano.

### 6.1. Conclusões Gerais

Foi gratificante com os conhecimentos adquiridos, aplicação dos saberes que obtivemos ao longo da licenciatura. Se o futuro tem sucessos garantidos a médio prazo, os robôs, os microcontroladores, os sensores, ... estão incluídos nesse conjunto. Assim, aprendemos como construir o futuro, para ir ao encontro das necessidades das pessoas e melhorar-lhes a vida nas suas necessidades. Ainda não sabemos tudo, porque os ‘saberes’ são infinitos e progressivos, mas tivemos um grande impulso.

### 6.2. Conclusões Específicas

No ESP32 existem 10 canais analógicos disponíveis no ADC2, mas estes canais são compartilhados por outros recursos. O ADC2 é compartilhado com drivers WiFi, portanto, inicialmente os IR foram ligados, o que deu problemas quando tentamos carregar o código e obter leituras dos sensores por WiFi. Além da ocupação de memória para muito código, também o gasto das baterias aumentou drasticamente.

A luz ambiente ou luz solar podem causar interferências em sinais infravermelhos. O sol emite muita luz infravermelha, é uma das responsáveis para aquecer a terra. Existe um problema na transmissão de dados via luz infravermelha, que é a interferência da luz infravermelha do ambiente, e causa diferenças de leitura para a mesma distância ao objeto. Assim, os valores do sensor infravermelho normalmente flutuam em condições variantes de luz.

A navegação e implementação dos algoritmos como Flood-Fill, que é talvez o mais adequado a este projeto, mas também existem Astar (A\*) e ainda outros<sup>50</sup>.

### 6.3. Conclusões Finais

Do trabalho a desenvolver no futuro, destaca-se:

- Melhor calibração dos sensores
- Otimizações dos algoritmos e inclusão dos dados de um acelerómetro
- Otimização da rede de sensores com a inclusão de 6 sensores IR
- Conclusão do protótipo a nível da estrutura
- Um sistema motriz de 4 rodas
- Ventilador de sucção
- Bateria mais compacta

---

<sup>50</sup> Consultar: [https://en.wikipedia.org/wiki/Maze-solving\\_algorithm](https://en.wikipedia.org/wiki/Maze-solving_algorithm) - Acedido em Setembro, 2021



## REFERÊNCIAS

---

- Dowson, D. (1997). *History of Tribology* (2 ed.). Publicação Profissional de Engenharia. Retrieved from [https://pt.wikipedia.org/wiki/Atrito#cite\\_note-:1-12](https://pt.wikipedia.org/wiki/Atrito#cite_note-:1-12)
- Euler, L. (2002). *Friction Module*.
- Irodov, I. E. (1988). *Problems in General Physics*. Moscow: Mir Publishers of Moscow.
- Makkonen, L. (2012). *A thermodynamic model of sliding friction* (1 ed.). AIP Advances. doi:10.1063/1.3699027
- Palma, D. S. (2008). *Domótica KNX/EIB de Baixo Custo*. Porto: Universidade do Porto, Faculdade de Engenharia.
- Resnick, R., & Halliday, K. (2003). *Física 1*. doi:ISBN 8521613520
- UK Micromouse Maze Solver Rules. (2000). Retrieved from [https://www.cs.york.ac.uk/micromouse/Rules/Maze\\_Solver\\_Rules.pdf](https://www.cs.york.ac.uk/micromouse/Rules/Maze_Solver_Rules.pdf)
- WALKER, J. (2016). *Fundamentos de Física* (10 ed.). Rio de Janeiro. doi:ISBN 978-85-216-3035-7



## ANEXOS

---

/\* Elaborado por Pedro Daniel Rocha ISEC 2021

Esta versão do programa conta com :

- Detecção binária de paredes por sensores IR
- Decisão de curvar em ultimo caso, caso possa, prefere seguir em frente
- Controlo de Velocidade PID com ganhos DESAJUSTADOS
- Controlo da trajetoria curvilinea por medição de pulsos de encoder

correspondentes

- Tabela de correspondencia da função transferencia  $H(s)$  em função do SetPoint

imposto

- Rampa de Aceleração controlada, evitando potenciais BurnOuts, e introdução

de erros

- Medição de distancia de objetos na frente atraves do LIDAR
- Capacidade de localizar a celula ocupada, nao reconhecendo ainda o centro

como objetivo

- Funções de teste, medição e calibração de entradas e saidas comentadas no

loop()

\*/

// \*\*\*\*\* BIBLIOTECAS \*\*\*\*\*

#include <math.h>

#include <pgmspace.h>

#include <Wire.h>

#include <vl53lx\_class.h>

#include "Tabelas.h"

#include "1\_Config.h"

#include "8\_Maze.h"

//#include <EEPROM.h>

// \*\*\*\*\* PROTOTIPOS \*\*\*\*\*

void move\_Motors(int direcao);

void controlarVelocidade(float targetSpeed);

void controlarAceleracao(float top\_speed, float final\_speed, float acceleration);

void controlarCurva ( float pulsosLeft, float targetSpeed);

void calcularVelocidade(void);

void calcularAnguloParedes (void);

void spin\_turn(float degrees, float speed, float acceleration);

void stop\_at(float position);

void avance (float distance);

void turn(int sentido, float angle);

void turnBack (int sentido, int graus);

void invert (int sentido);

void zerarErro (void);

void calcularDistancia (void);

void calcPosicao (void);

void calcOrientacao (int rotacao);

void atualizarPosicao (void);

void atualizarOrientacao (void);

void IRread (void);

int lidarRead (void);

int detetarParedes (void);

void run\_maze (void);

```
void search_run (void);
void speed_run (void);
void end_run (void);

void configs (void);
void calibrarSensores (void);
void lidarConfig (void);
void monit_sensor(void);
void monit_motors(void);
void systemModel(void);
void initialize (void);

// ***** INTERRUPTCOES *****

void interrupt1() {
    encoder1++; pulsosT++; pulsosD1++;
}
void interrupt2() {
    encoder2++; pulsosT++; pulsosD2++;
}
void IRAM_ATTR onTimer() {
    portENTER_CRITICAL_ISR(&timerMux);
    interruptCounter++;
    portEXIT_CRITICAL_ISR(&timerMux);

    vRead1old = vRead1; vRead1 = encoder1; encoder1 = 0;
    vRead2old = vRead2; vRead2 = encoder2; encoder2 = 0;
}
void IRAM_ATTR intaISR() {
    portENTER_CRITICAL(&syncINTA);
    intaINT = true; // Signal that MCP INTA
occured
    portEXIT_CRITICAL(&syncINTA);
}

void setup() {

    configs();
    initialize();
    delay(1000);
    // detetar objeto em movimento para iniciar a prova

}

void loop() {

    if (interruptCounter > 0) {
        portENTER_CRITICAL(&timerMux);
        interruptCounter--;
        portEXIT_CRITICAL(&timerMux);
        totalInterruptCounter++;
        partialInterruptCounter++;

        //if (lidarRead()<30)
        // move_MM(0);

//////////////////////////////////// CALIBRAÇÕES
        if (etapa <= 0) {
            //calibrarSensores()
            //monit_sensor(); // ler os sensores IR por Serial Monitor
            //monit_motors(); // monitorizar e controlar os motores por COM Serie
            //systemModel(); // Obter a resposta a degrau do sistema
        }
    }
}
```

```

    etapa = 1; state = 1;
}

if (state == 1 )
    search_run ();

//run_maze ();
//speed_run ();
//end_run ();

// PLOTTER
//Serial.print(V1[0]); Serial.print("\t"); Serial.println(V2[0]);
}

////////// EXECUTAR A CADA 1 SEGUNDO
if (partialInterruptCounter * cicle >= 1000000) {

    debug();

    partialInterruptCounter = 0;
}
}

// ***** PIN OUT
*****

//+++++ ENCODER
#define A1 36
#define B1 39
#define A2 34
#define B2 35
//+++++ MOTORES
#define nSLP1 32
#define PH1 33
#define EN1 25
#define nSLP2 26
#define PH2 27
#define EN2 12
#define PWM1 1
#define PWM2 2
//+++++ IR
#define LED1 19
#define LED2 18
#define LED3 5
#define LED4 21
#define FT1 4
#define FT2 14 // 0
#define FT3 2
#define FT4 15
//+++++ LIDAR
#define INT 23
#define XSHUT 22
#define SCL 1
#define SDA 3

#define DEVICE_I2C_ADDR 0x12
VL53LX v153lx(&Wire, XSHUT);

// Holds latest measured distances (up to 4)
#define MAX_OBJECTS 1
uint16_t dist[MAX_OBJECTS];

```

```
portMUX_TYPE syncINTA = portMUX_INITIALIZER_UNLOCKED;        // For handling ISR
entry/exit
bool          intaINT  = false;                                // True = Interrupt
occured
unsigned long intervalTimer;                                   // Timer used for
checking for lost interrupts

//+++++
+++++

int mapWide = 16;                                              // dimensao do labirinto

//+++++ MECHANICS
+++++

const float WHEEL_PERIMTER = 73.827;
const float ENCODER_PULSES = 12.0;
const float GEAR_RATIO = 30.0;
const float MILIMETERS_OVER_PULSES = WHEEL_PERIMTER / (ENCODER_PULSES * GEAR_RATIO);

const float MOUSE_RADIUS = 38.5;
const float PULSES_OVER_DEGREES = ( MOUSE_RADIUS * 4 * PI ) / ( 360 *
MILIMETERS_OVER_PULSES );
const float ROTATION_OFFSET = 0.000; // Negative makes robot curve to left

//***** MOTION CONTROL CONSTANTS
*****//

float kDC = 0;
// forward motion controller constants
const float FWD_KP = 2.0;
const float FWD_KD = 1.1;

// rotation motion controller constants
const float ROT_KP = 1.4;
const float ROT_KD = 2.2;
const float ROT_KI = 0.1;

// controller constants for the steering controller
const float STEERING_KP = 0.065;
const float STEERING_KD = 0.375;
const float STEERING_ADJUST_LIMIT = 200.0;

//***** PERFORMANCE CONSTANTS
*****//
// search and run speeds in mm/s and accel mm/s2
const float DEFAULT_TURN_SPEED = 350;
const float DEFAULT_SEARCH_SPEED = 450;
const float DEFAULT_MAX_SPEED = 800;
const float DEFAULT_SEARCH_ACCEL = 1000;

//***** SENSOR CALIBRATION
*****//
// wall sensor thresholds and constants
// RAW values for the front sensor when the robot is backed up to a wall
const int FRONT_L_CALIBRATION = 2700;
const int FRONT_R_CALIBRATION = 2800;
// RAW values for the side sensors when the robot is centred in a cell
// and there is no wall ahead
const int LEFT_CALIBRATION = 2250;
```

```

const int RIGHT_CALIBRATION = 2250;

// This is the normalised value seen by the front sensor when the mouse is
// in its calibration position
const int LEFT_NOMINAL = 100;
const int FRONT_L_NOMINAL = 100;
const int FRONT_R_NOMINAL = 100;
const int RIGHT_NOMINAL = 100;

// Sensor brightness adjustment factor. The compiler calculates these so it saves
// processor time
const float FRONT_L_SCALE = (float)FRONT_L_NOMINAL / FRONT_L_CALIBRATION;
const float FRONT_R_SCALE = (float)FRONT_R_NOMINAL / FRONT_R_CALIBRATION;
const float LEFT_SCALE = (float)LEFT_NOMINAL / LEFT_CALIBRATION;
const float RIGHT_SCALE = (float)RIGHT_NOMINAL / RIGHT_CALIBRATION;

// the values above which, a wall is seen
const int LEFT_THRESHOLD = 3000; // minimum value to register a wall
const int FRONT_L_THRESHOLD = 3400; // minimum value to register a wall
const int FRONT_R_THRESHOLD = 3400; // minimum value to register a wall
const int RIGHT_THRESHOLD = 3300; // minimum value to register a wall
const int FRONT_L_REFERENCE = 2144; // reading when mouse centered with wall ahead
const int FRONT_R_REFERENCE = 2312; // reading when mouse centered with wall ahead

//*****
//*****//

// with robot against back wall, how much travel is there to the cell center?
const int BACK_WALL_TO_CENTER = 48;

//*****
//*****//

// Control loop timing. Pre-calculate to save time in interrupts
const float LOOP_FREQUENCY = 200.0;
const float LOOP_INTERVAL = (1.0 / LOOP_FREQUENCY);

//*****
//*****//

int vMax = 500, vMin = 135 ; // Limites maximos e minimos de PWM out

long encoder1 = 0, encoder2 = 0; // utilizado para
// feedback de velocidade
double pulsos = 0, pulsosT = 0, pulsosD1 = 0, pulsosD2 = 0; // utilizado para
// calcular distancias, fazer o tracking, e medir distancias percorridas em curvas
float remaining = 0 , erro = 0 , erroOld = 0 , erroAcumulado = 0; // utilizado no
// controlo das curvas
float erroEsqP = 0, erroEsqD = 0, erroDirP = 0, erroDirD = 0; // utilizado no
// calculo do ajuste de distancia as paredes

float STEERING_ADJUST = 0;

float erro1 = 0, erro1Old = 0, erro1Acumulado = 0; // utilizado para
// controlo de velocidade PID do motor 1
float erro2 = 0, erro2Old = 0, erro2Acumulado = 0; // utilizado para
// controlo de velocidade PID do motor 2
long vRead1 = 0, vRead2 = 0, vRead1Old = 0, vRead2Old = 0; // Pulsos de
// Encoder por unidade de tempo ( periodo de interrupção )
long vOut = 0, vOut1 = 0, vOut2 = 0 , vOut1OLD = 0, vOut2OLD = 0; // Valor
// referencia, historico, Ajustado para o motor 1 e motor 2 respetivamente

```

```
float pesoEncoderAtual = 0.05, pesoEncoderOld = 1 - pesoEncoderAtual;    // filtro IIR
float V1[10], V2[10];

float m_speed = 0, m_target_speed = 0, m_final_speed = 0, m_one_over_acc =
1/DEFAULT_SEARCH_ACCEL;          // rampa de aceleração

int posicaoX = 0, posicaoY = 0, sentido = 0;    // utilizado para o Tracking,
localizando a célula ocupada pelo MM
char orientacao = '0';

//+++++ Sensores
+++++

float pesoSensorAtual = 0.1, pesoSensorOld = 1 - pesoSensorAtual;
float Sensor[5][10];
float SensorDark[5], SensorLight[5], SensorRead[5], SensorOld[5];          //
utilizado na leitura dos sensores IR
bool paredeFrente, paredeEsq, paredeDir;          // existencia de paredes

long distancia = 0, distTotal = 0;
int distanciaFrente = 0, distanciaEsq = 0, distanciaDir = 0; // distancia as paredes

//+++++ Controlo de Fluxo
+++++

bool done = false, doneT = false, doneF = false;          // utilizado para evitar
executar repetidamente determinadas ações
int etapa = 0 , state = 0;          // utilizado para controlar o
fluxo de tarefas no loop()

//+++++ COM Serie
+++++

int valor, consolaAux, reads[1000][3];          // utilizado para comunicação no Monitor
Serie
int amostra = 200;          // numero de valores recolhidos para
modelizar a resposta a degrau

//+++++ Cadencia
+++++

float cicle = 1000000 * LOOP_INTERVAL;    // us

volatile int interruptCounter;
int totalInterruptCounter, partialInterruptCounter;
hw_timer_t * timer = NULL;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;

//+++++
+++++

typedef struct {
    bool N;
    bool S;
    bool E;
    bool O;
} maze;

maze mOpen [mapWide][mapWide] ;
maze mClose [mapWide][mapWide] ;
```

```

// ***** INICIALIZACOES *****
void initialize (void) {

    for ( int i; i < mapWide; i++) {
        for ( int j; j < mapWide; j++) {
            mOpen[i][j].N=false;
            mClose[i][j].N=true;
            mOpen[i][j].S=false;
            mClose[i][j].S=true;
            mOpen[i][j].E=false;
            mClose[i][j].E=true;
            mOpen[i][j].O=false;
            mClose[i][j].O=true;
        }
    }

    digitalWrite(EN1, LOW);                // garantir que os motores comecem
    parados                                // garantir que os motores comecem
    digitalWrite(EN1, LOW);

    posicaoX = 0; posicaoY = 0;                // posição inicial ( 0 ; 0 )

    etapa = 0;
    state = 0;
    distanciaFrente = 200;                  // 2 celulas = 320 mm
    totalInterruptCounter = 0;
    zerarErro();

    delay(1000);
}

void configs (void) {

    pinMode(EN1, OUTPUT);
    pinMode(EN2, OUTPUT);
    pinMode(PH1, OUTPUT);
    pinMode(PH2, OUTPUT);
    pinMode(nSLP1, OUTPUT);
    pinMode(nSLP2, OUTPUT);
    pinMode(A1, INPUT);
    pinMode(B1, INPUT);
    pinMode(A2, INPUT);
    pinMode(B2, INPUT);
    pinMode(FT1, INPUT);
    pinMode(FT2, INPUT);
    pinMode(FT3, INPUT);
    pinMode(FT4, INPUT);
    pinMode(LED1, OUTPUT);
    pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    pinMode(LED4, OUTPUT);
    pinMode(INT, OUTPUT);
    pinMode(XSHUT, OUTPUT);
    pinMode(SCL, OUTPUT);
    pinMode(SDA, OUTPUT);

    ledcAttachPin(nSLP1, PWM1);
    ledcAttachPin(nSLP2, PWM2);
    ledcSetup(PWM1, 1000, 10);
    ledcSetup(PWM2, 1000, 10);
    //          canal , HZ , bits

```

```
Serial.begin(115200);

timer = timerBegin(0, 80, true);
timerAttachInterrupt(timer, &onTimer, true);
timerAlarmWrite(timer, cicle , true);
timerAlarmEnable(timer);

attachInterrupt(digitalPinToInterrupt(A1), interrupt1, CHANGE);
attachInterrupt(digitalPinToInterrupt(B1), interrupt1, CHANGE);
attachInterrupt(digitalPinToInterrupt(A2), interrupt2, CHANGE);
attachInterrupt(digitalPinToInterrupt(B2), interrupt2, CHANGE);

//lidarConfig();

}

void lidarConfig (void)
{
  int i;
  VL53LX_Error rc;
  VL53LX_Version_t ver;
  uint8_t ProductRevisionMajor, ProductRevisionMinor, byteData;
  VL53LX_DeviceInfo_t devInfo;
  uint64_t Uid;
  /*
    Serial.begin(SERIAL_BAUD_RATE, SERIAL_8N1);
    while (!Serial);
    delay(5000);
    Serial.println("Starting...");
  */
  // Initialize the array
  for (i = 0; i < MAX_OBJECTS; i++)
    dist[i] = 0;

  // Initialize the I2C bus
  //Wire.begin();
  Wire.begin(SDA, SCL);
  Wire.setClock(400000);

  // Configure the VL53LX
  vl53lx.begin();
  vl53lx.VL53LX_Off();
  vl53lx.InitSensor(DEVICE_I2C_ADDR);

  // Print chip info
  rc = vl53lx.VL53LX_GetVersion(&ver);
  Serial.printf("GetVersion: rc=%d, Build=%d, Major=%d, Minor=%d, Revision=%d\n", rc,
ver.build, ver.major, ver.minor, ver.revision);
  rc = vl53lx.VL53LX_GetProductRevision(&ProductRevisionMajor, &ProductRevisionMinor);
  Serial.printf("GetProductRevision: rc=%d, Major=%d, Minor=%d\n", rc,
ProductRevisionMajor, ProductRevisionMinor);
  rc = vl53lx.VL53LX_GetDeviceInfo(&devInfo);
  Serial.printf("GetDeviceInfo: rc=%d, ModuleType(0xAA?)=0x%X, ProdRevMajor=%d,
ProdRevMinor=%d\n", rc, devInfo.ProductType, devInfo.ProductRevisionMajor,
devInfo.ProductRevisionMinor);
  rc = vl53lx.VL53LX_GetUID(&Uid);
  Serial.printf("GetUID: rc=%d, UID=0x%11X\n", rc, Uid);

  // To enable using the below I2CRead function, you need to move its definition in
vl53lx_class.h from the Protected section to the Public section
  rc = vl53lx.VL53LX_I2CRead(DEVICE_I2C_ADDR, 0x010F, &byteData, 1);
```

```

Serial.printf("I2CRead 0x010F: rc=%d, Model_ID=0x%02X\n", rc, byteData);
rc = vl53lx.VL53LX_I2CRead(DEVICE_I2C_ADDR, 0x0110, &byteData, 1);
Serial.printf("I2CRead 0x0110: rc=%d, Module_Type=0x%02X\n", rc, byteData);

pinMode(INT, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(INT), intaISR, FALLING);
intervalTimer = millis(); // Set timer

vl53lx.VL53LX_ClearInterruptAndStartMeasurement();
Serial.println("Ready...");
}

void debug (void) {

    Serial.println(); Serial.print ("Total Interrupts: "); Serial.println
(totalInterruptCounter);
    Serial.print ("Etapa: "); Serial.println (etapa);
    Serial.print ("Paredes: "); Serial.print (paredeEsq); Serial.print (paredeFrente);
Serial.println (paredeDir);
    Serial.print ("Distancia: "); Serial.println(distanciaFrente);
    Serial.print ("Posicao: ( "); Serial.print (posicaoX); Serial.print (" : ");
Serial.print(posicaoY); Serial.println(" )");
    Serial.print ("Orientacao: "); Serial.println(orientacao); Serial.println();
    Serial.print ("Vel RAW: "); Serial.print (vRead1); Serial.print (" ");
Serial.println(vRead2);
    Serial.print ("Velocidade: "); Serial.print (V1[0]); Serial.print (" ");
Serial.println(V2[0]);
    Serial.print ("vRef_Target: "); Serial.print (PWM[(int)m_target_speed]);
Serial.print (" V taget: "); Serial.println (m_target_speed);
    Serial.print ("vRef: "); Serial.print (PWM[(int)m_speed]); Serial.print (" V taget:
"); Serial.println (m_speed);
    Serial.print ("eP: "); Serial.print (erro1); Serial.print (" ");
Serial.println(erro2);
    Serial.print ("eD: "); Serial.print (erro1-erro10ld); Serial.print (" ");
Serial.println(erro2-erro20ld);
    Serial.print ("eRP: "); Serial.print (erroEsqP); Serial.print (" ");
Serial.println(erroDirP);
    Serial.print ("eRD: "); Serial.print (erroEsqD); Serial.print (" ");
Serial.println(erroDirD);
    Serial.print ("StrAdjst: "); Serial.println(STEERING_ADJUST);
    Serial.print ("PWM: "); Serial.print (vOut1); Serial.print (" ");
Serial.println(vOut2);

    /*
    for (int i = 10 ; i > 0; i--) {
        Serial.print ("v1["); Serial.print (i); Serial.print ("]="); Serial.print
(v1[i]); Serial.print (" ");
    }
    Serial.println();
    for (int i = 10 ; i > 0; i--) {
        Serial.print ("v2["); Serial.print (i); Serial.print ("]="); Serial.print
(v2[i]);Serial.print (" ");
    }
    Serial.println();
*/
    if (Serial.available() > 0)
        etapa = 2;
}

```

```
// ++++++ CALIBRAR MOTORES
+++++

/* A função systemModel() é executada pelo loop() com uma periodicidade igual ao ciclo
do timer
 * Aplica nas saídas que controlam os motores um setPoint dado por vOut
 * Guarda num vetor o valor dos pulsos de encoder entre cada interrupção do timer, que
podem depois ser convertidos em velocidade angular e linear
 * Depois de recolher os 200 valores que compoem a amostra, pára os motores, e inicia
uma contagem decrescente de 30 segundos
 * Imprime de seguida para o monitor serie os valores recolhidos formatados com
tabulação para poder colar diretamente na folha de Excel
 */

void systemModel (void) {
  char buffer[30];
  int i;
  i = totalInterruptCounter;

  if ( i <= amostra) {
    reads[i][1] = vRead1;
    reads[i][2] = vRead2;
  }
  else {
    move_Motors(0);
    etapa = -1;
    for (i = 20 ; i > 0; i--) {
      delay(1000);
      Serial.println(i);
    }
  }

  if (etapa == 0) {
    move_Motors(1);
  }
  else {
    Serial.print(" Time Step = "); Serial.print(cicle / 1000); Serial.print(" ms ");
    Serial.print(" Input Step = "); Serial.print(vOut); Serial.println(" PWM ");
    Serial.println();
    for (i = 0; i <= amostra; i++) {
      sprintf(buffer, "%d \t %d \t %d ", i, reads[i][1], reads[i][2]);
      Serial.println(buffer);
      delay(10);
    }
  }
}

/* monit_motors() permite controlar o setPoint de velocidade dos motores
 * monitorizado as leituras, e o erro entre a referencia e o feedback
 */
void monit_motors(void) {

  move_Motors(1);

  Serial.println(); Serial.println(totalInterruptCounter);
  Serial.print("R1 "); Serial.println(vRead1);
  Serial.print("R2 "); Serial.println(vRead2);
  Serial.print("V1 "); Serial.println(vOut1);
  Serial.print("V2 "); Serial.println(vOut2);
  Serial.print("erro1 "); Serial.println(erro1);
  Serial.print("erro2 "); Serial.println(erro2);
  Serial.print("erro1' "); Serial.println(erro1 - erro1old);
}
```

```

Serial.print("erro2' "); Serial.println(erro2 - erro20ld);

if (Serial.available() > 0) {
    valor = Serial.read();
    if ( valor != 13 && valor != 10) {
        consolaAux = consolaAux * 10 + (valor - 48);
    }
    else if ( valor == 13) {
        if (consolaAux < 10)
            etapa = consolaAux;
        else
            vOut = consolaAux;
        consolaAux = 0;
    }
}
}
/*Escrever um byte de informação na memória EEPROM*/
/*
#include <EEPROM.h>           //Incluimos a livreria EEPROM
int Direção = 0;             //Direção da memória EEPROM onde vamos armazenar o byte
byte Informação = B01011;    //Valor do byte que vamos a armazenar (É o número decimal
11 convertido a formato byte)
byte ValorEEPROM;            //Variável de tipo byte para ler os valores da memória
EEPROM

EEPROM.write(Direção, Informação);    //Escrevemos na direção 0 o byte B01011

ValorEEPROM = EEPROM.read(Direção);    //Lê-mos a direção 0 de la EEPROM
*/

// ++++++ CALIBRAR SENSORES
++++++

void monit_sensor(void) {

    IRread();
    /*
    Serial.println(totalInterruptCounter);
    Serial.print("S1 "); Serial.print( SensorDark[1]); Serial.print(" "); Serial.print(
SensorLight[1]); Serial.print(" "); Serial.println( SensorRead[1]);
    Serial.print("S2 "); Serial.print( SensorDark[2]); Serial.print(" "); Serial.print(
SensorLight[2]); Serial.print(" "); Serial.println( SensorRead[2]);
    Serial.print("S3 "); Serial.print( SensorDark[3]); Serial.print(" "); Serial.print(
SensorLight[3]); Serial.print(" "); Serial.println( SensorRead[3]);
    Serial.print("S4 "); Serial.print( SensorDark[4]); Serial.print(" "); Serial.print(
SensorLight[4]); Serial.print(" "); Serial.println( SensorRead[4]);
    Serial.println();
    */
    Serial.print( SensorRead[1]); Serial.print("\t");Serial.print( SensorRead[2]);
    Serial.print("\t");Serial.print( SensorRead[3]); Serial.print("\t"); Serial.println(
SensorRead[4]);

    if (Serial.available() > 0)
        etapa = 2;
    /*
        SensorDark[1]=analogRead(FT1);
        SensorDark[2]=analogRead(FT2);
        SensorDark[3]=analogRead(FT3);
        SensorDark[4]=analogRead(FT4);

        Serial.println(totalInterruptCounter);

```

```
Serial.print("S1 "); Serial.println( SensorDark[1]);
Serial.print("S2 "); Serial.println( SensorDark[2]);
Serial.print("S3 "); Serial.println( SensorDark[3]);
Serial.print("S4 "); Serial.println( SensorDark[4]);
Serial.println();

    char buffer[40];
    sprintf(buffer, "Orientacao: %c", orientacao);
    Serial.println(buffer);
*/
}

void search_run (void) {

    int passo = detetarParedes();

    //////////////////////////////////////////// AVANÇAR E DETETAR PAREDES
    if (etapa == 1) {
        //calcularDistancia();
        //atualizarPosicao();
        etapa = passo;
        avance( 0 );
    }
    //////////////////////////////////////////// FINALIZAR CURVAS
    if (etapa == 2) {
        avance( 1 );
    }
    //////////////////////////////////////////// CURVAR A ESQUERDA
    if (etapa == 3) {
        turn(-1,90);
    }
    //////////////////////////////////////////// CURVAR A DIREITA
    if (etapa == 4) {
        turn(1,90);
    }
    //////////////////////////////////////////// CURVA -180º
    if (etapa == 5) {
        invert(-2);
    }
    //////////////////////////////////////////// CURVA +180º
    if (etapa == 6) {
        invert(2);
    }
    ////////////////////////////////////////////
    if (etapa == 10) {
        avance( 50 );
    }
    //////////////////////////////////////////// MULTIPLAS DIREÇÕES POSSIVEIS
    if (etapa > 10)
        etapa = 1;
}

void run_maze (void){

}

void speed_run (void){

}

void end_run (void){

}
```

```

void calcularDistancia (void) {

    distanciaFrente = lidarRead();

}

int detetarParedes (void) {
    IRread();
    if (Sensor[2][0] > LEFT_THRESHOLD)
        paredeEsq = false;
    else
        paredeEsq = true;
    if (Sensor[3][0] > RIGHT_THRESHOLD)
        paredeDir = false;
    else
        paredeDir = true;
    if (Sensor[1][0] > FRONT_L_THRESHOLD and Sensor[4][0] > FRONT_R_THRESHOLD)
        paredeFrente = false;
    else
        paredeFrente = true;

    if (!paredeEsq && !paredeFrente && !paredeDir)    //000
        return 20;
    if (!paredeEsq && !paredeFrente && paredeDir)    //001
        return 20;
    if (!paredeEsq && paredeFrente && !paredeDir)    //010
        return 20;
    if (!paredeEsq && paredeFrente && paredeDir)    //011
        return 3;
    if (paredeEsq && !paredeFrente && !paredeDir)    //100
        return 20;
    if (paredeEsq && !paredeFrente && paredeDir)    //101
        return 1;
    if (paredeEsq && paredeFrente && !paredeDir)    //110
        return 4;
    if (paredeEsq && paredeFrente && paredeDir)    //111
        if (Sensor[2][0] > Sensor[3][0])    // se estiver mais perto da parede
            esquerda curva para a direita
                return 5;
            else
                return 6;
}

void IRread (void) {

    digitalWrite(LED1, LOW);
    digitalWrite(LED2, LOW);
    digitalWrite(LED3, LOW);
    digitalWrite(LED4, LOW);

    SensorDark[1] = analogRead(FT1);
    digitalWrite(LED1, HIGH);
    delayMicroseconds(100);
    SensorLight[1] = analogRead(FT1);
    digitalWrite(LED1, LOW);

    SensorDark[2] = analogRead(FT2);
    digitalWrite(LED2, HIGH);
    delayMicroseconds(100);
    SensorLight[2] = analogRead(FT2);
    digitalWrite(LED2, LOW);
}

```

```
SensorDark[3] = analogRead(FT3);
digitalWrite(LED3, HIGH);
delayMicroseconds(100);
SensorLight[3] = analogRead(FT3);
digitalWrite(LED3, LOW);

SensorDark[4] = analogRead(FT4);
digitalWrite(LED4, HIGH);
delayMicroseconds(100);
SensorLight[4] = analogRead(FT4);
digitalWrite(LED4, LOW);

for (int n = 1; n < 5; n++) {

    for (int i = 10; i < 0 ; i--)
        Sensor[n][i] = Sensor[n][i-1];

    SensorOld[n] = SensorRead[n];
    SensorRead[n] = SensorLight[n] - (4095 - SensorDark[n]);
    Sensor[n][0] = pesoSensorAtual * SensorRead[n] + pesoSensorOld * SensorOld[n];
}
}

int lidarRead (void)
{
    VL53LX_MultiRangingData_t MultiRangingData;
    VL53LX_MultiRangingData_t *rangingData = &MultiRangingData;
    uint8_t NewDataReady = 0;
    int objectsFound = 0, j, status;
    bool dataChanged = false;

    if (intaINT) {

        status = vl53lx.VL53LX_GetMeasurementDataReady(&NewDataReady);
        if (status == 0 && NewDataReady != 0) {

            status = vl53lx.VL53LX_GetMultiRangingData(rangingData);
            objectsFound = rangingData->NumberOfObjectsFound;
            // Serial.print("objectsFound"); Serial.println(objectsFound);
            if (objectsFound > 0) {
                for (j = 0; j < objectsFound; j++) {
                    // Check only if status is OK
                    if (rangingData->RangeData[j].RangeStatus == 0) {
                        // Only consider changes of over 10 mm
                        //if (abs(rangingData->RangeData[j].RangeMilliMeter - dist[j]) > 10) {
                        // Movement happened
                        dist[j] = rangingData->RangeData[j].RangeMilliMeter;
                        dataChanged = true;
                        // }
                    }
                }
                // Clear rest of values
                for (j = objectsFound; j < MAX_OBJECTS; j++)
                    dist[j] = 0;

                if (dataChanged) {
                    dataChanged = false;
                    //Serial.printf(" %d %d %d %d\n", dist[0], dist[1], dist[2], dist[3]);
                }
            }
        }
    }
}
```

```

    intaINT = false;
    vl53lx.VL53LX_ClearInterruptAndStartMeasurement();
    intervalTimer = millis();

} else {

    // Did we lose an interrupt?

    if ((millis() - intervalTimer) > 2000) {

        //      Serial.println("Lost interrupt?");
        vl53lx.VL53LX_ClearInterruptAndStartMeasurement();
        intervalTimer = millis();
    }
}
return dist[0];
}

// ++++++ FUNÇÕES MOVIMENTO MicroMouse ++++++
+++++

void calcularAnguloParedes (void) {

    erroEsqP = LEFT_CALIBRATION - Sensor[2][0];
    erroDirP = RIGHT_CALIBRATION - Sensor[3][0];

    erroEsqD = (Sensor[2][10] - Sensor[2][0]) / (V1[0]*LOOP_INTERVAL);
    erroDirD = (Sensor[3][10] - Sensor[3][0]) / (V2[0]*LOOP_INTERVAL);

    if (erroEsqP > 1000)
        erroEsqP = 1000;
    if (erroEsqP < -1000)
        erroEsqP = -1000;
    if (erroEsqD > 1000)
        erroEsqD = 1000;
    if (erroEsqD < -1000)
        erroEsqD = -1000;

    if (erroDirP > 1000)
        erroDirP = 1000;
    if (erroDirP < -1000)
        erroDirP = -1000;
    if (erroDirD > 1000)
        erroDirD = 1000;
    if (erroDirD < -1000)
        erroDirD = -1000;

    STEERING_ADJUST = STEERING_KP * erroEsqP + STEERING_KD * erroEsqD - STEERING_KP *
erroDirP - STEERING_KD * erroDirD;

    if (STEERING_ADJUST > STEERING_ADJUST_LIMIT)
        STEERING_ADJUST = STEERING_ADJUST_LIMIT;

    if (STEERING_ADJUST < - STEERING_ADJUST_LIMIT)
        STEERING_ADJUST = - STEERING_ADJUST_LIMIT;

}

void calcularVelocidade(void) {
    /*
    for (int i = 10; i < 0 ; i--) {

```

```
V1[i] = V1[i-1];
V2[i] = V2[i-1];
}
*/
V1[0] = (pesoEncoderAtual * vRead1 + pesoEncoderOld * vRead1Old) *
MILIMETERS_OVER_PULSES * LOOP_FREQUENCY;
V2[0] = (pesoEncoderAtual * vRead2 + pesoEncoderOld * vRead2Old) *
MILIMETERS_OVER_PULSES * LOOP_FREQUENCY;
}

float get_braking_distance() {
    return fabsf(m_speed * m_speed - m_final_speed * m_final_speed) * 0.5 *
m_one_over_acc;
}

void controlarAcelaracao ( float top_speed, float final_speed, float acceleration ) {

    float delta_v = acceleration * LOOP_INTERVAL;
    m_final_speed = final_speed;

    if (m_speed < top_speed) {
        if (remaining * MILIMETERS_OVER_PULSES < get_braking_distance()) {
            if (m_final_speed < 100) {
                m_target_speed = 100;
            } else {
                m_target_speed = m_final_speed;
            }
        }
        else
            m_target_speed = top_speed;
    }
    // try to reach the target speed
    if (m_speed < m_target_speed) {
        m_speed += delta_v;
        if (m_speed > m_target_speed) {
            m_speed = m_target_speed;
        }
    }
    if (m_speed > m_target_speed) {
        m_speed -= delta_v;
        if (m_speed < m_target_speed) {
            m_speed = m_target_speed;
        }
    }
}

void controlarCurva ( float pulsosLeft, float targetSpeed) {

    erroOld = erro;
    erro = pulsosLeft;
    erroAcumulado = erroAcumulado + erro;

    // PID Controler

    vOut = ROT_KP * erro + ROT_KD * (erro-erroOld) + ROT_KI * (erroAcumulado) ;

    if ( vOut > PWM[(int)targetSpeed] ) {
        calcularVelocidade();

        erro1Old = erro1;
        erro1 = targetSpeed - V1[0];
        erro2Old = erro2;
```

```

    erro2 = targetSpeed - V2[0];

    vOut1 = PWM[(int)targetSpeed] + FWD_KP * erro1 + FWD_KD * (erro1-erro10ld) ;
    vOut2 = PWM[(int)targetSpeed] + FWD_KP * erro2 + FWD_KD * (erro2-erro20ld) ;

    // Proteção valores invalidos
    if (vOut1 > vMax)
        vOut1 = vMax;
    if (vOut1 < vMin)
        vOut1 = vMin;
    if (vOut2 > vMax)
        vOut2 = vMax;
    if (vOut2 < vMin)
        vOut2 = vMin;
}
else {
// Proteção valores invalidos
    if (vOut > vMax)
        vOut = vMax;
    if (vOut < vMin)
        vOut = vMin;

    vOut1 = vOut;
    vOut2 = vOut;
}
}

void controlarVelocidade(float targetSpeed) {

    calcularVelocidade();
    calcularAnguloParedes();

    float targetSpeed1 = targetSpeed + STEERING_ADJUST;
    float targetSpeed2 = targetSpeed - STEERING_ADJUST;

    erro10ld = erro1;
    erro1 = targetSpeed1 - V1[0];
    //erro1Acumulado = erro1Acumulado + erro1;
    //vOut1OLD = vOut1;

    erro20ld = erro2;
    erro2 = targetSpeed2 - V2[0];
    //erro2Acumulado = erro2Acumulado + erro2;
    //vOut2OLD = vOut2;

    // PD Controler

    vOut1 = PWM[(int)targetSpeed1] + FWD_KP * erro1 + FWD_KD * (erro1-erro10ld) ;
    vOut2 = PWM[(int)targetSpeed2] + FWD_KP * erro2 + FWD_KD * (erro2-erro20ld) ;

    // PID Controler
    /*
        vOut1 = targetSpeed + (kP1 * erro1 + kI1*erro1Acumulado + kD1 * (erro1 -
    erro10ld)) / 256;
        vOut2 = targetSpeed + (kP2 * erro2 + kI2*erro2Acumulado + kD2 * (erro2 -
    erro20ld)) / 256;

    // Lag Lead Compensator

    vOut1 = (K1 * erro1 - K2 * erro10ld + K3 * vOut1OLD + 128) / 256;
    vOut2 = (K1 * erro2 - K2 * erro20ld + K3 * vOut2OLD + 128) / 256;
    */
}

```

```
// Proteção valores invalidos
if (vOut1 > vMax)
    vOut1 = vMax;
if (vOut1 < vMin)
    vOut1 = vMin;
if (vOut2 > vMax)
    vOut2 = vMax;
if (vOut2 < vMin)
    vOut2 = vMin;

}

void avance (float distance ) {
    if (distance > 0) {
        if (doneF == false) {                // executar apenas uma vez
            pulsosD1 = 0; pulsosD2 = 0;
            pulsos = distance / MILIMETERS_OVER_PULSES;    // pulsos de encoder
correspondestes
            remaining = pulsos;
            zerarErro();
            doneF = true;
        }
        if (remaining > 0) {
            remaining = 2 * pulsos - (pulsosD1 + pulsosD2);
        }
        else {
            calcPosicao();                // apos avançar uma celula
            doneF = false;
            etapa = 1;
        }
    }
    else {
        remaining = distanciaFrente;
    }
    controlarAcelaracao(DEFAULT_SEARCH_SPEED, DEFAULT_SEARCH_SPEED/2,
DEFAULT_SEARCH_ACCEL);
    controlarVelocidade(m_speed);
    move_Motors(1);
}

void turn(int sentido, float angle) {

    if (doneT == false) {                // executar apenas uma vez
        pulsosD1 = 0; pulsosD2 = 0;        // zerar as variaveis incrementadas
pelos pulsos de enconder
        pulsos = PULSES_OVER_DEGREES * angle ;    // pulsos de encoder correspondestes
a um arco de 90º com 75mm de raio
        remaining = pulsos;                // erro inicial = pulsos totais
correspondentes ao arco a descrever
        zerarErro();
        doneT = true;
    }

    if (remaining > 0 and paredeFrente == true) {    // caso nao tenha completado
a rotaçao
        controlarCurva ( remaining, DEFAULT_TURN_SPEED );
        if (sentido > 0) {                // curva a direita ou a
esquerda
            move_Motors(2);
            remaining = pulsos - pulsosD1 + pulsosD2;    // atualiza o erro
        }
    }
}
```

```

        else {
            remaining = pulsos - pulsosD2 + pulsosD1;           // atualiza o erro
            move_Motors(3);
        }
    }
    else {
        zerarErro();                                           // quando termina a curva
        calcOrientacao(sentido);                             // zerar erros acumulados
        pulsosT = 0;                                          // atualiza a orientacao no tracking
        de posicao                                             // inicia a contagem para atualização
        etapa = 2;                                           // avança uma celula antes de voltar a
        detetar paredes
        doneT = false;                                       // habilita um novo calculo de
        trajetoria
    }
}

void invert (int sentido) {

    if (done == false) {
        pulsosD1 = 0; pulsosD2 = 0;
        pulsos = PULSES_OVER_DEGREES * 180;
        remaining = pulsos - (pulsosD1 + pulsosD2);
        zerarErro();
        done = true;
    }
    controlarCurva ( remaining, DEFAULT_TURN_SPEED );
    if (paredeFrente == false)
        remaining = 0;
    else
        remaining = pulsos - (pulsosD1 + pulsosD2);
    if (remaining > 0) {
        if (sentido > 0)
            move_Motors(6);
        else
            move_Motors(7);
    }
    else {
        zerarErro();
        calcOrientacao(sentido);
        pulsosT = 0;
        etapa = 2;
        done = false;
    }
}

void zerarErro (void) {
    erro = 0; erro1 = 0; erro2 = 0;
    erroAcumulado = 0; erro1Acumulado = 0; erro2Acumulado = 0;
    erroOld = 0; erro1Old = 0; erro2Old = 0;
    vOut1OLD = 0; vOut2OLD = 0;
    erroEsqP = 0; erroEsqD = 0; erroDirP = 0; erroDirD = 0;
}

oid move_Motors(int direcao) {

    if (direcao == -1) // TRAS
    {
        digitalWrite(EN1, HIGH);
        digitalWrite(EN2, HIGH);
        digitalWrite(PH1, HIGH);
        digitalWrite(PH2, HIGH);
    }
}

```

```
    ledcWrite(PWM1, vOut1);
    ledcWrite(PWM2, vOut2);
}

if (direcao == 0) // PARADO
{
    digitalWrite(EN1, LOW);
    digitalWrite(EN1, LOW);
    ledcWrite(PWM1, 0);
    ledcWrite(PWM2, 0);
}

if (direcao == 1) // FRENTE
{
    digitalWrite(EN1, HIGH);
    digitalWrite(EN2, HIGH);
    digitalWrite(PH1, LOW);
    digitalWrite(PH2, LOW);
    ledcWrite(PWM1, vOut1);
    ledcWrite(PWM2, vOut2);
}

if (direcao == 2) // CURVA LENTA DIREITA
{
    digitalWrite(EN1, HIGH);
    digitalWrite(EN2, LOW);
    digitalWrite(PH1, LOW);
    digitalWrite(PH2, LOW);
    ledcWrite(PWM1, vOut1);
    ledcWrite(PWM2, 0);
}

if (direcao == 3) // CURVA LENTA ESQUERDA
{
    digitalWrite(EN1, LOW);
    digitalWrite(EN2, HIGH);
    digitalWrite(PH1, LOW);
    digitalWrite(PH2, LOW);
    ledcWrite(PWM1, 0);
    ledcWrite(PWM2, vOut2);
}

if (direcao == 4) // CURVA RECUADA DIREITA
{
    digitalWrite(EN1, LOW);
    digitalWrite(EN2, HIGH);
    digitalWrite(PH1, LOW);
    digitalWrite(PH2, HIGH);
    ledcWrite(PWM1, 0);
    ledcWrite(PWM2, vOut2);
}

if (direcao == 5) // CURVA RECUADA ESQUERDA
{
    digitalWrite(EN1, HIGH);
    digitalWrite(EN2, LOW);
    digitalWrite(PH1, HIGH);
    digitalWrite(PH2, LOW);
    ledcWrite(PWM1, vOut1);
    ledcWrite(PWM2, 0);
}
```

```

if (direcao == 6) // CURVA RAPIDA 180 DIREITA
{
    digitalWrite(EN1, HIGH);
    digitalWrite(EN2, HIGH);
    digitalWrite(PH1, LOW);
    digitalWrite(PH2, HIGH);
    ledcWrite(PWM1, vOut1);
    ledcWrite(PWM2, vOut2);
}

if (direcao == 7) // CURVA RAPIDA 180 ESQUERDA
{
    digitalWrite(EN1, HIGH);
    digitalWrite(EN2, HIGH);
    digitalWrite(PH1, HIGH);
    digitalWrite(PH2, LOW);
    ledcWrite(PWM1, vOut1);
    ledcWrite(PWM2, vOut2);
}
}

void atualizarPosicao (void) {
    if (pulsosT > 320 / MILIMETERS_OVER_PULSES) { // 160 mm x 2 pulsos por encoder
        calcPosicao();
        distTotal = distTotal + (pulsosT) / 2;
        pulsosT = 0;
    }
}

void calcPosicao (void) {
    if (orientacao == 'N') {
        posicaoY++;
    }
    if (orientacao == 'S') {
        posicaoY--;
    }
    if (orientacao == 'E') {
        posicaoX++;
    }
    if (orientacao == 'O') {
        posicaoX--;
    }
}

void calcOrientacao (int rotacao) {
    if (orientacao == 'N'); {
        if (rotacao == -2 || rotacao == 2)
            orientacao = 'S';
        else if (rotacao == -1)
            orientacao = 'O';
        else if (rotacao == 1)
            orientacao = 'E';
    }
    if (orientacao == 'S'); {
        if (rotacao == -2 || rotacao == 2)
            orientacao = 'N';
        else if (rotacao == -1)
            orientacao = 'E';
        else if (rotacao == 1)
            orientacao = 'O';
    }
    if (orientacao == 'E'); {
        if (rotacao == -2 || rotacao == 2)
            orientacao = 'O';
    }
}

```

```

    else if (rotacao == -1)
        orientacao = 'N';
    else if (rotacao == 1)
        orientacao = 'S';
}
if (orientacao == 'O'); {
    if (rotacao == -2 || rotacao == 2)
        orientacao = 'E';
    else if (rotacao == -1)
        orientacao = 'S';
    else if (rotacao == 1)
        orientacao = 'N';
}
}
}

```

//////////////////////////////////// TABELA DE CONVERSAO DA FUNÇÃO  
TRANSFERENCIA H(s)////////////////////////////////////

/\*

\* Obtida atraves da Step Response do sistema a diferentes Set Points, contruindo um  
grafico em Excel e fazendo uma regressao Polinomial de 6º grau

\*/

```

const int PWM[1024] PROGMEM = {
126, 126, 126, 126, 126, 127, 127, 127, 127, 127, 127, 127, 128, 128, 128, 128, 128, 128, 128, 129, 129, 129,
129, 129, 129, 130, 130, 130, 130, 131, 131, 131, 131, 131, 131, 132, 132, 132, 132, 132, 133, 133, 133,
133, 133, 134, 134, 134, 134, 134, 135, 135, 135, 135, 135, 136, 136, 136, 136, 136, 137, 137, 137, 137,
138, 138, 138, 138, 139, 139, 139, 139, 139, 140, 140, 140, 140, 141, 141, 141, 141, 141, 142, 142, 142, 142,
143, 143, 143, 143, 144, 144, 144, 144, 145, 145, 145, 145, 145, 146, 146, 146, 146, 147, 147, 147, 147, 148,
148, 148, 148, 149, 149, 149, 149, 150, 150, 150, 150, 151, 151, 151, 151, 152, 152, 152, 152, 153, 153, 153,
153, 154, 154, 154, 155, 155, 155, 155, 156, 156, 157, 157, 157, 157, 158, 158, 158, 159, 159, 159,
159, 159, 160, 160, 160, 160, 161, 161, 161, 162, 162, 162, 162, 163, 163, 163, 163, 164, 164, 164, 165,
165, 165, 165, 166, 166, 166, 166, 167, 167, 167, 167, 168, 168, 168, 169, 169, 169, 169, 170, 170, 170, 170,
171, 171, 171, 171, 172, 172, 172, 173, 173, 173, 173, 174, 174, 174, 174, 175, 175, 175, 176, 176, 176, 176,
177, 177, 177, 177, 178, 178, 178, 179, 179, 179, 179, 180, 180, 180, 180, 181, 181, 181, 182, 182, 182, 182,
183, 183, 183, 183, 184, 184, 184, 185, 185, 185, 185, 186, 186, 186, 187, 187, 187, 187, 188, 188, 188, 189,
189, 189, 189, 190, 190, 190, 190, 191, 191, 191, 192, 192, 192, 192, 193, 193, 193, 194, 194, 194, 194, 195,
195, 195, 196, 196, 196, 196, 197, 197, 197, 198, 198, 198, 199, 199, 199, 199, 200, 200, 200, 201, 201, 201,
201, 202, 202, 202, 203, 203, 203, 204, 204, 204, 204, 205, 205, 205, 206, 206, 206, 207, 207, 207, 208,
208, 208, 209, 209, 209, 210, 210, 210, 211, 211, 211, 212, 212, 212, 213, 213, 213, 214, 214, 214, 215,
215, 215, 216, 216, 216, 217, 217, 217, 218, 218, 218, 219, 219, 219, 220, 220, 220, 221, 221, 221, 222, 222,
222, 223, 223, 223, 224, 224, 224, 225, 225, 225, 226, 226, 226, 227, 227, 227, 228, 228, 228, 229, 229, 230,
230, 230, 231, 231, 231, 232, 232, 232, 233, 233, 234, 234, 234, 235, 235, 235, 236, 236, 237, 237, 237, 238,
238, 238, 239, 239, 240, 240, 240, 241, 241, 242, 242, 243, 243, 244, 244, 245, 245, 246, 246, 246,
247, 247, 248, 248, 248, 249, 249, 250, 250, 251, 251, 252, 252, 253, 253, 254, 254, 255, 255, 256,
256, 257, 257, 258, 258, 259, 259, 260, 260, 261, 261, 262, 262, 263, 263, 264, 264, 265, 265, 266,
266, 267, 267, 268, 268, 269, 269, 270, 270, 271, 271, 272, 272, 273, 273, 274, 274, 275, 275, 276, 276, 277,
277, 278, 278, 279, 280, 280, 281, 281, 282, 282, 283, 283, 284, 284, 285, 286, 286, 287, 287, 288, 288, 289,
289, 290, 291, 291, 292, 292, 293, 294, 294, 295, 295, 296, 297, 297, 298, 298, 299, 300, 300, 301, 301, 302,
303, 303, 304, 305, 305, 306, 306, 307, 308, 308, 309, 310, 310, 311, 312, 312, 313, 314, 314, 315, 316, 316,
317, 318, 318, 319, 320, 321, 321, 322, 323, 323, 324, 325, 325, 326, 327, 328, 328, 329, 330, 331, 331, 332,
333, 334, 334, 335, 336, 337, 337, 338, 339, 340, 340, 341, 342, 343, 344, 344, 345, 346, 347, 348, 348, 349,
350, 351, 352, 353, 353, 354, 355, 356, 357, 358, 358, 359, 360, 361, 362, 363, 364, 364, 365, 366, 367, 368,
369, 370, 371, 372, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389,
389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411,
412, 413, 414, 415, 416, 417, 418, 420, 421, 422, 423, 424, 425, 426, 427, 429, 430, 431, 432, 433, 434, 435,
437, 438, 439, 440, 441, 442, 444, 445, 446, 447, 448, 450, 451, 452, 453, 455, 456, 457, 458, 460, 461, 462,
463, 465, 466, 467, 468, 470, 471, 472, 474, 475, 476, 478, 479, 480, 482, 483, 484, 486, 487, 488, 490, 491,
492, 494, 495, 497, 498, 499, 501, 502, 504, 505, 507, 508, 509, 511, 512, 514, 515, 517, 518, 520, 521, 523,
524, 526, 527, 529, 530, 532, 533, 535, 536, 538, 539, 541, 543, 544, 546, 547, 549, 551, 552, 554, 555, 557,
559, 560, 562, 563, 565, 567, 568, 570, 572, 573, 575, 577, 579, 580, 582, 584, 585, 587, 589, 591, 592, 594,
596, 598, 599, 601, 603, 605, 607, 608, 610, 612, 614, 616, 618, 619, 621, 623, 625, 627, 629, 631, 632, 634,
636, 638, 640, 642, 644, 646, 648, 650, 652, 654, 656, 658, 660, 662, 664, 666, 668, 670, 672, 674, 676, 678,
680, 682, 684, 686, 688, 690, 692, 695, 697, 699, 701, 703, 705, 707, 710, 712, 714, 716, 718, 721, 723, 725,
727, 729, 732, 734, 736, 738, 741, 743, 745, 748, 750, 752, 755, 757, 759, 762, 764, 766, 769, 771, 773, 776,
778, 781, 783, 785, 788, 790, 793, 795, 798, 800, 803, 805, 808, 810, 813, 815, 818, 820, 823, 825, 828, 831,
833, 836, 838, 841, 844, 846, 849, 851, 854, 857, 859, 862, 865, 868, 870, 873, 876, 878, 881, 884, 887, 889,
892, 895, 898, 901, 903, 906, 909, 912, 915, 918, 921, 924, 926, 929, 932, 935, 938, 941, 944, 947, 950, 953,
956, 959, 962, 965, 968, 971, 974, 977, 980, 983, 986, 990, 993, 996, 999, 1002, 1005, 1008, 1012, 1015,
1018, 1021, 1024 } ;

```